

Verdi用法小结

原创 芯片验证那些事儿 芯片验证日记 2023-02-06 00:00 发表于上海

收录于合集

#EDA 17 #Verdi 2

前言

Verdi是一个功能强大的debug工具，可以配合不同的仿真软件进行debug，很多企业常用VCS+Verdi或者Xcelium/xrun+Verdi的方式进行代码的仿真与检查。Verdi支持对所有设计和验证流程进行全面的调试。它能够帮助工程师理解复杂和不熟悉的设计行为，让困难和乏味的调试过程自动化，并能让多样和复杂的设计环境一致化。在芯片设计过程中，可以根据仿真结果检查或者理解设计；在芯片验证过程中，常常用来调试（debug）一些错误的情况。Verdi既可以阅读调试代码，又可以查看波形，包括数字设计的波形和模拟设计的波形。

本文内容标题截图

本文一共一万一千多字，上百张截图，基本涵盖了Verdi的日常用法和常规技巧，内容比较多我对标题做了如下的截图，方便大家阅读查找。整理的过程中参考了很多大佬的文章，文末也都一一列举出来了。由于时间仓促，整理的还不是太理想，还是希望能帮助到一些朋友吧。

标题 页面 结果

▲ Verdi用法总结

前言

本文内容标题截图

▲ 一、verdi使用教程

(一) Verdi环境配置

(二) VCS产生Verdi波形

▲ (三) Verdi基础使用

1 打开Verdi

▲ 2. nTrace使用

(1)查看包含的设计架构

(2)查寻模块实例化的位置

(3) nSchema使用

(4) nState使用

▲ (5) nWave使用

[1] 添加波形到nWave

[2] 查找某信号，添加到nWave中

[3] 给Group重命名，排序，以及调整信号位置

[4] 改变信号的颜色

[5] 查看信号及变化

[6] 进制之间的转换

[7] 符号数之间的转换

[8] 状态寄存器显示为状态名字

 芯片验证日记

[9] 通过移除关联, 从状态机名字恢复到原来的进制显示

[10] 通过逻辑操作创建新信号

[11] 如何添加marker

[12] 总线操作:

[13] 如何查找信号的某个值、某个值跳转到某个值

[14] 如何对比两个波形

[15] 显示信号全路径

[16] 对信号的简单操作

[17] 移动波形窗口信号的位置

[18] 放大缩小

[19] 显示状态机的跳转

[20] 参考信号数值

[21] 新建组并对组重命名

[22] Signal Event Report统计

[23] Waveform Compare比较波形

▲ (四) Verdi使用技巧

1、连续信号有效数量的测量

2、非连续有效信号量测方法

▲ 3、搜索查找

[1] 数值搜索、上升沿和下降沿搜索

[2] 层次浏览器窗格中寻找实例 (instance)

▲ [3] 搜索源代码

1) Find scope

2) Find Signal/Instance/instport

3) find string

[4] nWave里面的信号追溯源代码

4、总线拆分

5、统计

6、逻辑运算

7、manage.rc

8、Trace代码过程中波形窗口保持前置

9、Parameter 提示的默认进制

10、使用 Verdi 分析 NBA Delay 的问题

11、数字信号的模拟化呈现

12、手动调节模拟波形的显示范围

13、自动调节: 切换 FSDB 文件

14、自动调节: 加载特定时间范围

15、跳转到指定行号

16、源文件之间或不同逻辑层次的切换

▷ 17、计数

18 用verdi打开覆盖率文件查看覆盖率

19 查看包含的设计架构

20 查询模块实例化位置

21 在nTrace中如何查找 模块和信号



芯片验证日记



芯片验证日记

22 查找位于不同层次的驱动信号【跨层次查看信号，看最终驱动谁，或者看最终被谁驱动】

23 如何查看设计有哪些信号

24 如何打开原理图

25 如何查看nShema Window中符号对应的源代码

26 使用 Fan-In Cone追踪某个信号的驱动逻辑

27 如何产生 partial hierarchy schematic

28 如何移动原理图中的器件

▲ 29 状态机相关操作

[1] 打开原理图

[2] 双击相应状态机视图后，呈现状态机图

[3] 查看状态的执行和跳转

[4] 查看状态以及节点的相关源代码

[5] 查看在什么时候状态机转移到某个状态？《与tb有关》

[6] 查看某一种状态转移【状态循环】情况是否会发生

[7] 如何查看某个状态被执行了多少次（与TB有关）

▲ (五) Verdi波形(含emulation)加载

1 打开波形

2 打开 source code

3 加载设计

4 重新加载设计和波形

5 波形文件保存*.rc

(六) Verdi常用快捷键总结

(七) 赠送跟文相关的一些文档

参考文章

感谢关注微信公众号《芯片验证日记》



芯片验证日记



芯片验证日记

一、verdi使用教程

(一) Verdi环境配置

bashrc中配置

```
1 export Verdi_HOME=$Synopsys_Dir/Verdi2015
2 #export NOVAS_HOME=$Synopsys_Dir/Verdi2015
3 export PATH=$Verdi_HOME/bin:$PATH
4 export LD_LIBRARY_PATH="/opt/Synopsys/Verdi2015/share/PLI/lib/linux64"$PATH
5 export LD_LIBRARY_PATH="/opt/Synopsys/Verdi2015/share/PLI/VCS/linux64"$PATH
```

上面三个环境变量的解释：

VERDI_HOME/NOVAS_HOME：仿真器默认，且为设置PATH做准备

PATH: 让系统(Linux)找到verdi

LD_LIBRARY_PATH: 让系统(Linux)能够找到Verdi需要的库文件.so(静态库), .a(动态库), .tab(表格文件).

(二)VCS产生Verdi波形

[1]使用Verilog系统函数,在tb文件中加入相应的系统函数

```
1 initial begin
2   $fsdbDumpfile("fifo.fsdb");
3   $fsdbDumpvars(0);
4 end
```

[2] 使用UCLI、TCL接口(VCS使用tcl脚本)

```
1 global env
2 fsdbDumpfile "$env(demo_fifo).fsdb"
3 fsdbDumpvars 0 "top_tb"
4 run
```

(三) Verdi基础使用

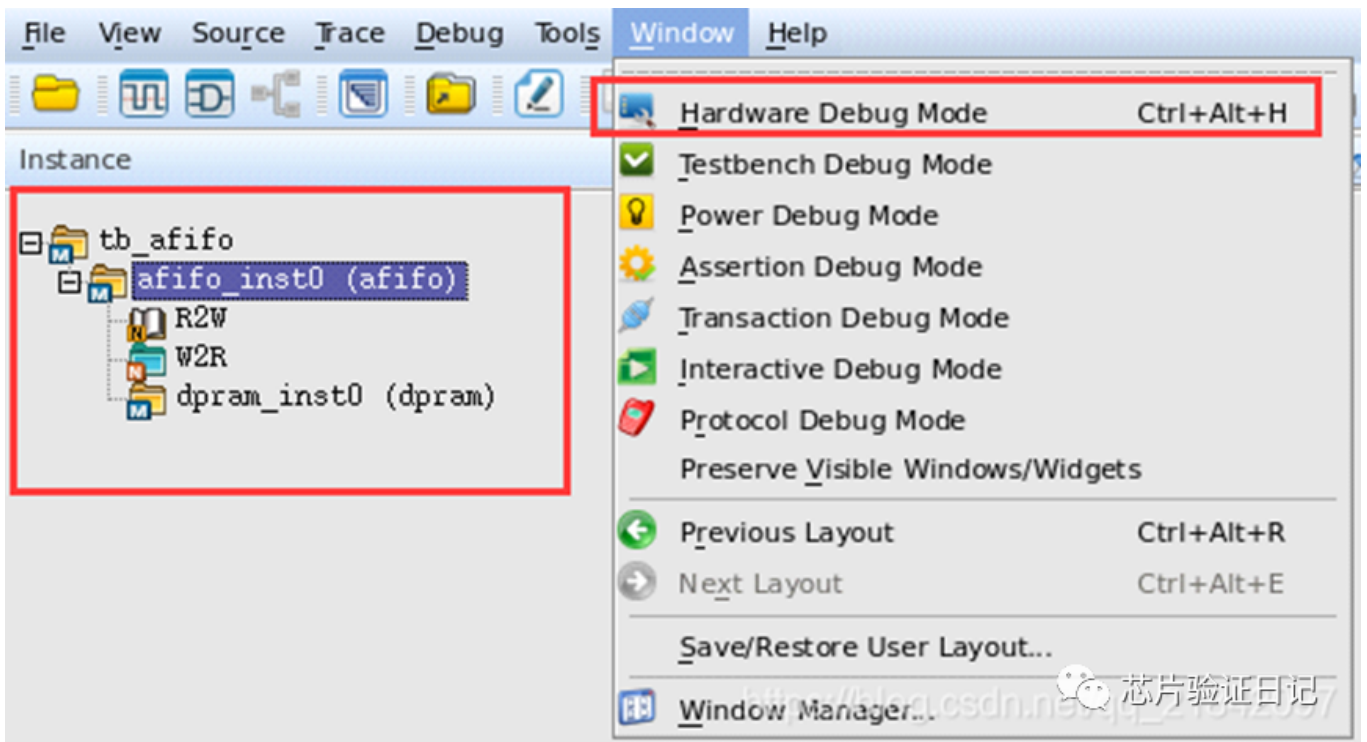
1 打开Verdi

(1)verdi -nologo

(2)makefile命令

2. nTrace使用

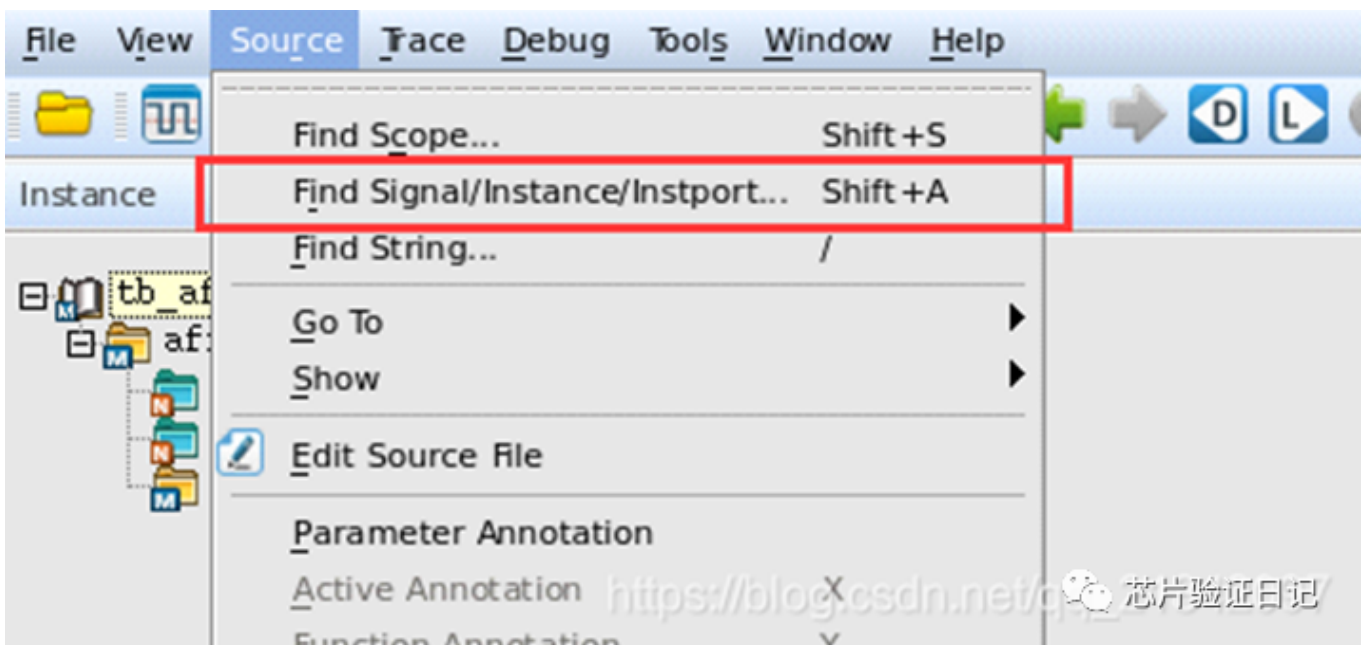
(1)查看包含的设计架构



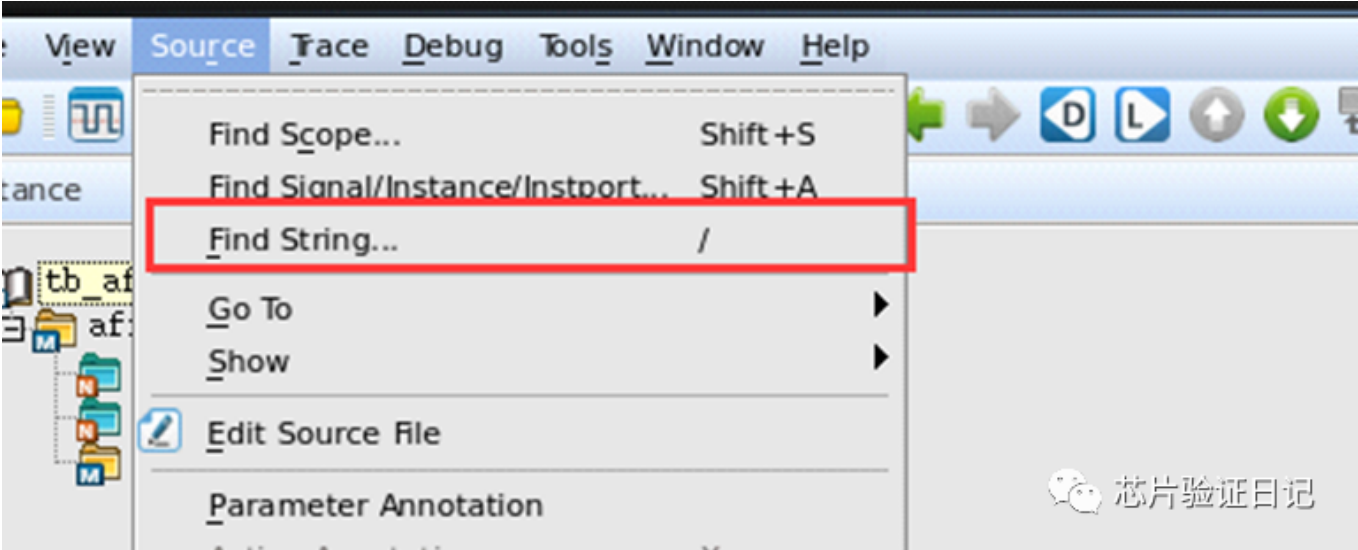
(2)查寻模块实例化的位置

- ①在设计结构中双击实例化模块的名字->源代码窗口高亮了模块名
- ②再双击高亮的模块名->看到在上层模块中的哪一行被调用了

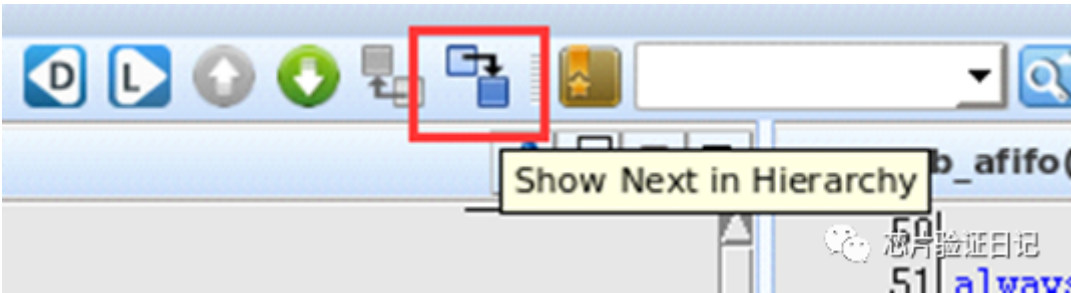
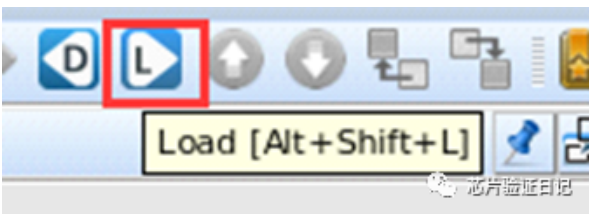
在nTrace中查找模块和信号



查寻字符串

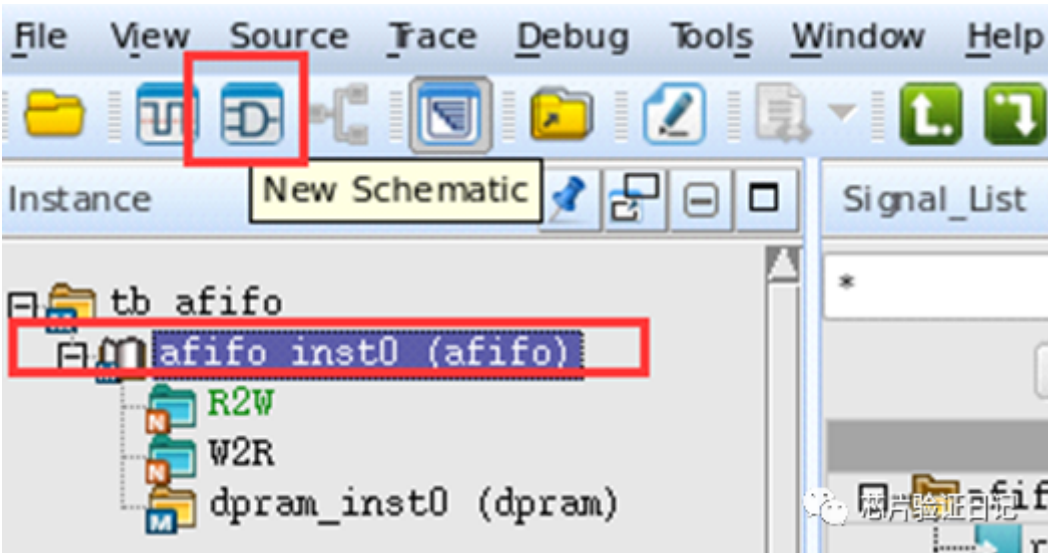


查看驱动信号相关

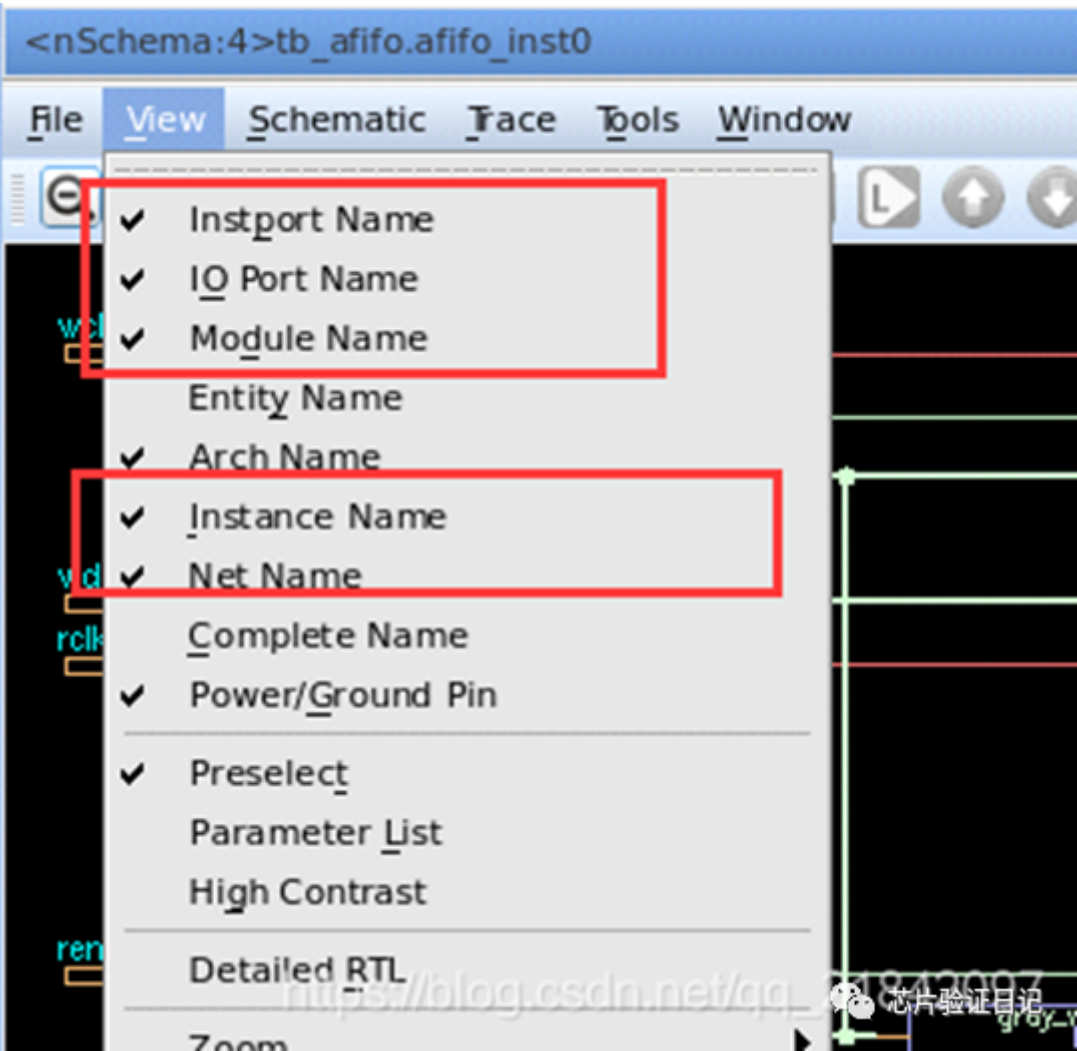


(3) nSchema使用

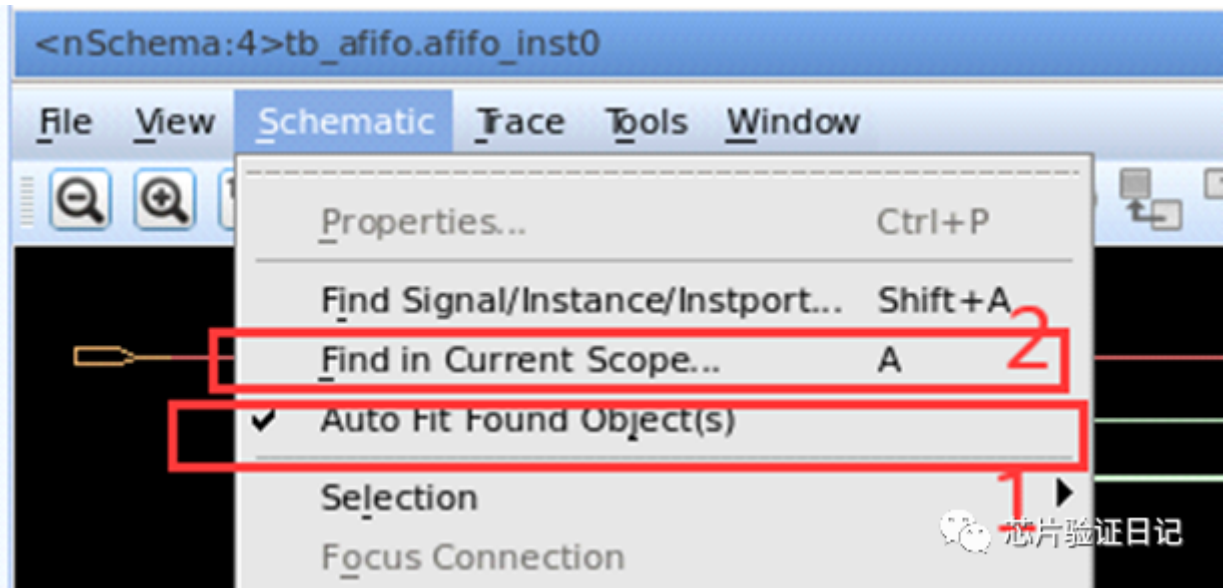
查看原理图

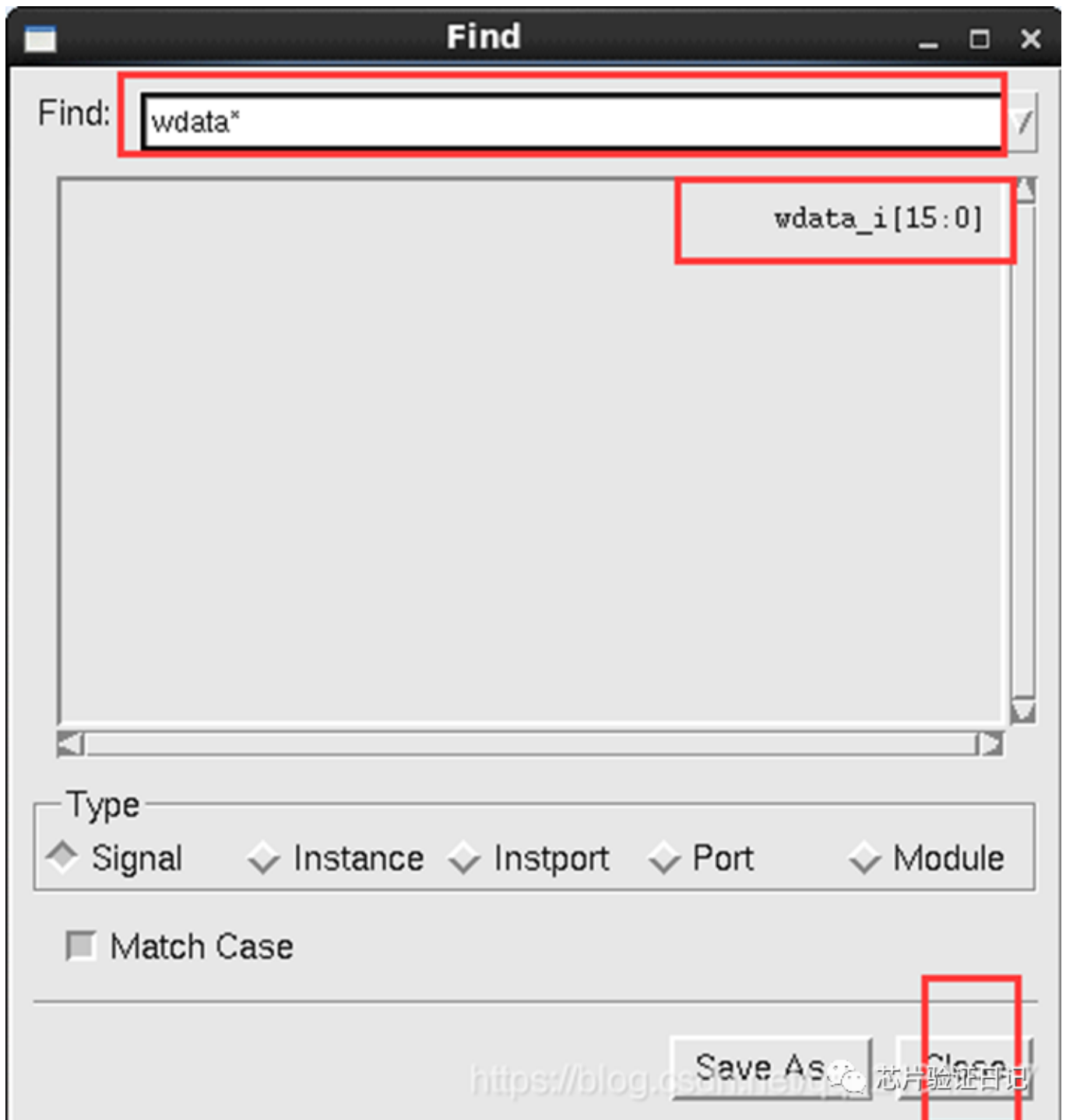


显示原理图各信号名



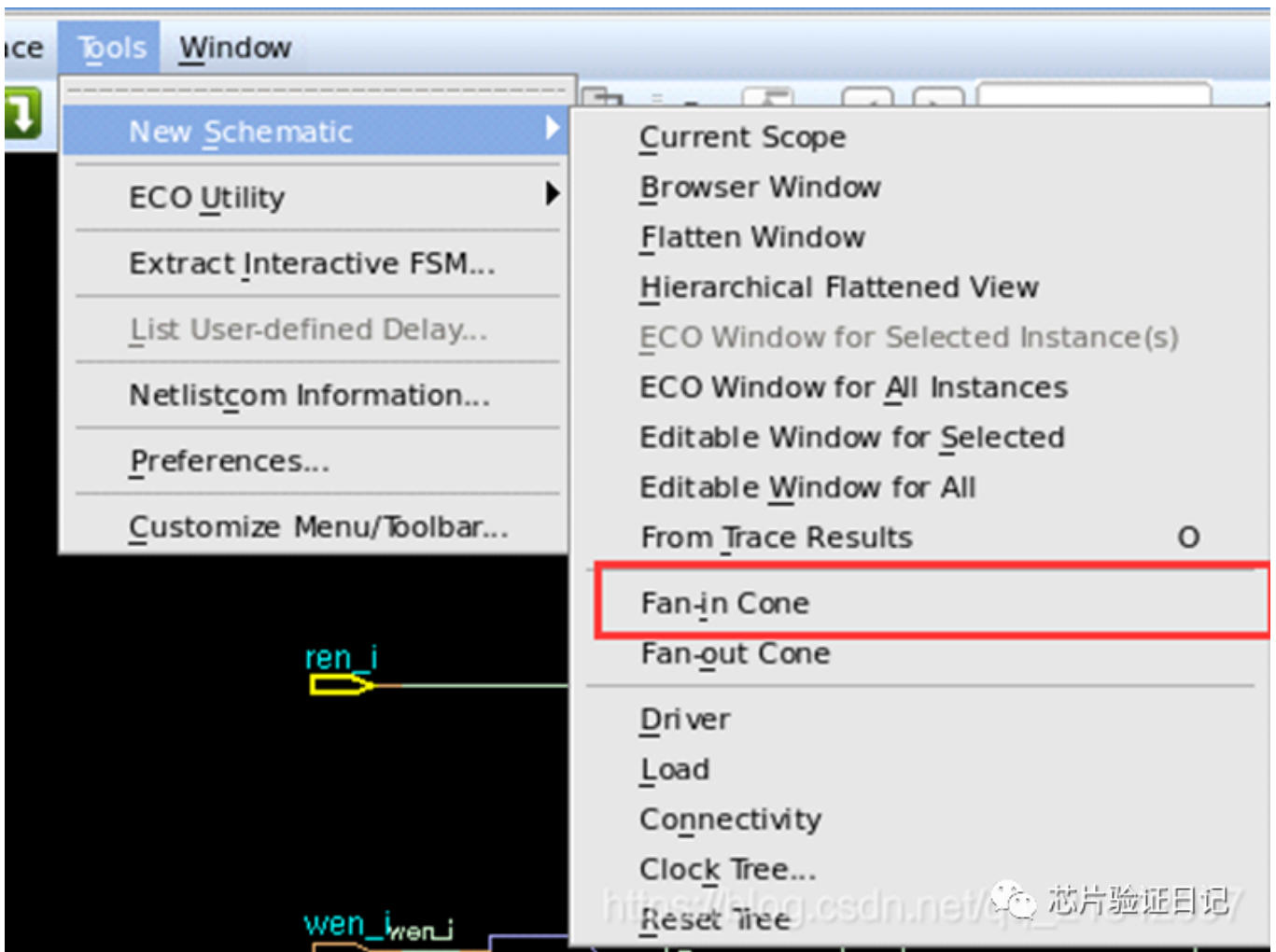
RTL级:在当前的层次中查找某个信号





使用Fan-In Cone追踪某个信号的驱动逻辑

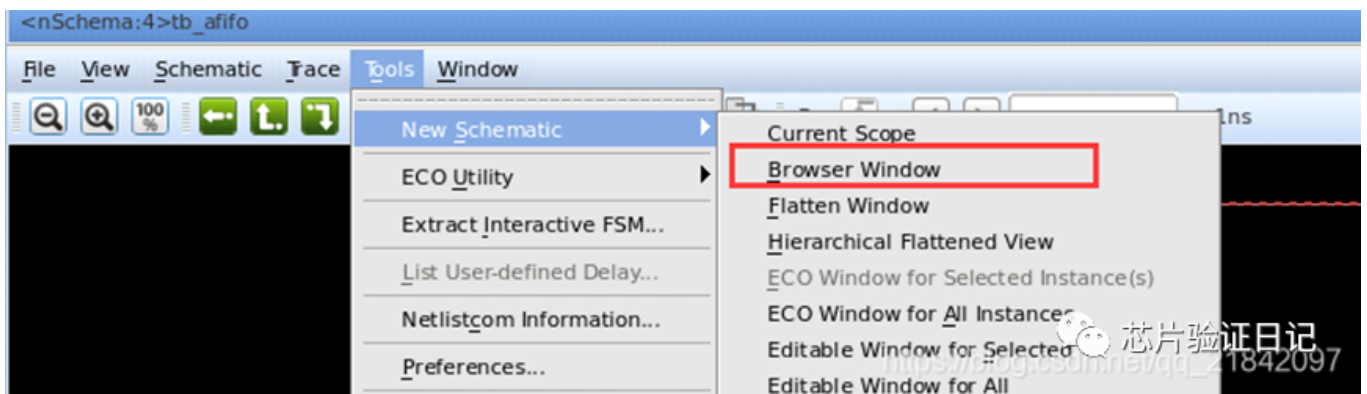
①首先要用上面的办法来查找信号②调用该工具：



产生partial hierarchy schematic

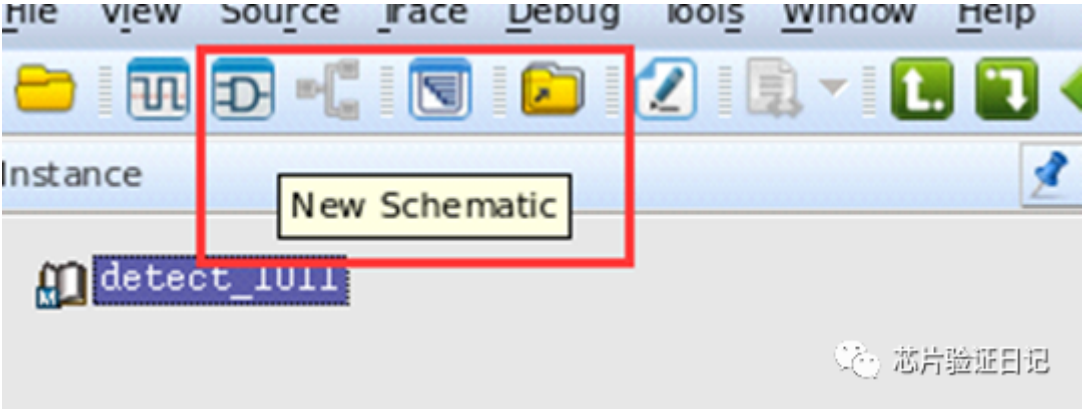
用来查看与选择特定信号有关的逻辑/模块

①选择需要查看的信号（可以通过shift键来选择多个信号）

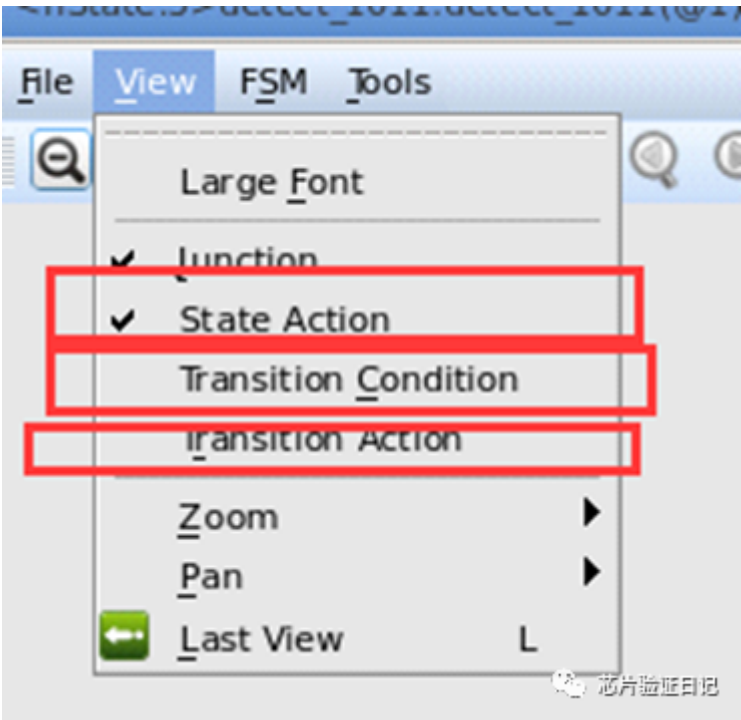


(4) nState使用

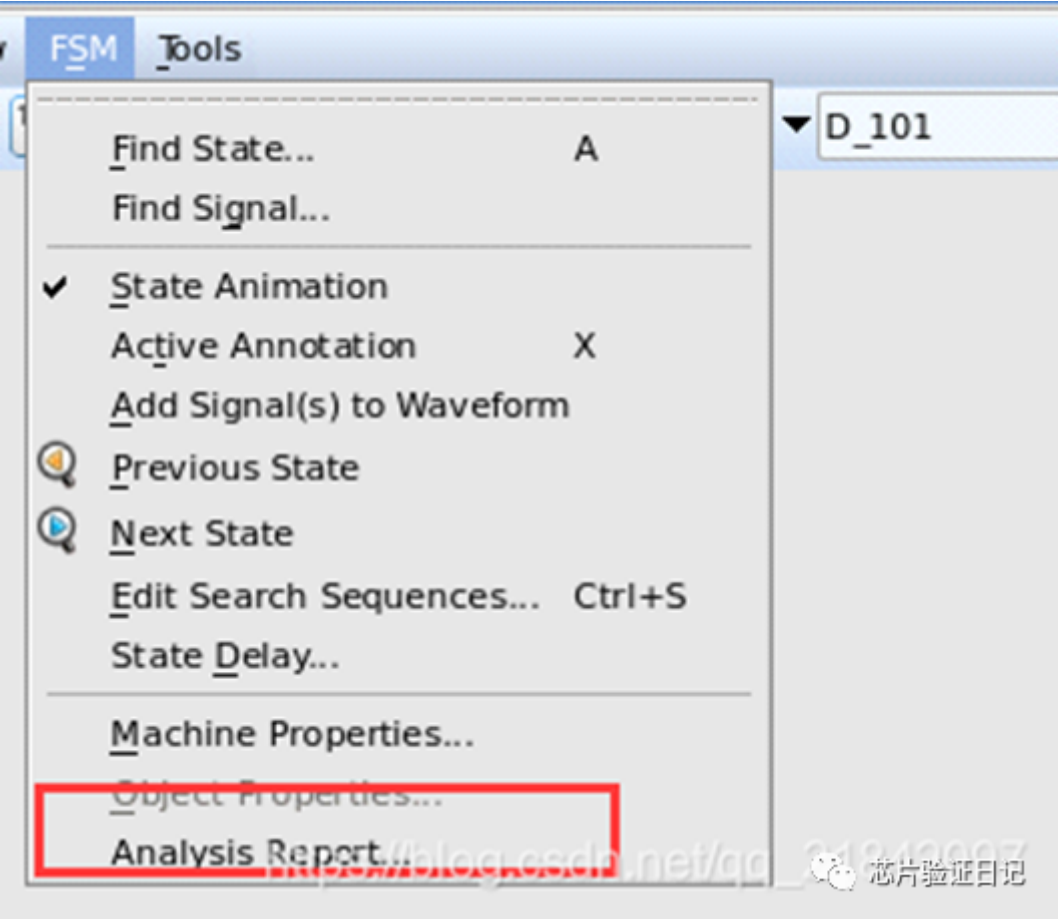
查看状态机视图

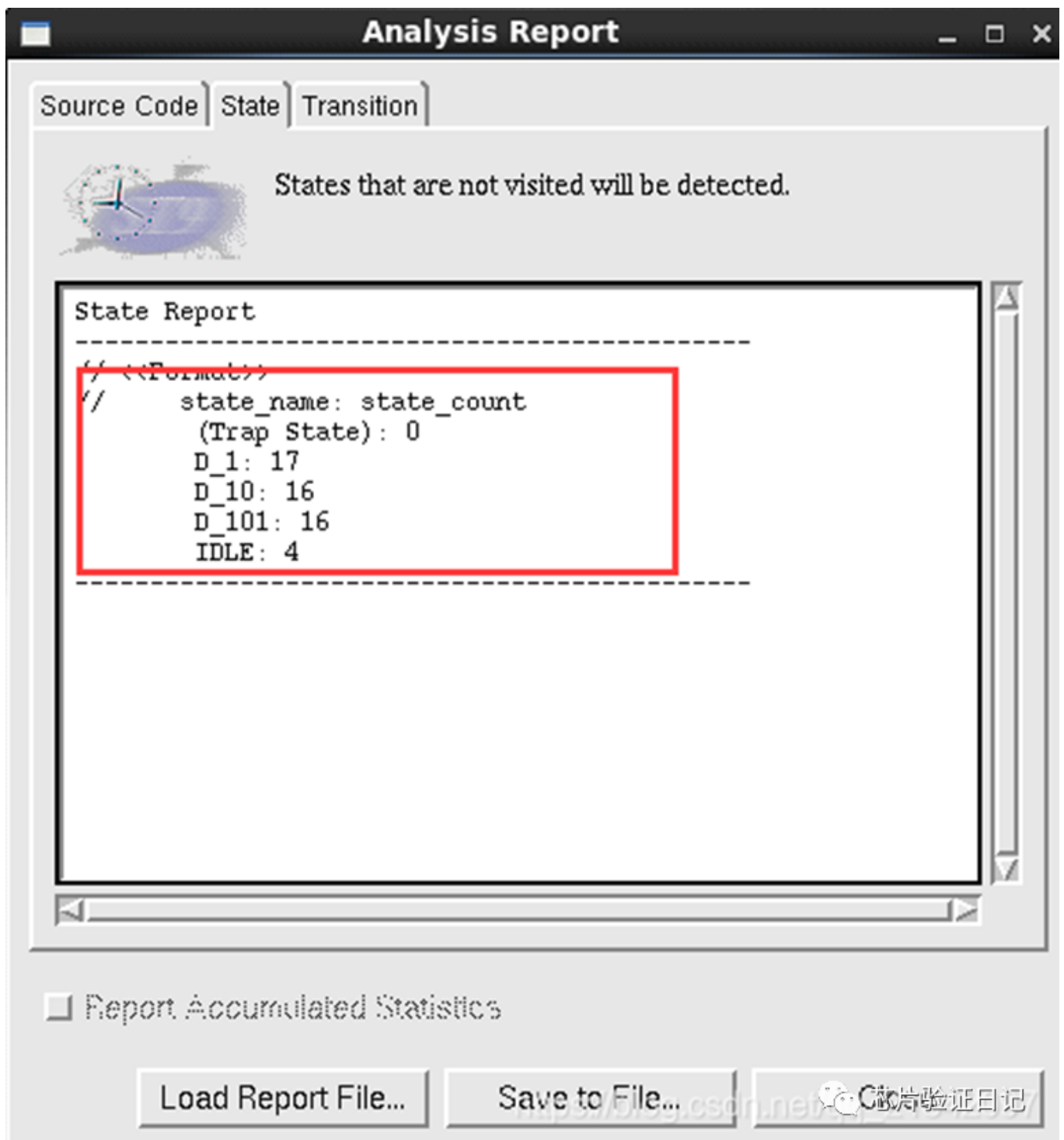


查看状态的执行语句和转移条件



查看某个状态的执行次数





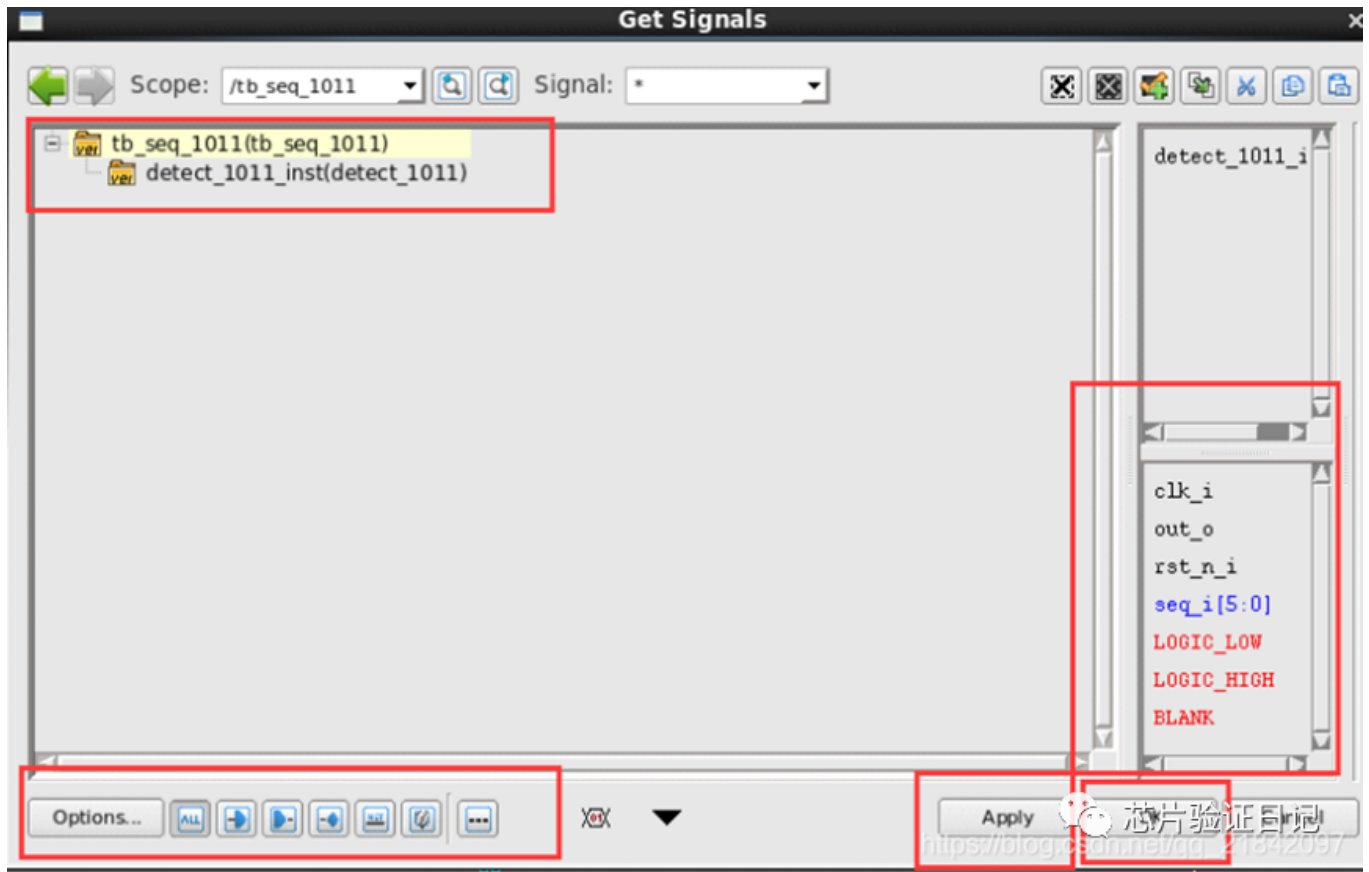
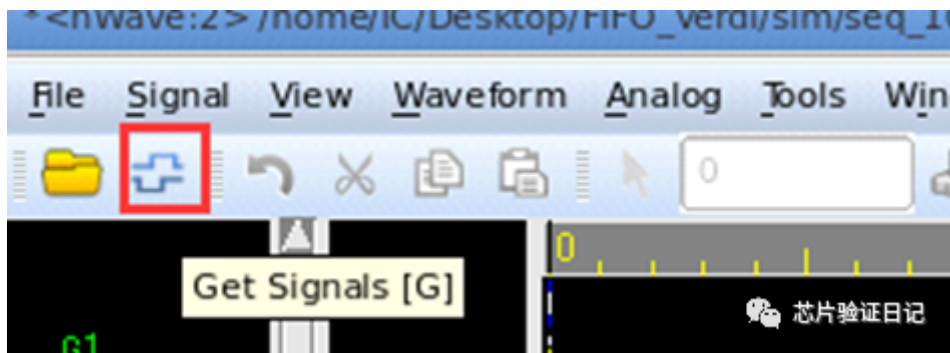
(5) nWave使用

[1] 添加波形到nWave

①我们要打开.v文件

②打开nWave然后打开.fsdb波形文件③正式添加波形：

快捷键G或者：



在nTrace中选中信号后，鼠标中键拖拽，或者ctrl+w进行添加

自动加载,用-ssf 命令

verdi -sv -f filelist.f -top tb_top -ssf dut.fsdb &

[2] 查找某信号，添加到nWave中

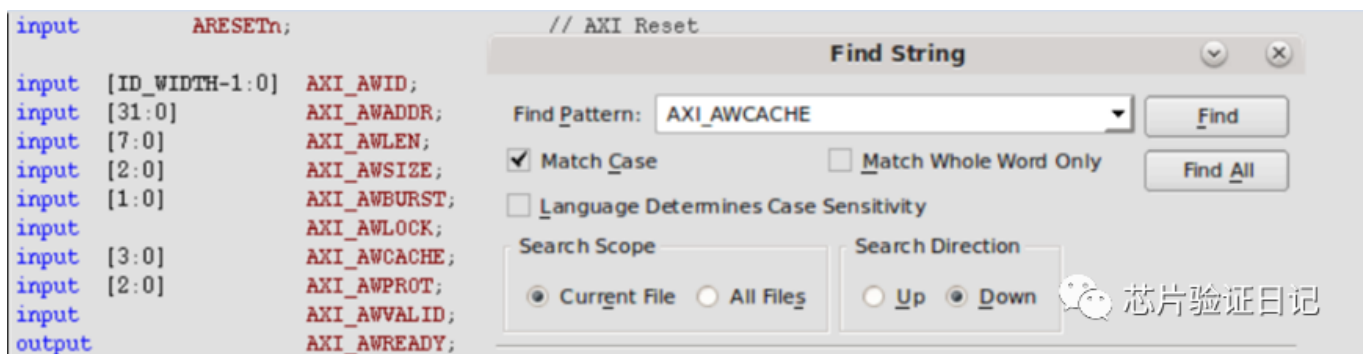
①通过nTrave查找到该信号②通过上面的方法进行添加或者通过get signal来查找

- 在nTrace界面
 - ◆ 左键拖选框
 - ◆ Ctrl+w
- 在nWave界面
 - ◆ 可以使用通配符
 - ◆ 可以方便的选择输入/输出/inout/wire(not)/reg信号

nWave波形窗口，快捷键 g 加载信号(Get signals)。在弹出的窗口选中需要添加的信号，点击滚轮选择需要添加信号的位置，也可在代码中(nTrac界面)选中信号 Ctrl+W 将信号加载在波形中

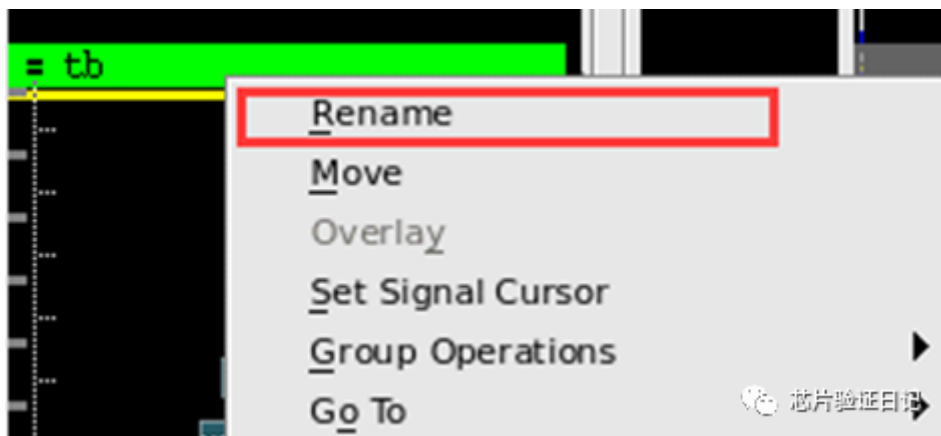
[2--]查找信号

找到模块里的某个信号，按？弹出find string对话框进行查找。



[3] 给Group重命名，排序，以及调整信号位置

[1]重命名

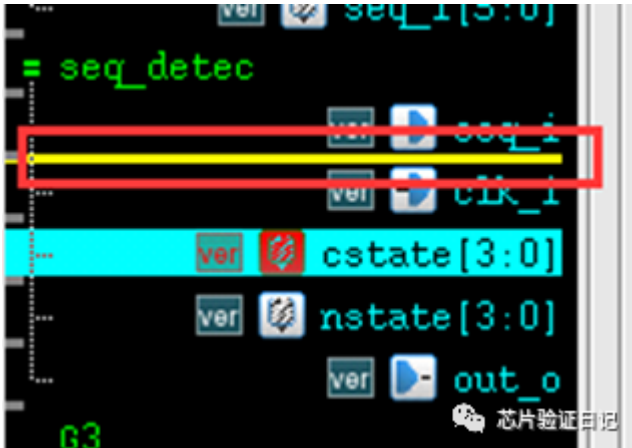


[2]信号的拖拽：通过鼠标中键进行拖拽。

[3]缩小：快捷键z，或者100%匹配：F

[4]放大：左键拖选放大范围shift+Z ctrl+滑轮：进行放大或者缩小

[5]移动：鼠标中键按住信号拖拽中键选择位置：



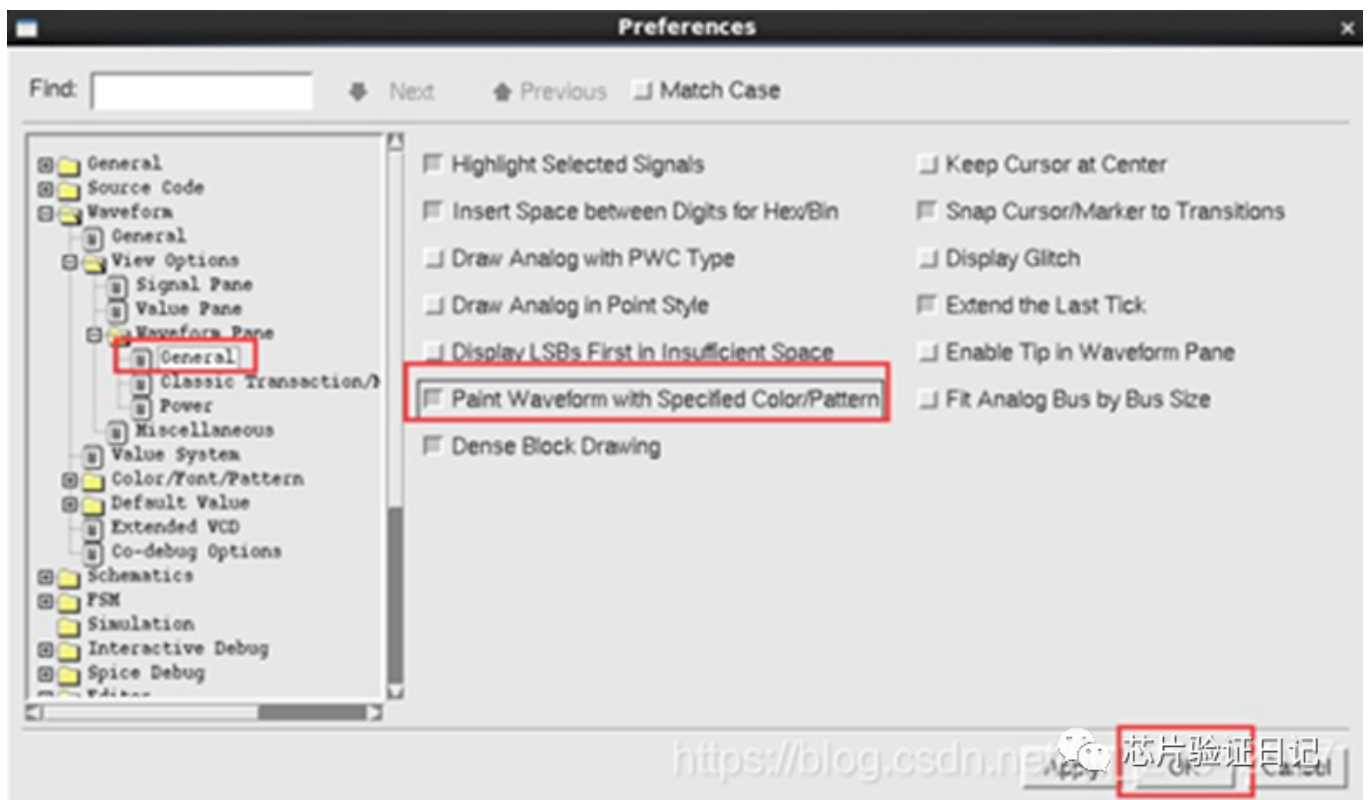
[6]复制：ctrl+P,或者右击信号，选择复制的选项

[7]粘贴：ins键，或者右击信号，选择粘贴的选项（注意：可以通过鼠标中键选择粘贴的位置）

[8]删除：delete键

[4] 改变信号的颜色

第一步：Tools->preferences->waveform->viewoption->waveformpane->general->paint waveform with specified color/pattern

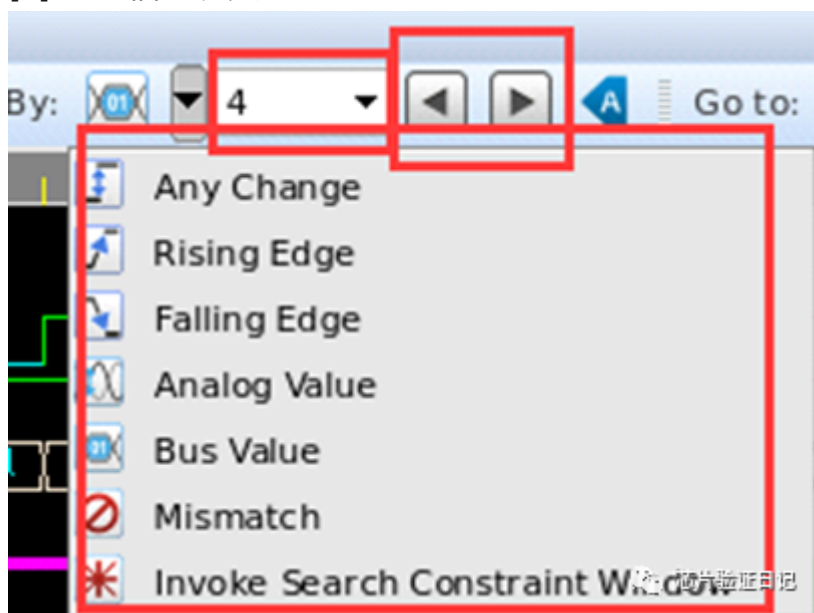


第二步：选中信号，然后按快捷键C改变信号颜色。

按C或者T修改信号或者波形颜色，按T更实用，每按一次颜色自动改变。不区分大小写。



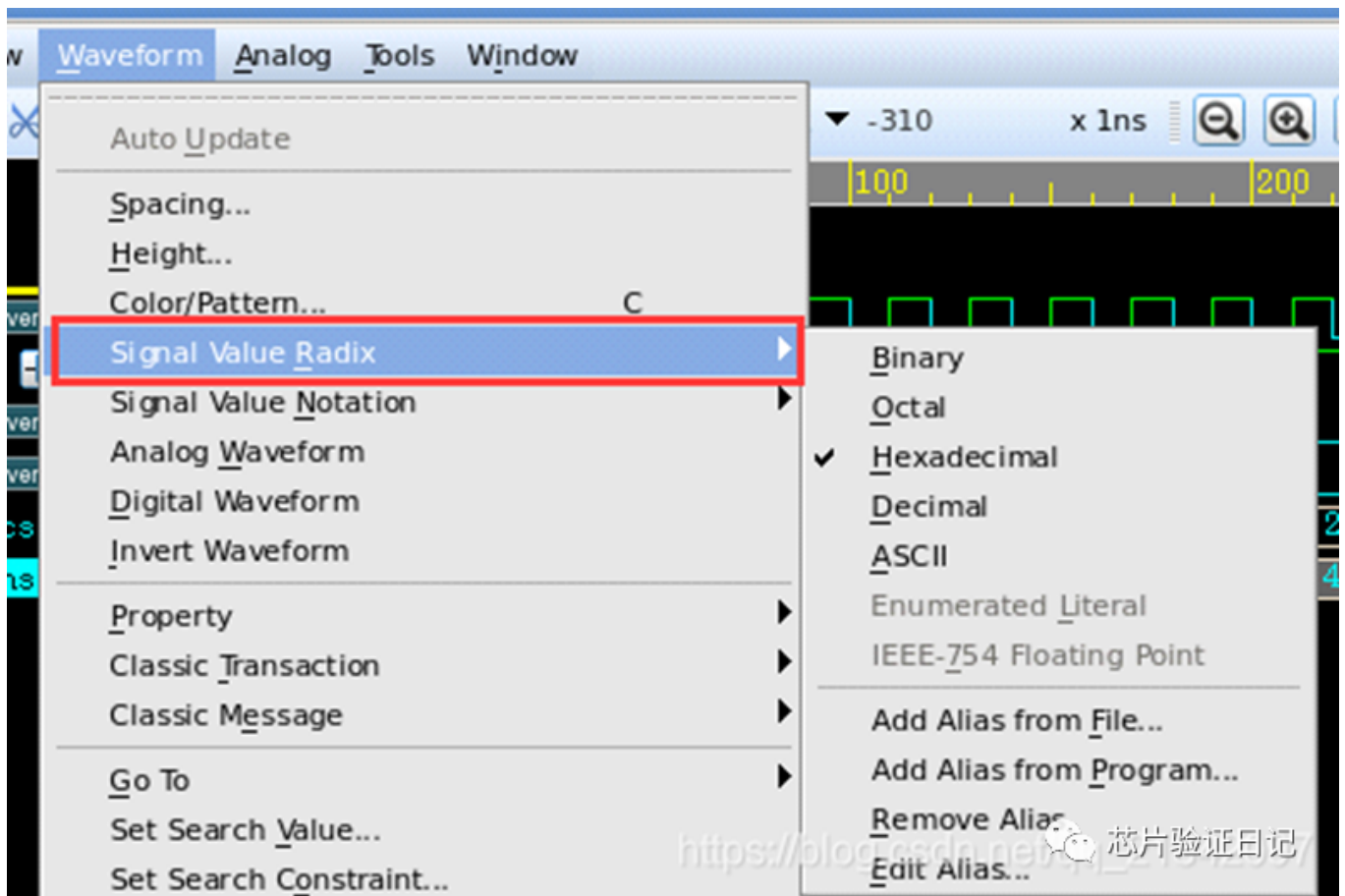
[5] 查看信号及变化



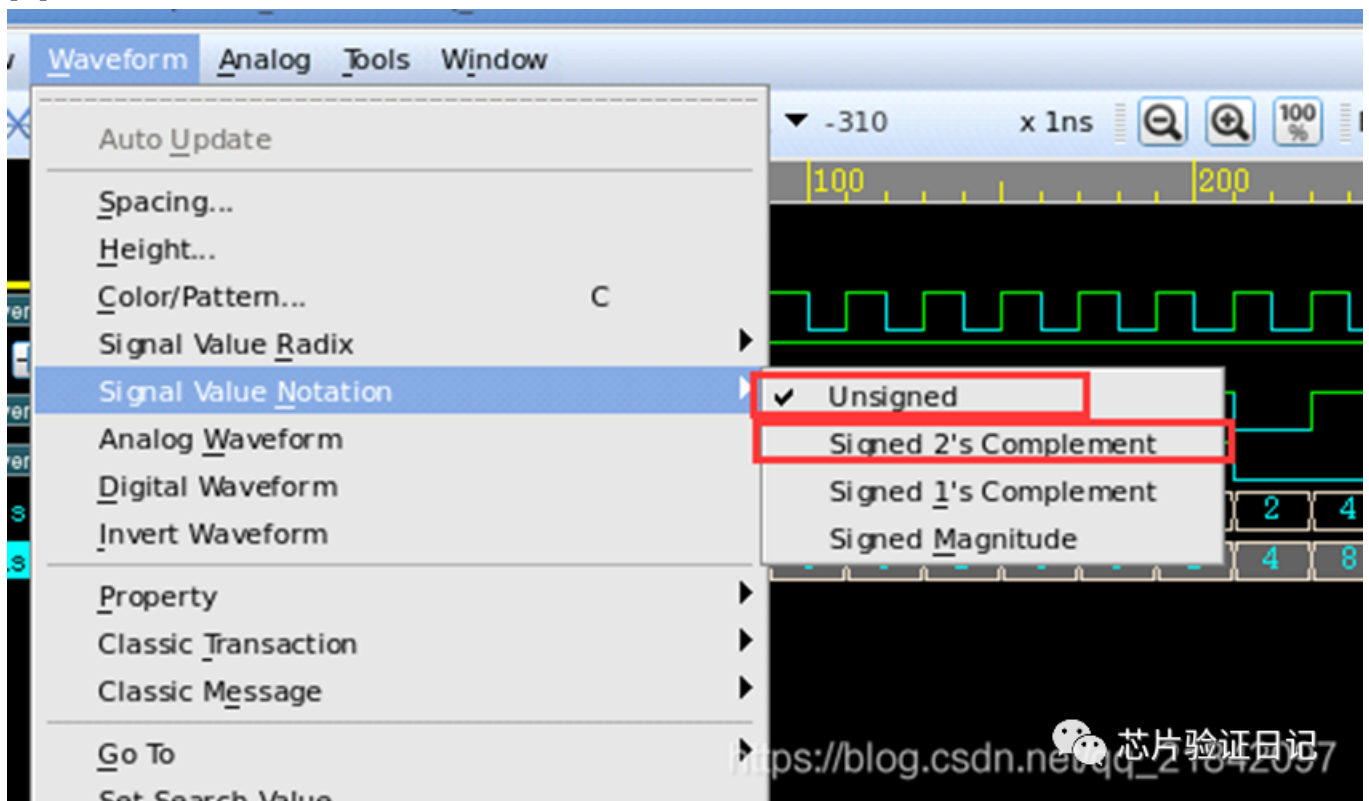
任意值：所有信号变化，一般会应用到组合逻辑的指示信号信号沿：用于查找有效指示信号，比如使能信号

总线值：主要是用来查找数据信号模拟值：查看比如函数发生器产生的正弦波信号进制之间的转换：

[6] 进制之间的转换

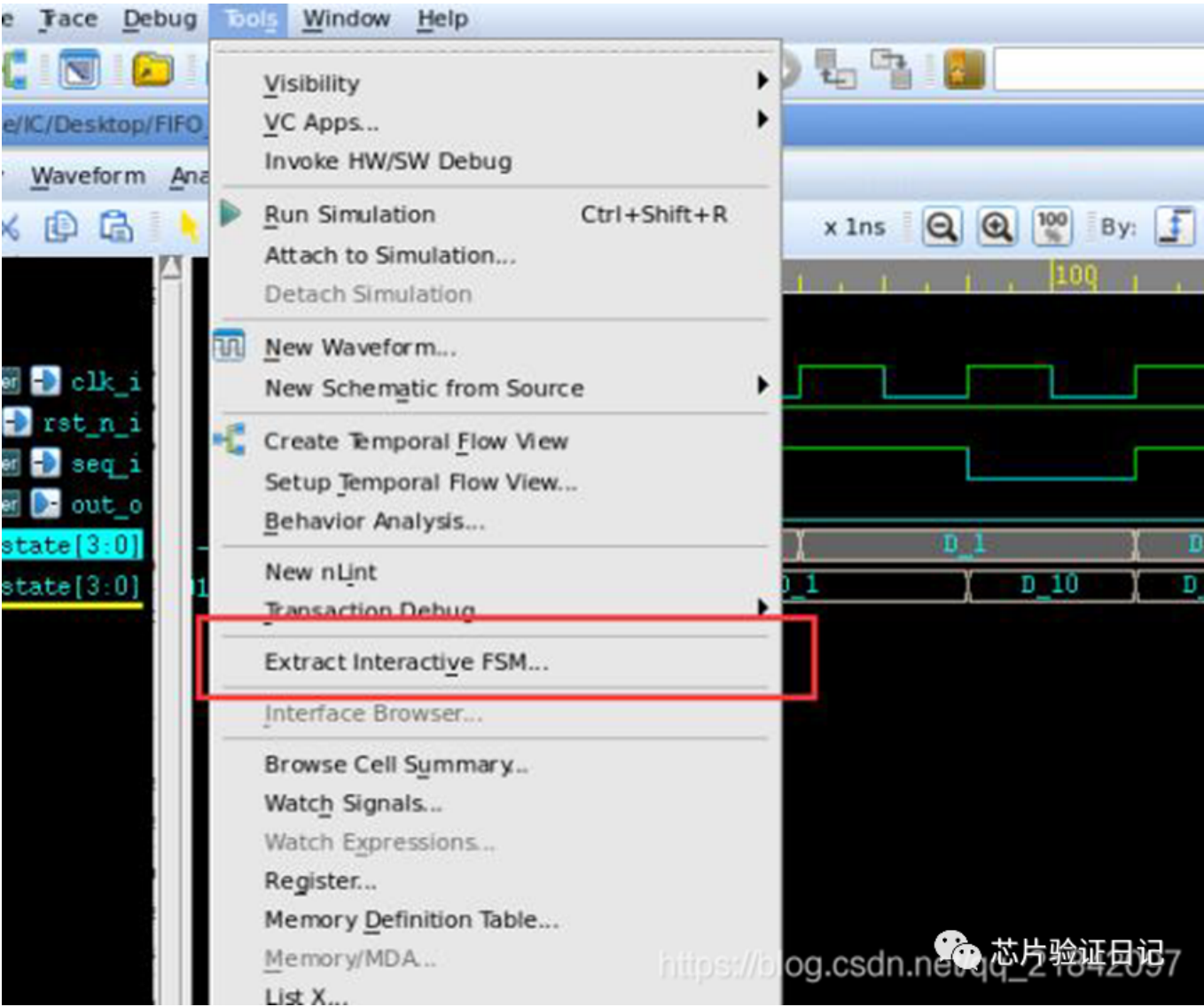


[7] 符号数之间的转换

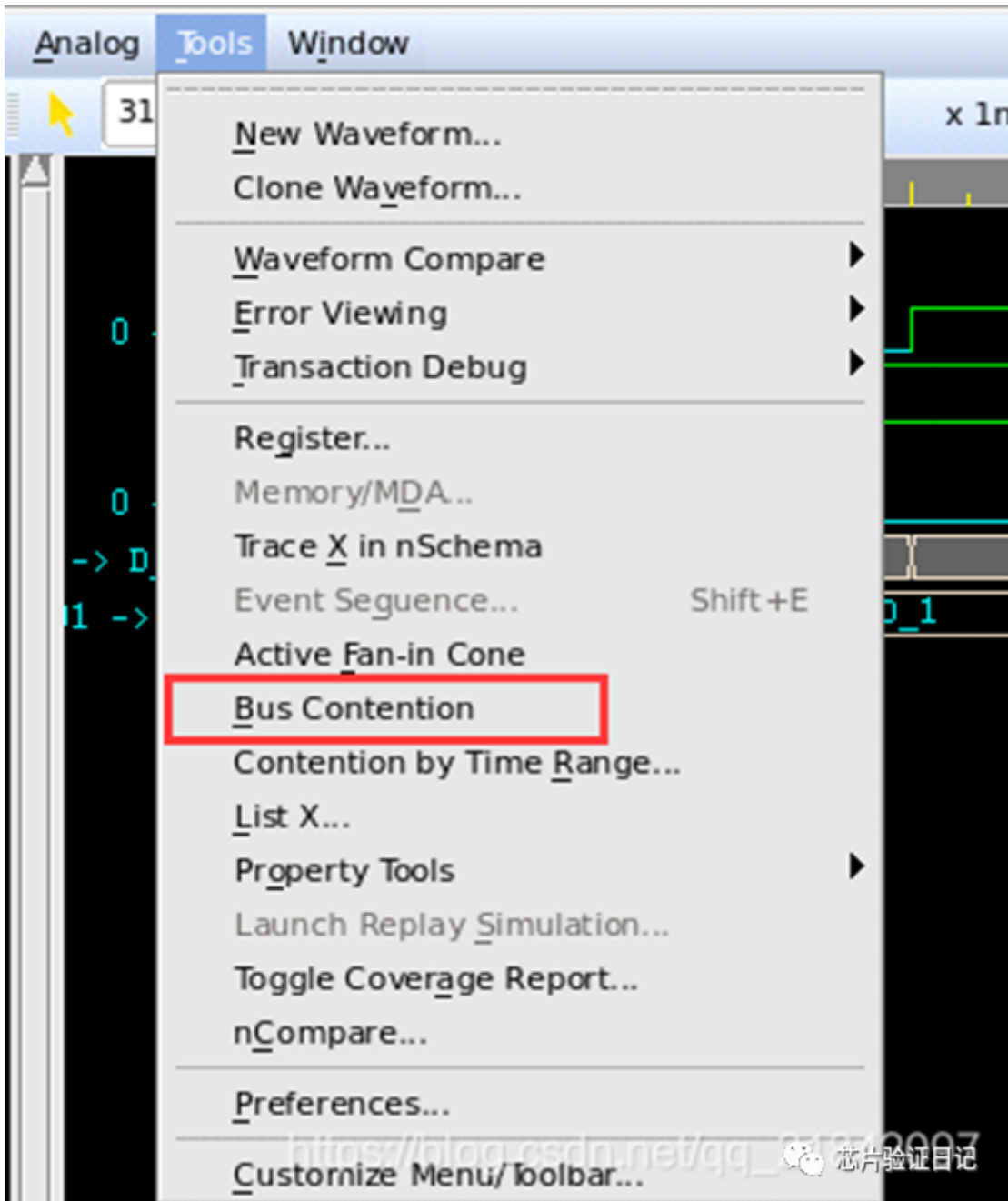


[8] 状态寄存器显示为状态名字

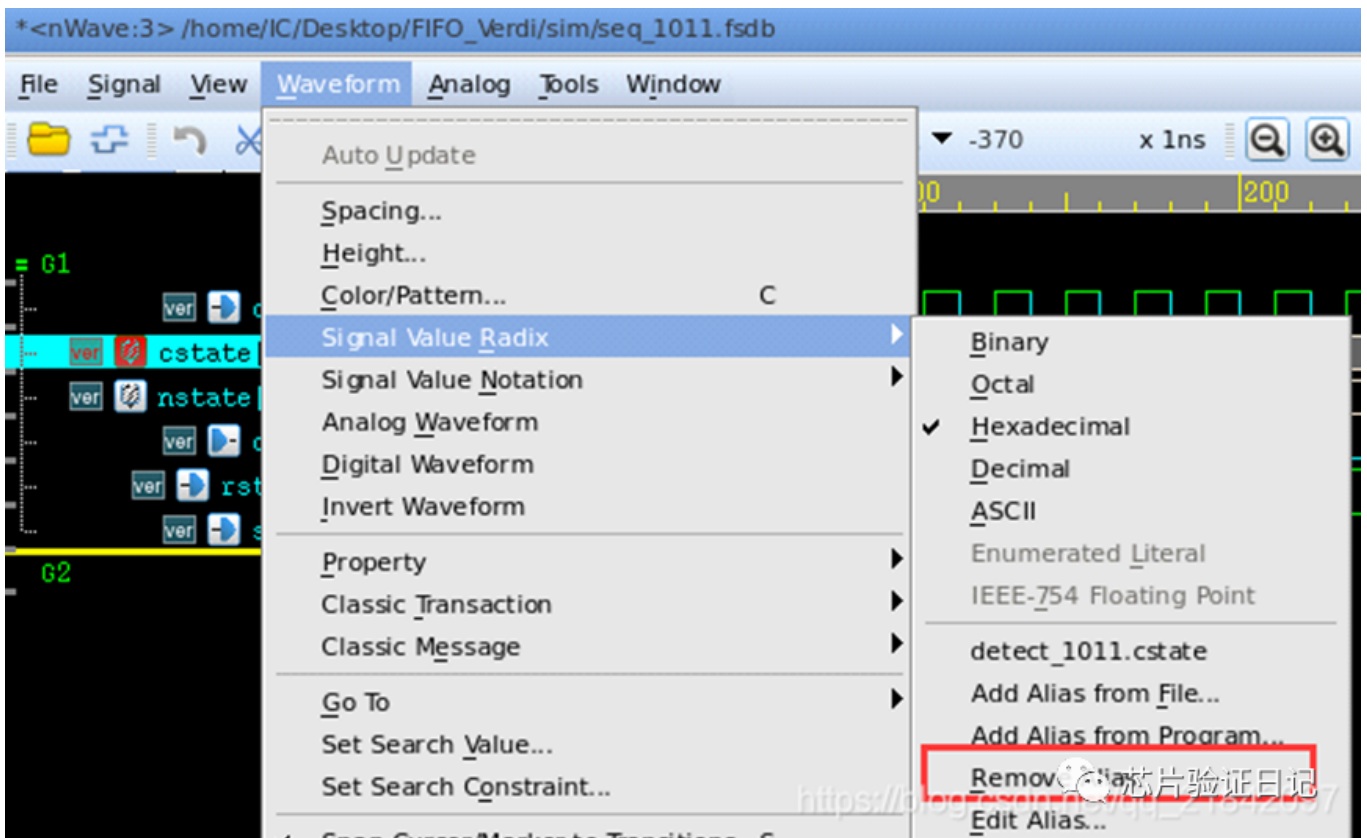
nTrace:



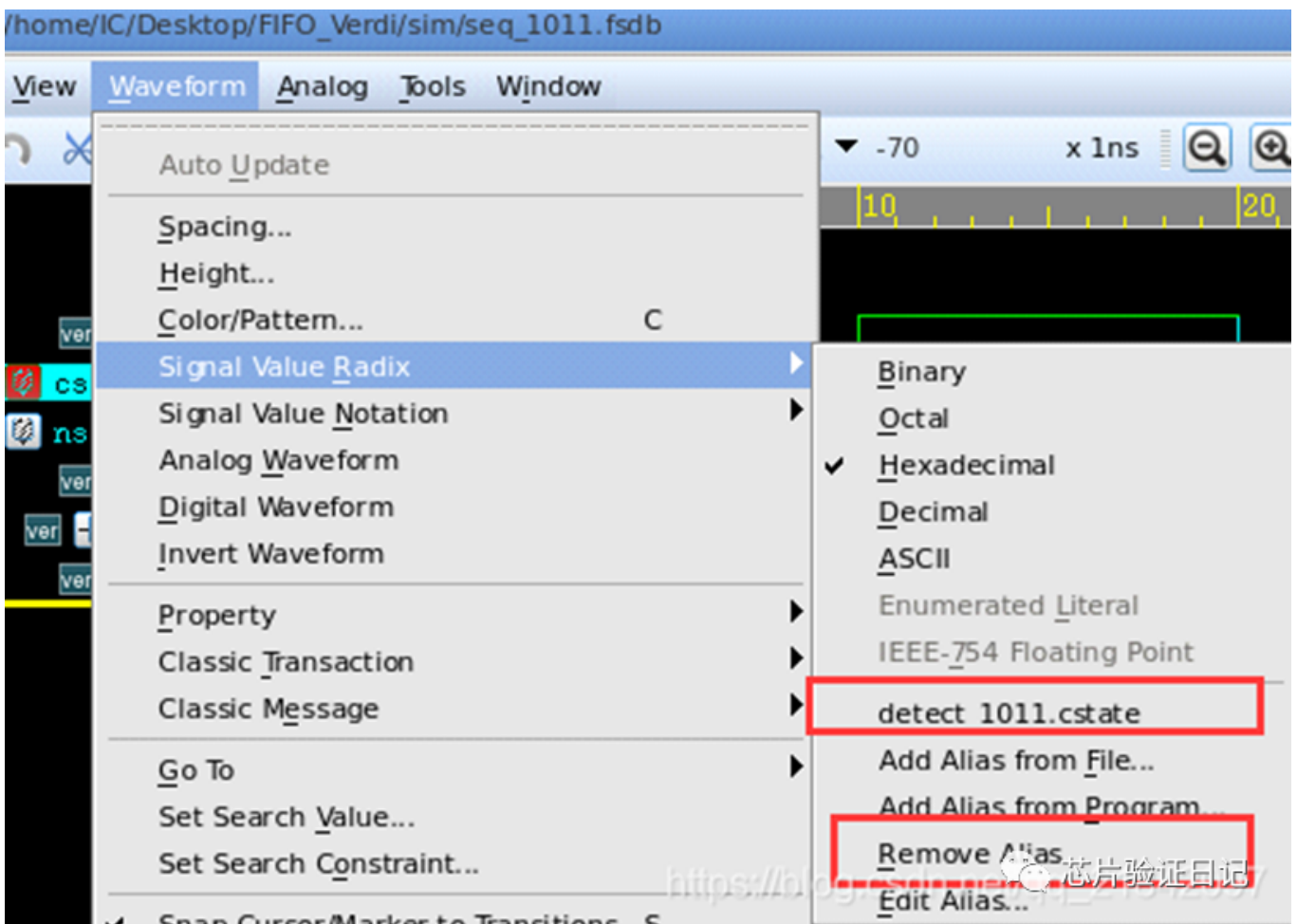
或者nWave:



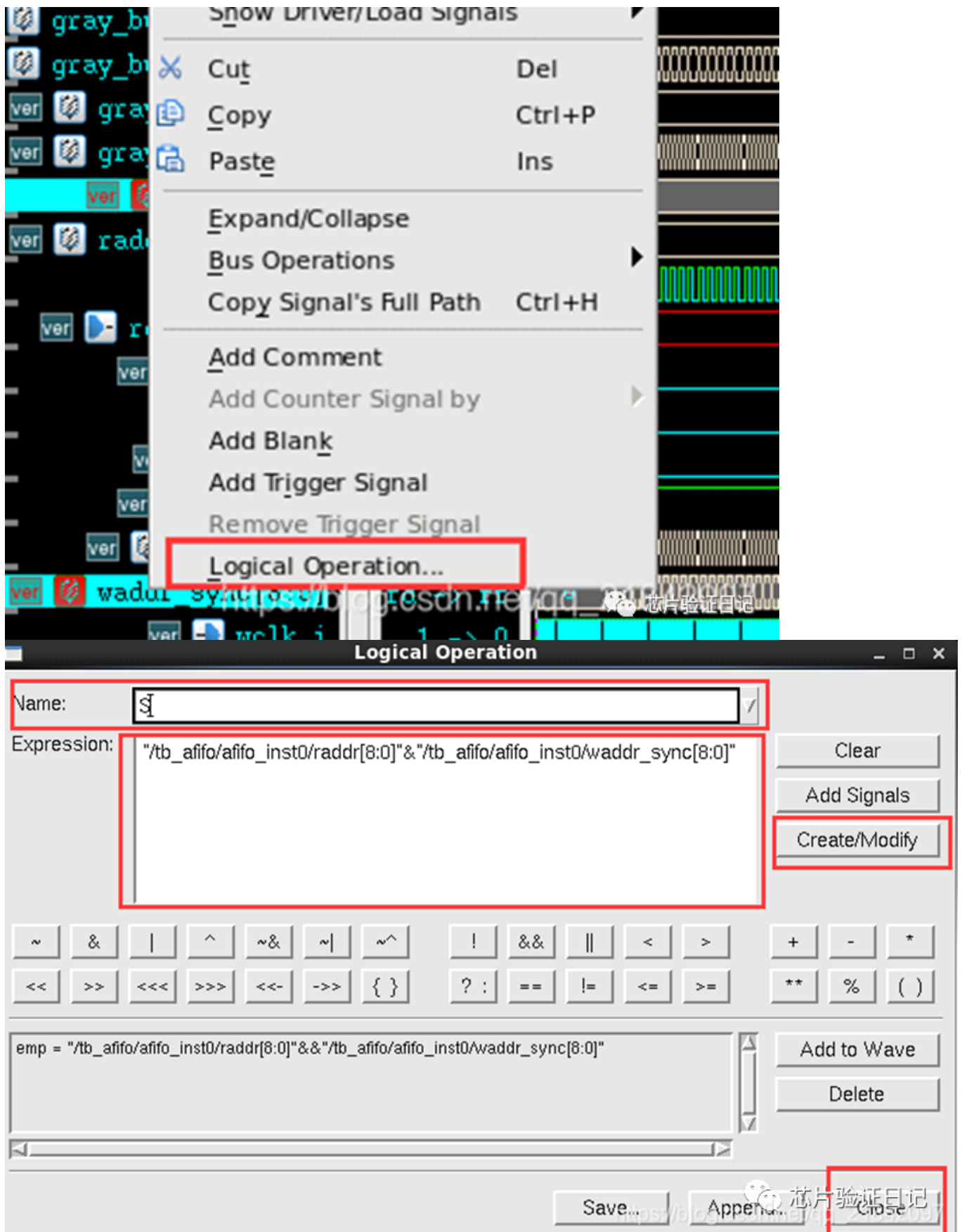
[9] 通过移除关联，从状态机名字恢复到原来的进制显示



可以看到，我们打开过状态机视图之后，我们也就可以在状态名和进制之间转换了：

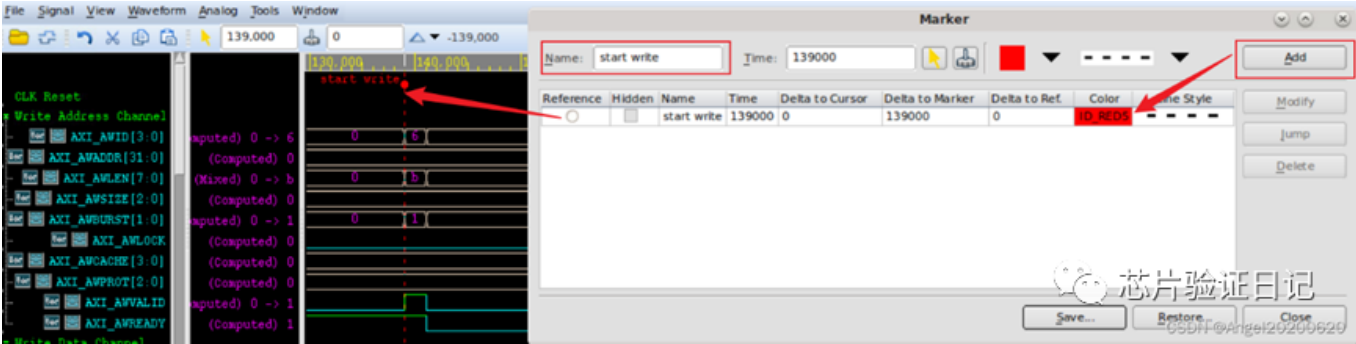


[10] 通过逻辑操作创建新信号



[11] 如何添加marker

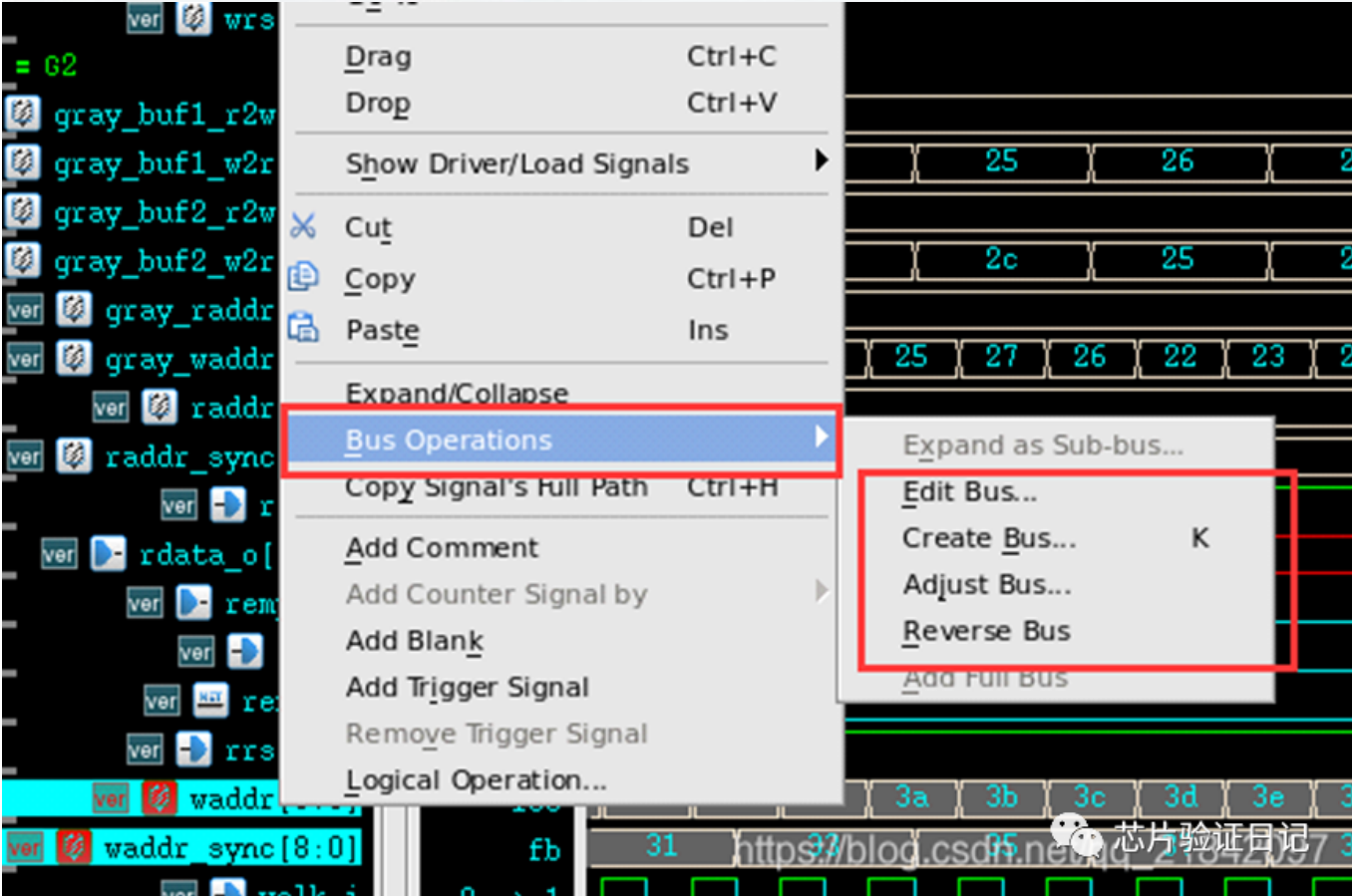
shift+M。为方便波形定位，按键shift+M使用mark功能标记一下，可以自定义名称和颜色，方便查找。



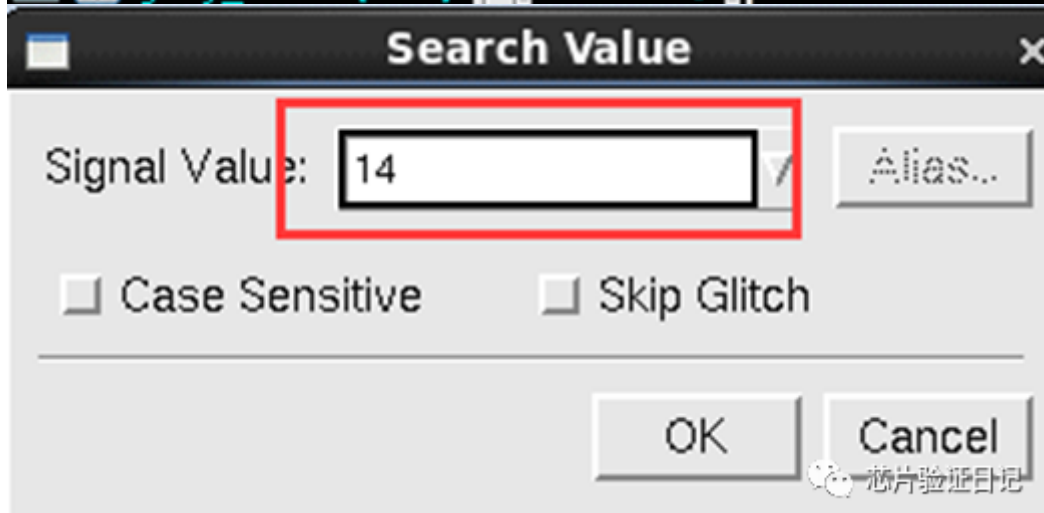
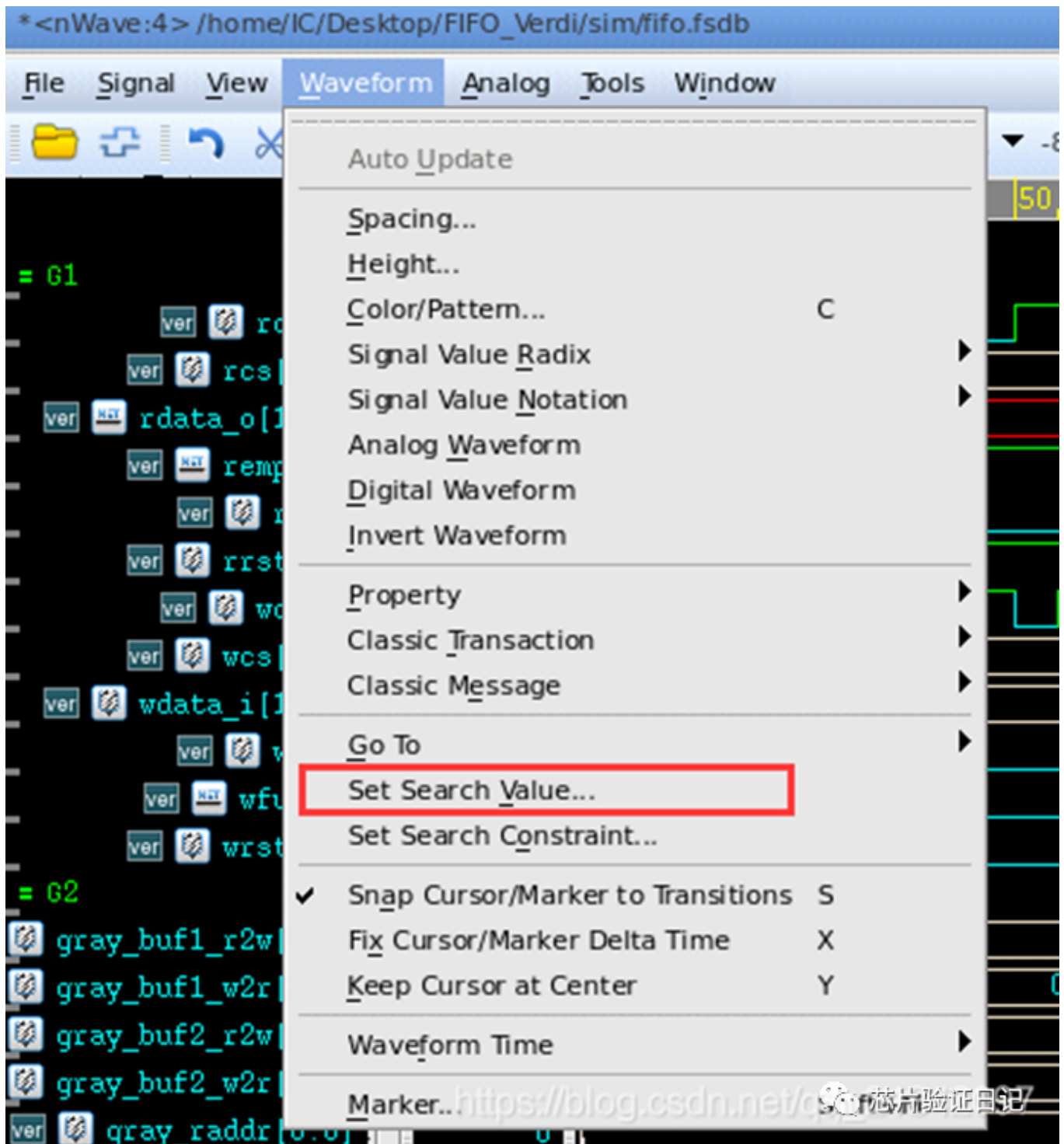
[12] 总线操作:

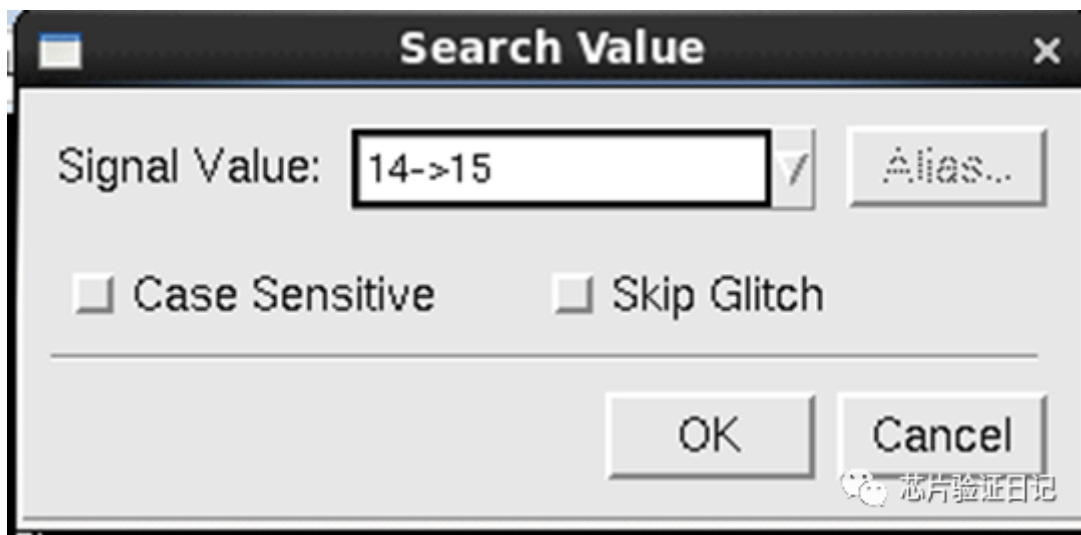
操作	应用场景
生成	将多个 (组) 信号组成总线
抽取	选择多bit信号 (如2048bit) 中的部分, 生成新总线, 方便查看 (基于生成功能)
编辑	处理当前数据总线信息
取反	可以应用于大小端的场景

芯片验证日记

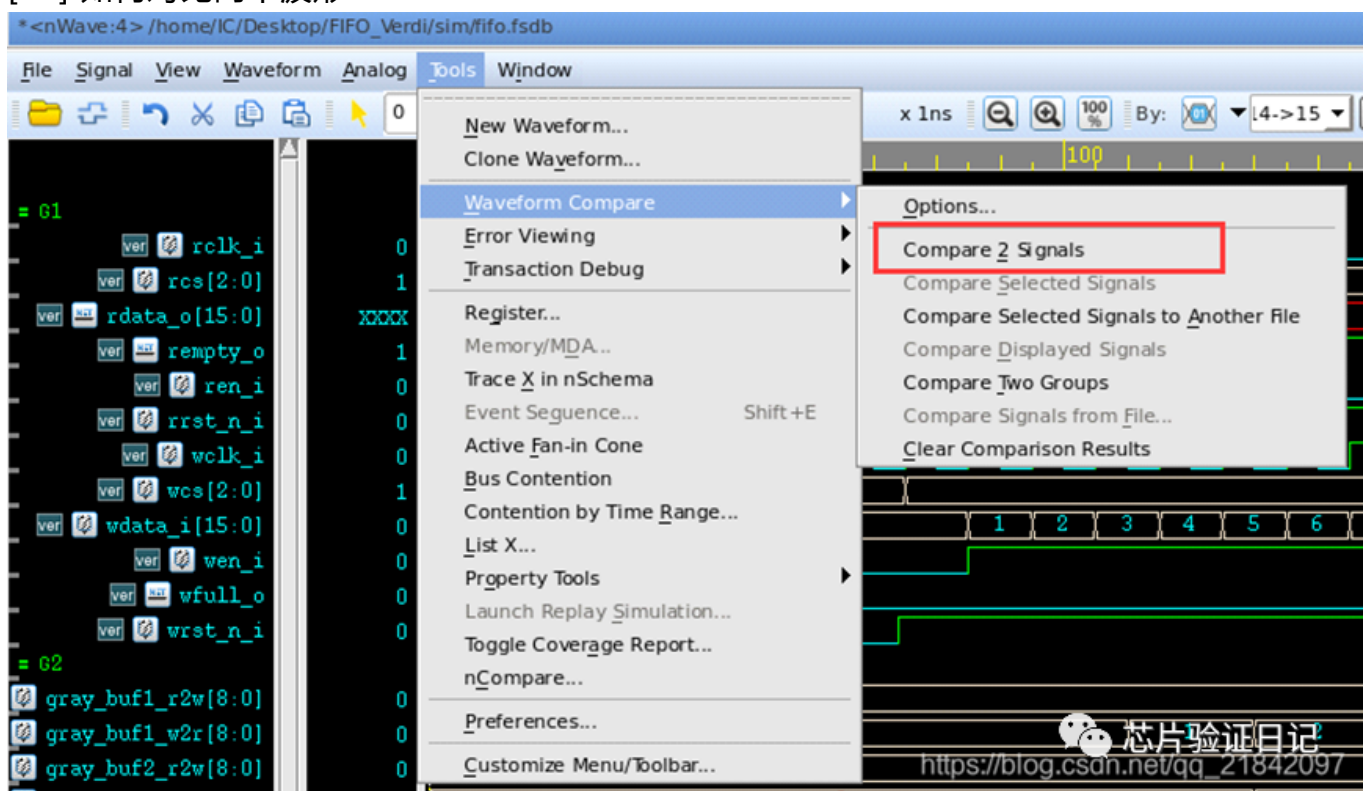


[13] 如何查找信号的某个值、某个值跳转到某个值





[14] 如何对比两个波形



[15] 显示信号全路径

按 H 显示波形信号全路径，再按H撤销。不区分大小写。



[16] 对信号的简单操作

点击波形窗口，选中信号，使用Delete删除信号。选中信号，使用Ctrl+p复制选中的信号，然后使用insert按钮插入，粘贴的位置由黄线所在位置决定。

[17] 移动波形窗口信号的位置

使用滚轮在波形窗口选中信号，然后拖到代码窗口即可查看选中波形的代码逻辑

[18] 放大缩小

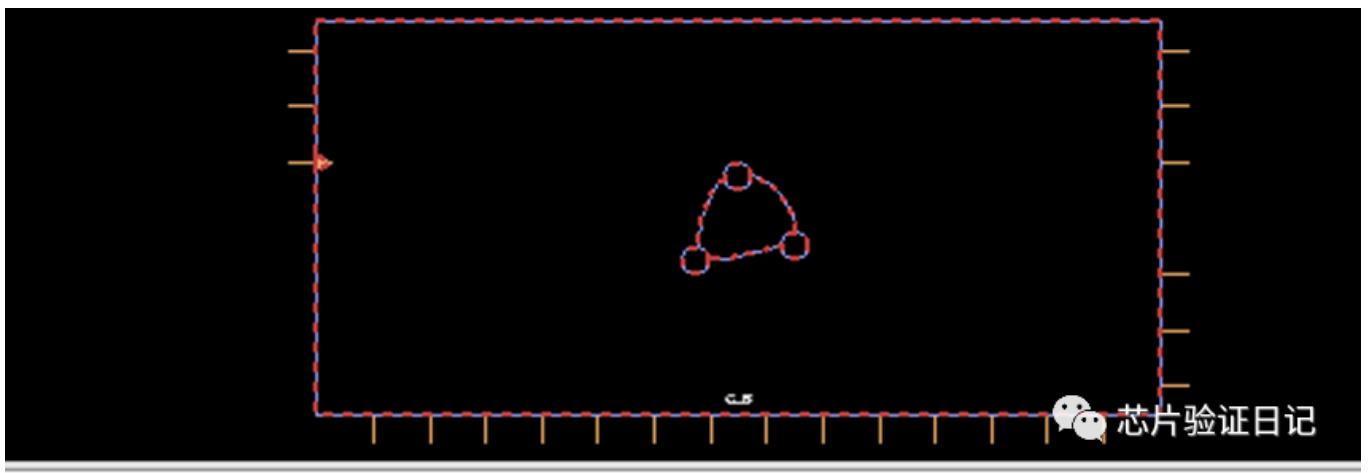
Z:Zoom In; z:Zoom Out; f:Zoom All

[19] 显示状态机的跳转

选中状态机状态的波形，在nWave窗口选中Tools>Bus Contention>打开状态机，（关闭），**返回查看波形已经变为状态的名称**。补充：选择具有状态机的模块选中，然后在主工具栏选择Tools>Extract interactive FSM



点击OK，然后点击下面的状态机，即可看到状态的跳转。



[20] 参考信号数值

选中Source>Active Annotation即可看到每一个信号，参数的数据显示。

[21] 新建组并对组重命名

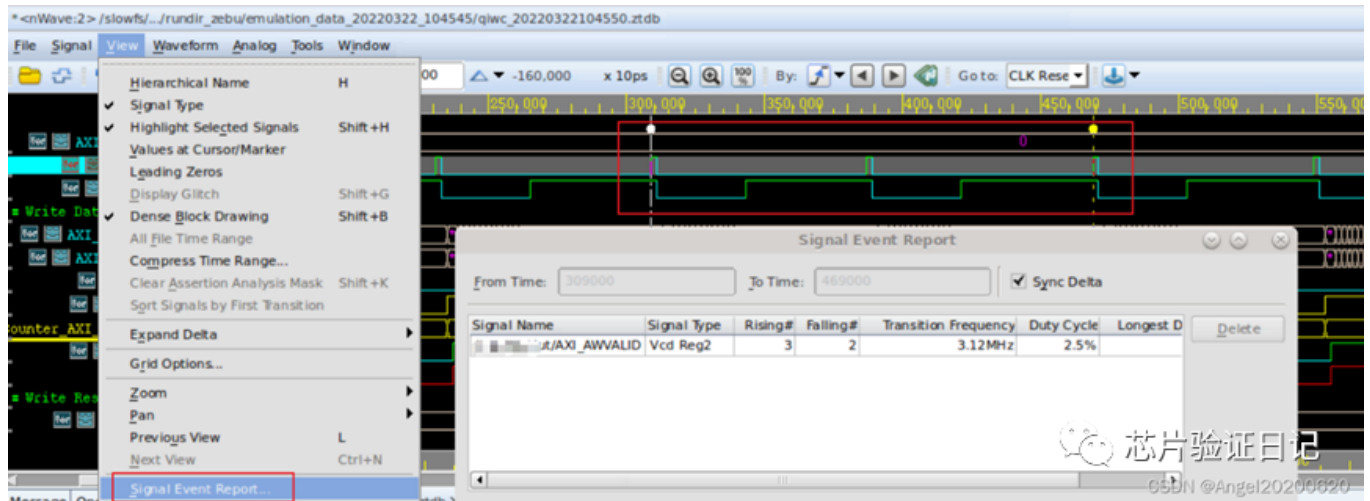
将黄线放到最后一行，添加信号即可新建一个信号组。然后鼠标选中右键即可进行相关操作，也可以选中一个模块，直接Ctrl+4即可将接口信号加入波形窗口。

[22] Signal Event Report统计

统计出某个信号在特定时间段内的跳变的次数。

按鼠标滑轮中建，添加一条mark标记线用于选择统计区间。

View -> Signal Event Report



Rising: 指的是指定Marker时间段内上升沿数;

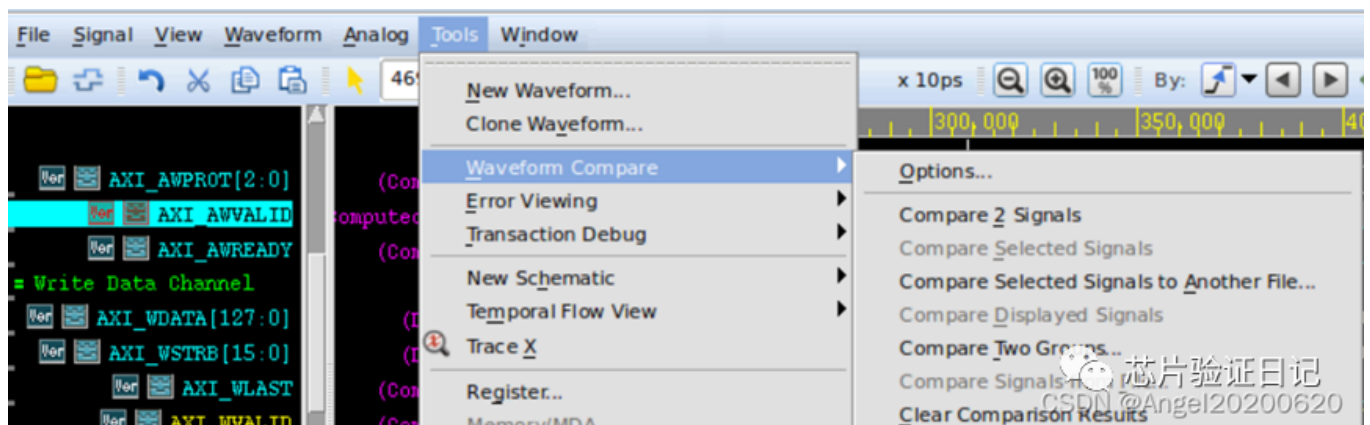
Falling: 指的是指定Marker时间段内下降沿数;

备注: 利用Logical Operation和Signal Event Report可以统计非连续有效信号

[23] Waveform Compare比较波形

nWave提供了一种综合比较自动比较不同结果的能力。在比较完成后，nWave以图形方式显示波形窗格中的任何不匹配，然后通过每个不匹配来分析差异。

Tools -> Waveform Compare

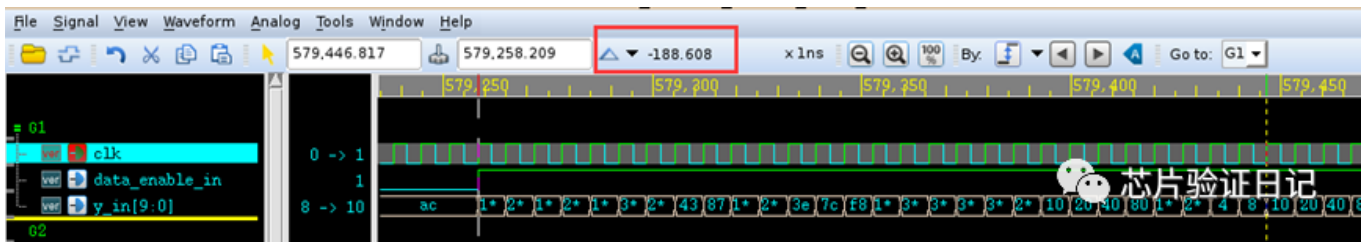


(四) Verdi使用技巧

1、连续信号有效数量的测量

[1] 下面分享一些在实际做项目过程中verdi在debug时的小技巧：连续信号有效数量的测量。

如下图所示，clk是输入时钟信号，data_enable_in是输入数据有效标志信号，y_in则是输入数据。在debug的时候，有时我们想要知道白色光标和黄色光标时间内有多少个连续有效的数据y_in。我们该如何做到呢？



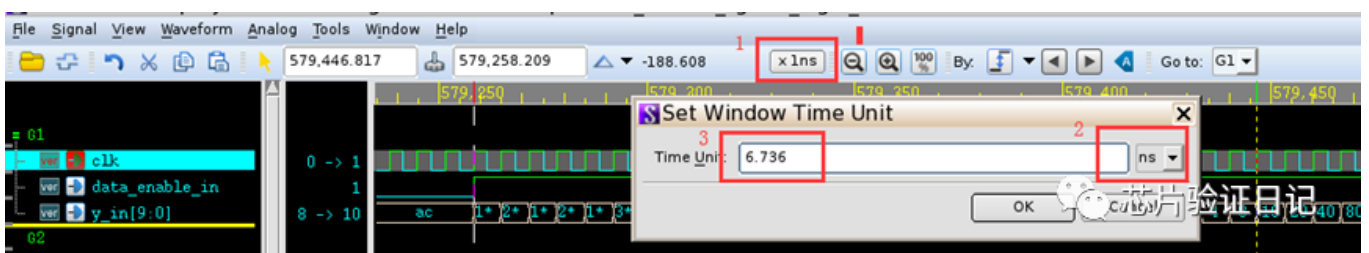
一般可以使用以下3种方法：

方法1：最直接也是最笨的办法，当然是手动去数,1、2、3...；如果该时间段内，数据量较少，很快就可以数出来了。当数据量较多时，只要你有足够的耐心和时间，有时还需要一副好眼睛，最终也是可以得到正确的结果。不过效率非常低下，比较麻烦的是还容易数错。

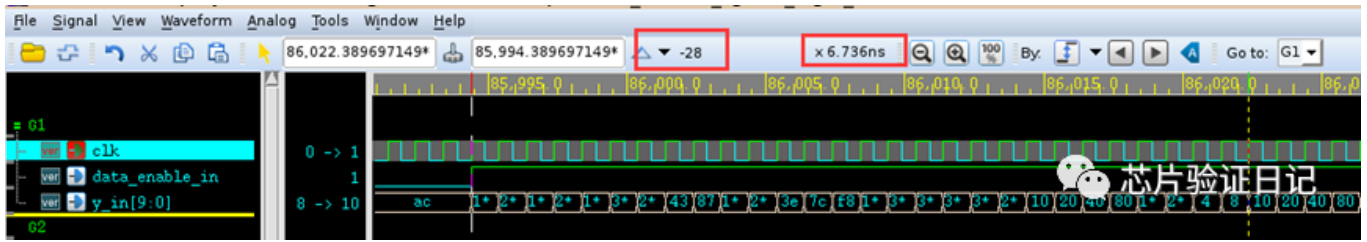
方法2：如果已知时钟周期（本例子中clkp=6.736ns），则可以计算该时间段内时钟个数来获知数据量；上面例子的图中两个光标之间的时间差 $\Delta t = 188.608\text{ns}$ ， $\Delta t / T = 28$ ，因此就是28个数据了。是不是要比直接用手动数快很多了？这个办法虽然准确，但是每次都要计算一次，如果需要频繁的获知不同时间段内的有效数据还是太麻烦了一点。

方法3：本文重点要介绍的方法。该方法是通过把时钟周期设置为时间单位，这样两个光标之间的时间差就是该段时间内的有效数据量了。具体操作方法如下：

如下图用鼠标点击1，会弹出对话框；点击2位置倒三角形，下拉选项中选时钟周期单位是ns；然后再把timeunit 改为clock周期6.736ns，点击OK。



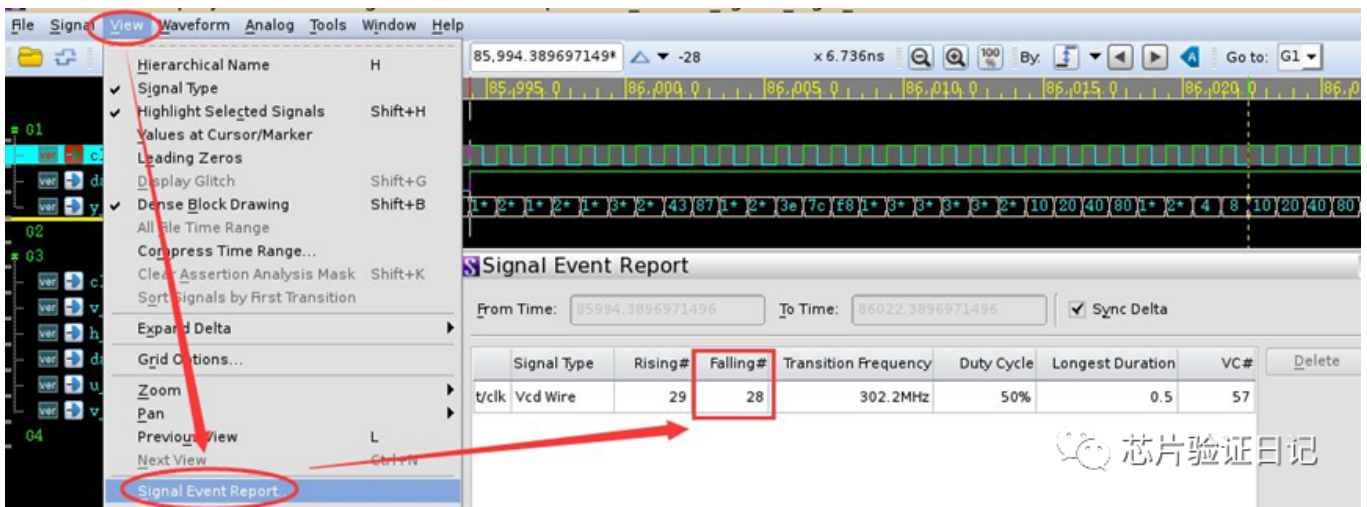
再看delta显示出来的值，已经变成28了，即为我们要的数据量了。



这个方法对于一段时间内的连续有效数据来说，应该是最简单有效的获取数据个数的方法了。任意改变两光标的距离或位置，都很直接的得到该段时间内的连续有效数据量。该方法在debug过程中常常会用到，请大家善加利用。

[2]另外还有一个比较有效的办法是利用verdi的nWave里面的Signal Event Report 功能。该方法虽然稍显麻烦，但善于利用，也可以变得非常好用。方法如下图：

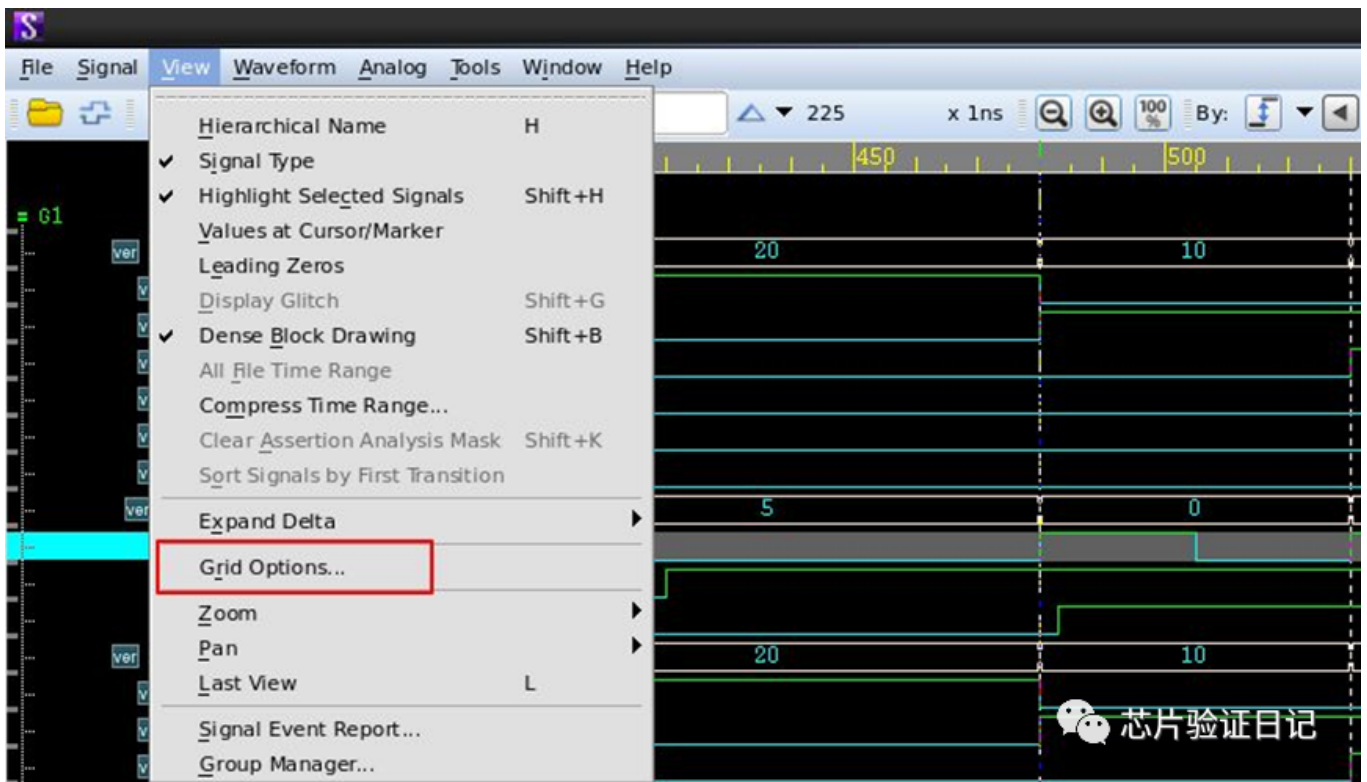
a.鼠标点击选中clk 这个信号；b.点击view菜单，选择下拉菜单里面的SignalEvent report，就会弹出对话框；c.进度条往后拖，会看到Falling#这一栏显示的数据28就是我们要的数据量了。



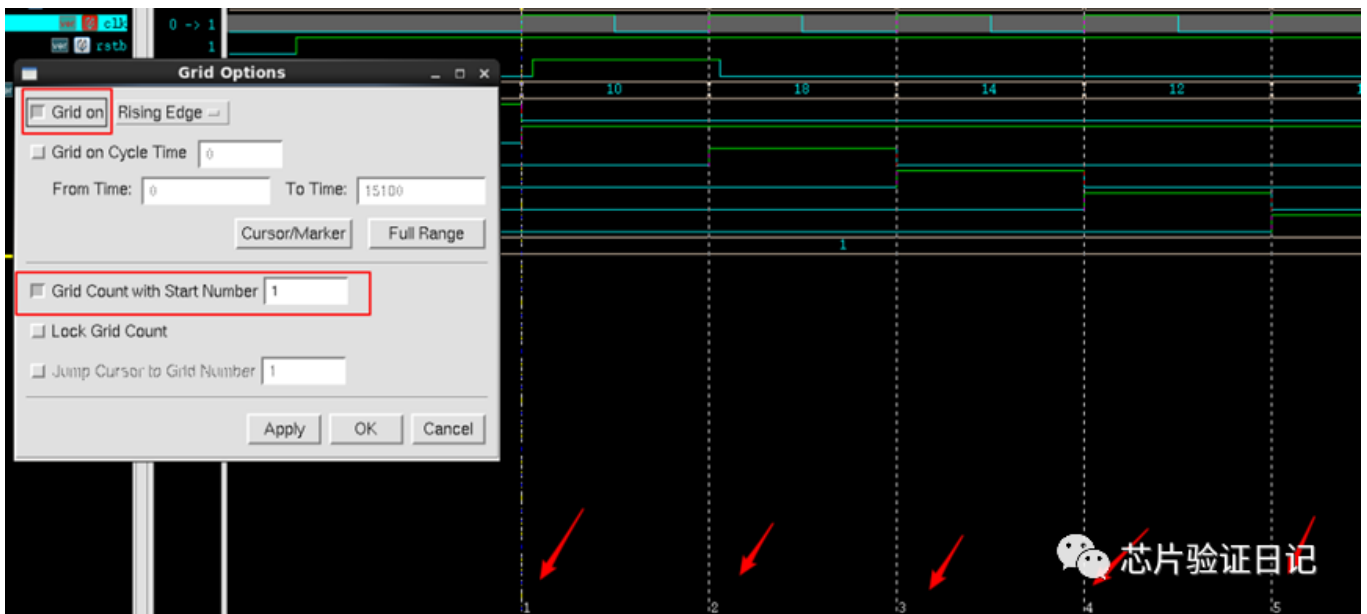
以上就是一段时间内连续有效数据的几个测量方法。debug的时候方法3最简单便捷，应用频率也较高；但是其他几个方法在特定的场景下也可以发挥出自己的作用，需要根据实际debug的时候灵活应用。

[2]网格法

点击菜单里面的view选项，选择Grid Options，如下图所示：

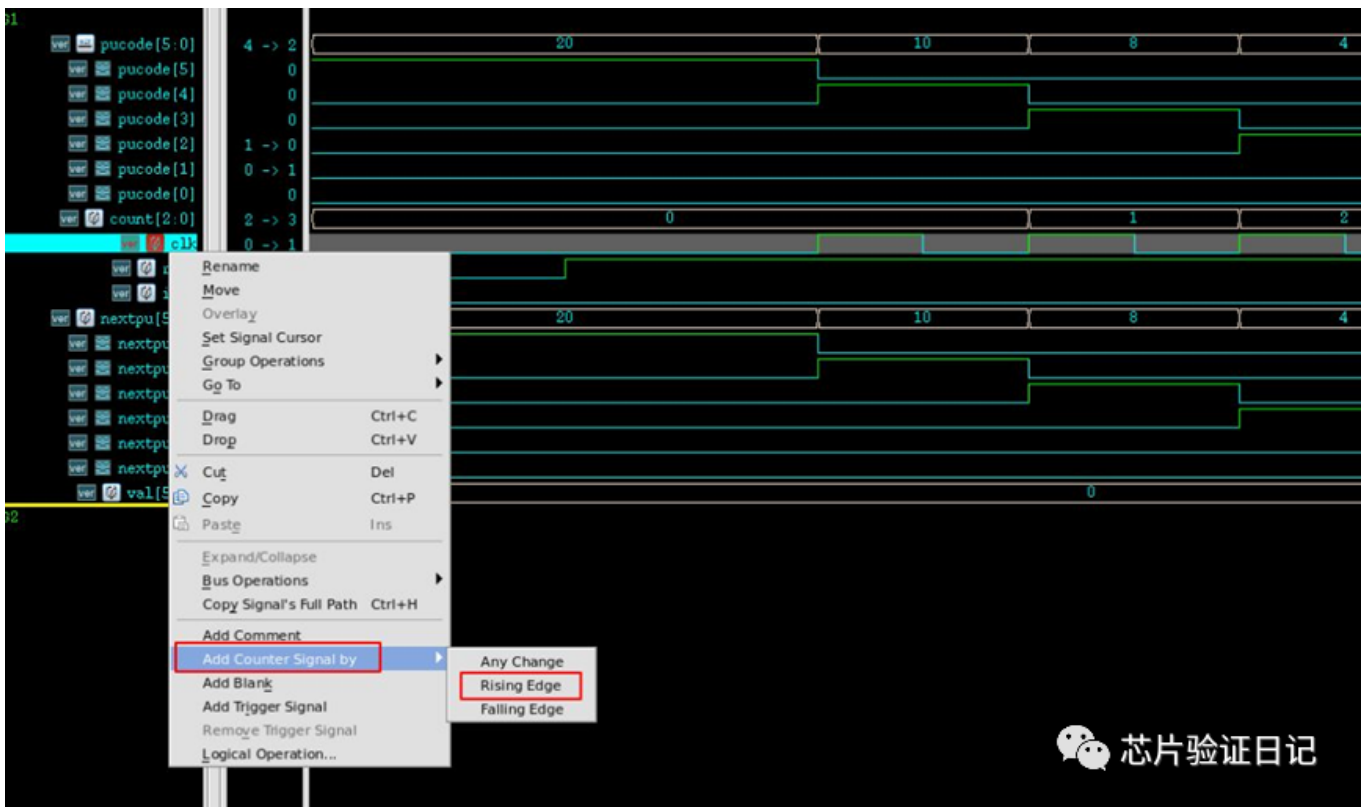


选中选项Grid on 以及选项Grid Counter with Start Number，如下图所示，点击Apply或者OK之后，就可以看到波形里面出现了网格，最下面有数字出现。这种方法还有一个好处是鼠标左键点到CLK信号的任意位置，计数都是从新的地方开始，具有很强的灵活性。



[4] 自动添加计数器:

右击CLK信号，选择Add Counter Signal by，选择上升沿、下降沿、任意沿中的一个，就会自动产生一个16进制的计数信号，不过它的计数起点是从CLK的0时刻开始，不能选择任意时刻，所以缺乏灵活性。



芯片验证日记

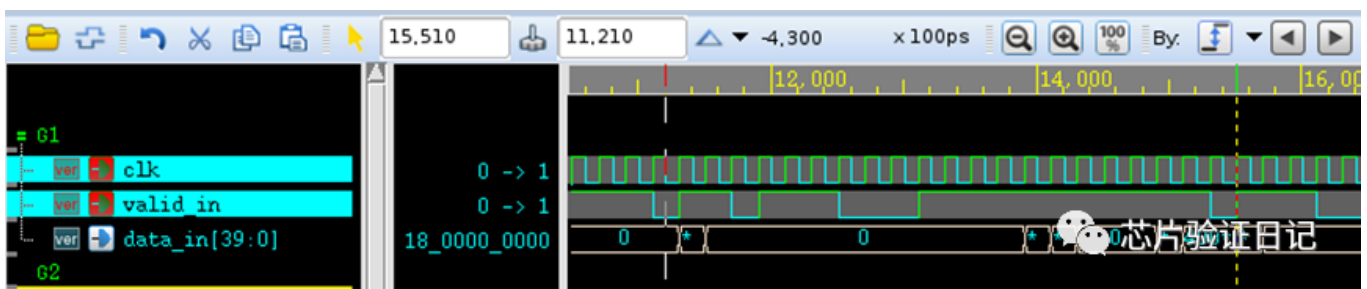
关于方法3和方法4，更推荐方法3网格法，看起来很强大，大家可以继续探索探索。

[5]打开路径下的verdi.cmd记录了上一次的全部动作。如果verdi意外关闭，可以使用 `verdi -play verdi.cmd`来复原。

[6]可以给clk的上升沿打上标记（好像叫label），在两个光标之间会显示标记的个数，也就是周期数。

2、非连续有效信号量测方法

实际debug过程中，更常遇到的场景是**非连续有效的信号**，如下图所示：CLK并不是在每个时刻都是有效的，这时我们又该如何比较快速准确的获知某个时间段内有效信号（CLK）个数呢？

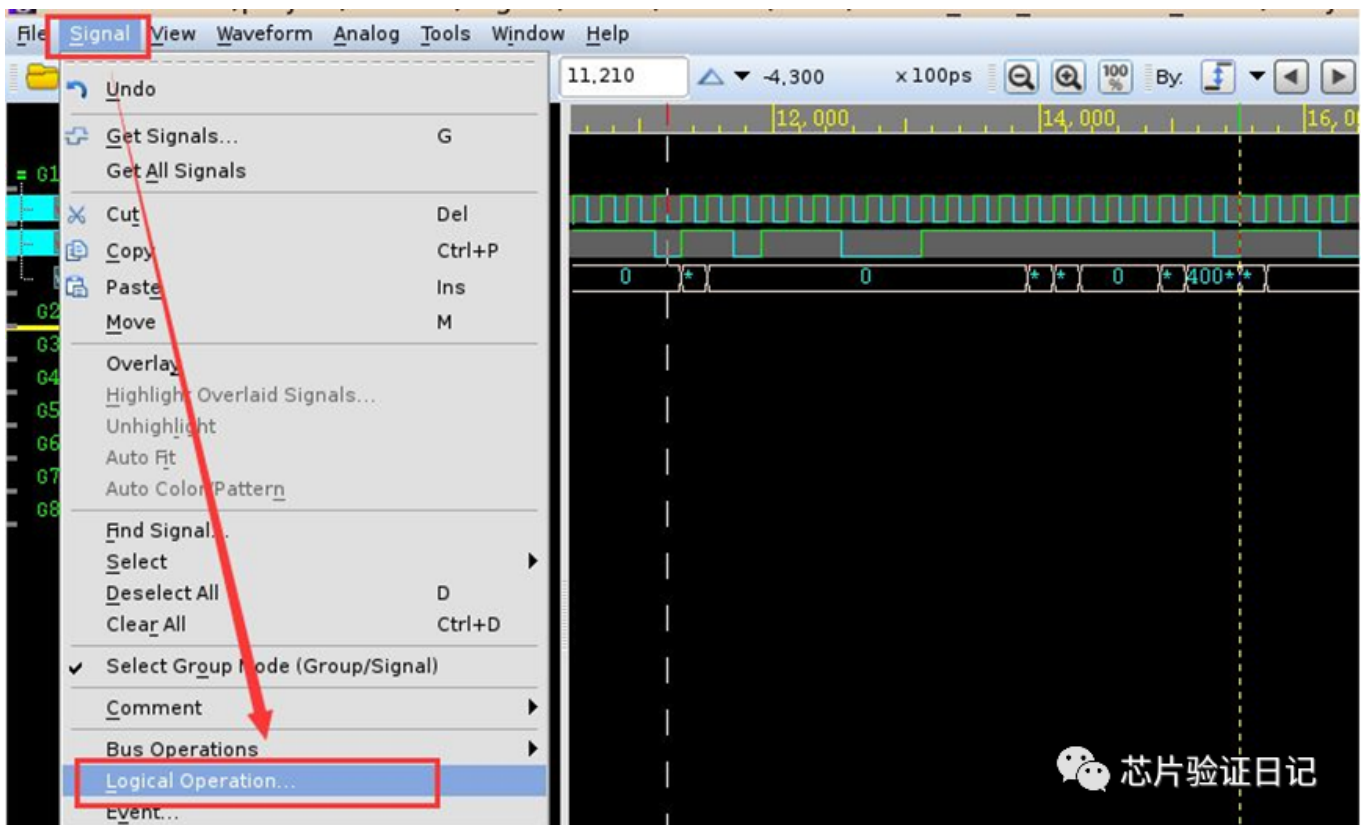


跟获取连续有效信号个数一样，如果某个时间段内有效信号个数较少（比如少于10个），直接手动数就可以了。当有效信号toggle次数比较多时，肉眼无法快速看出数量的时候，我们可以利用Verdi nWave里面的Logical operation 和Signal Event report 这两个功能来实现。

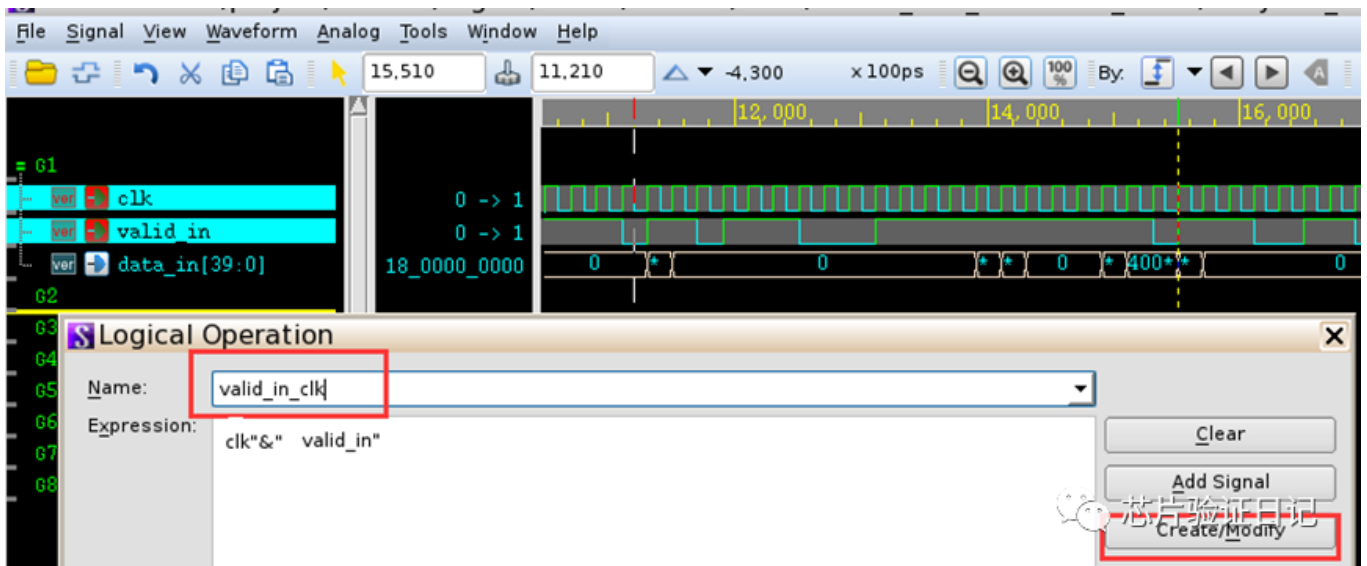
对于非连续有效数据传输，通常会有一个信号来标志哪笔数据是有效的，如上图中的valid_in信号为1时表示data_in数据有效。利用这个标志信号跟时钟进行“与”操作，就得到一个新的时钟。这个新的时钟只有在数据有效时才有，在数据无效时被遮掉了。再通过Signal Event report获知指定时间内新的时钟个数，就可以知道有效数据的个数了。

具体操作步骤如下：

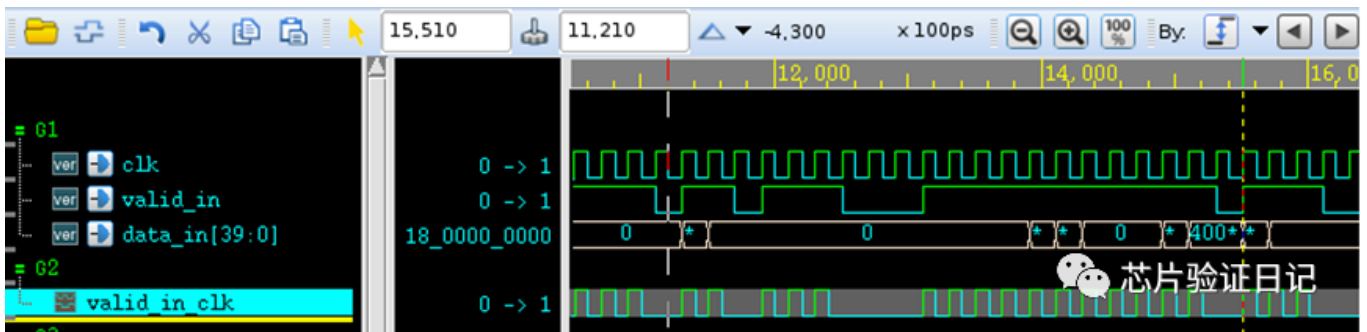
(1).鼠标选中时钟信号clk和数据有效标志信号valid_in;(2).点开nWave 菜单里面的Signal，再选择下拉菜单里的Logical Operation，如下图：



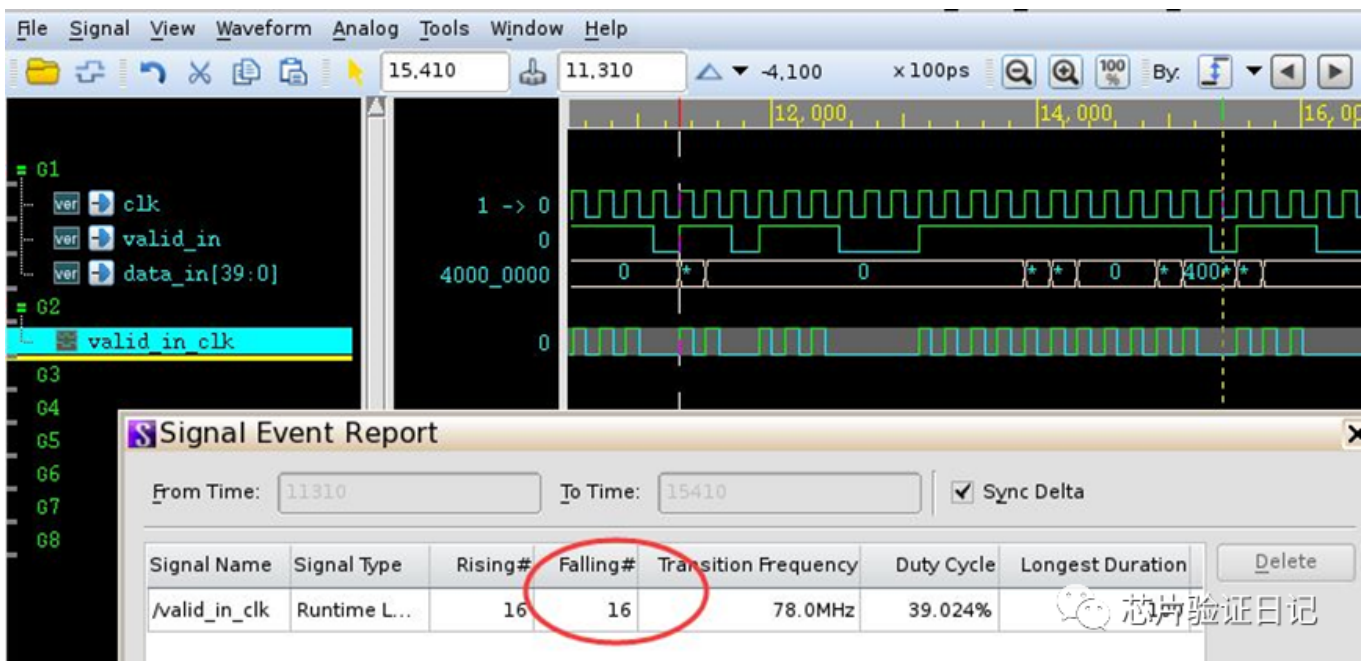
(3).在弹出的对话框里面，如下图所示，确认Expression里的信号操作是“&”；把Name改成valid_in_clk；然后点击Create/Modify按钮；



再回来看波形，如下图所示，就会发现多了一个信号valid_in_clk，这个信号就是我们需要的时钟信号。



(4)接下来就是Signal Event Report闪亮登场了。选中valid_in_clk,然后在View下拉菜单里面选Signal Event Report。如下图所示，在弹出的对话框里，找到Falling#这一列，里面列的数字16就是我们指定的时间段内有效的数据个数了。



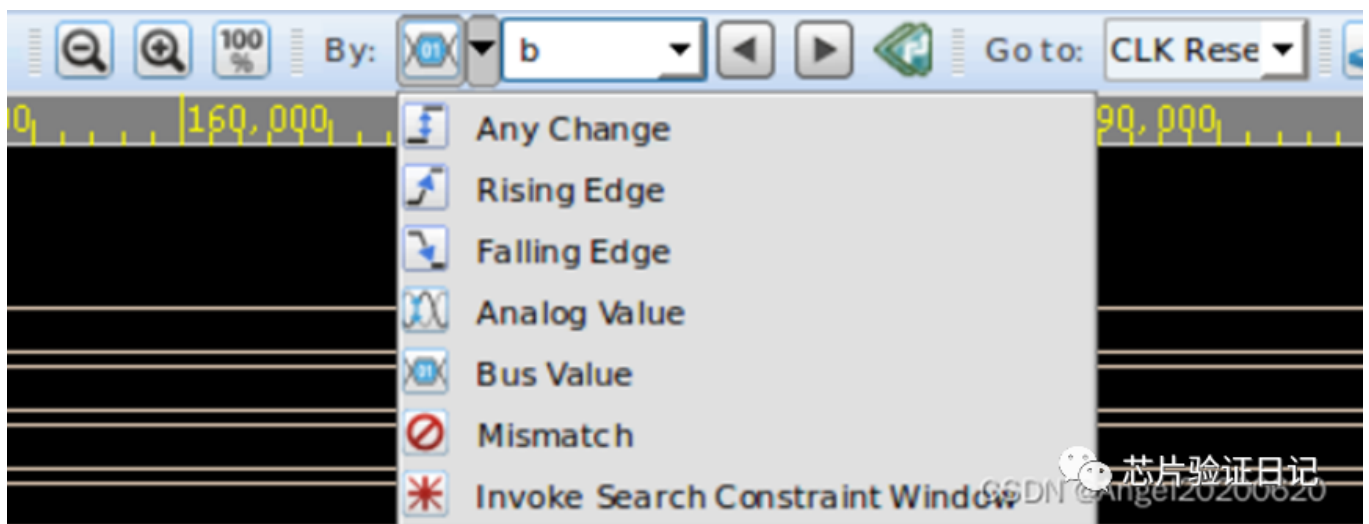
如果还是半信半疑的同学，可以手动数一数，看看Logical Operation加Signal Event Report这套组合拳的结果是否正确。

3、搜索查找

Verdi提供了很强大的搜索查找功能，充分利用好这些功能在IC设计/验证过程中可以大大提高debug 效率。下面我们就结合实际应用来介绍一下Verdi中常用的搜索查找功能以及一些使用技巧。

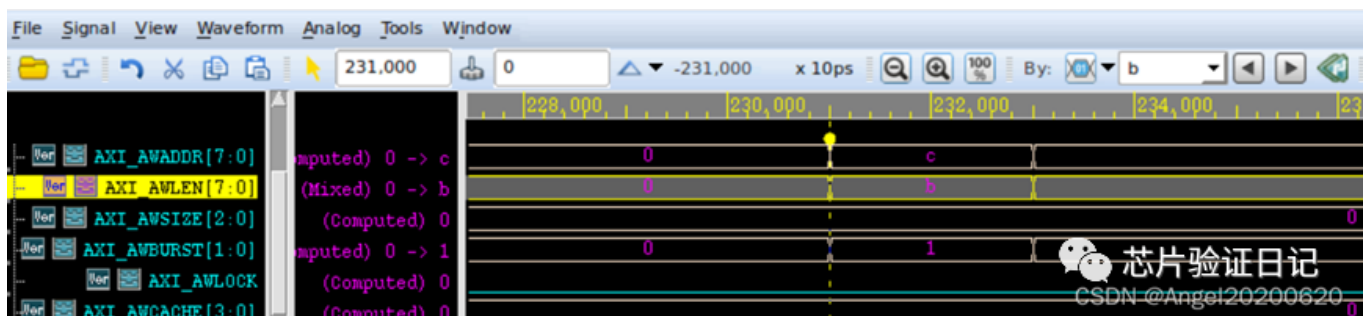
[1] 数值搜索、上升沿和下降沿搜索

进行数值搜索、上升沿和下降沿搜索

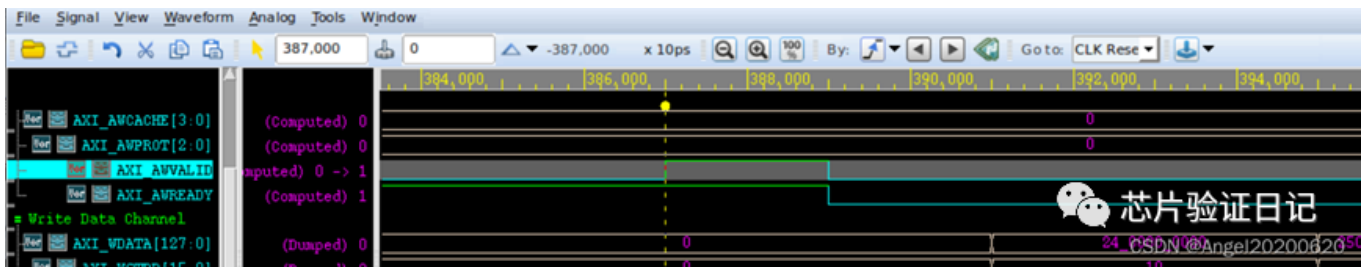


选中信号，按n向后搜索，N向前搜索；或者按左右小箭头进行查找搜索。例如：

数值搜索：



上下沿搜索：



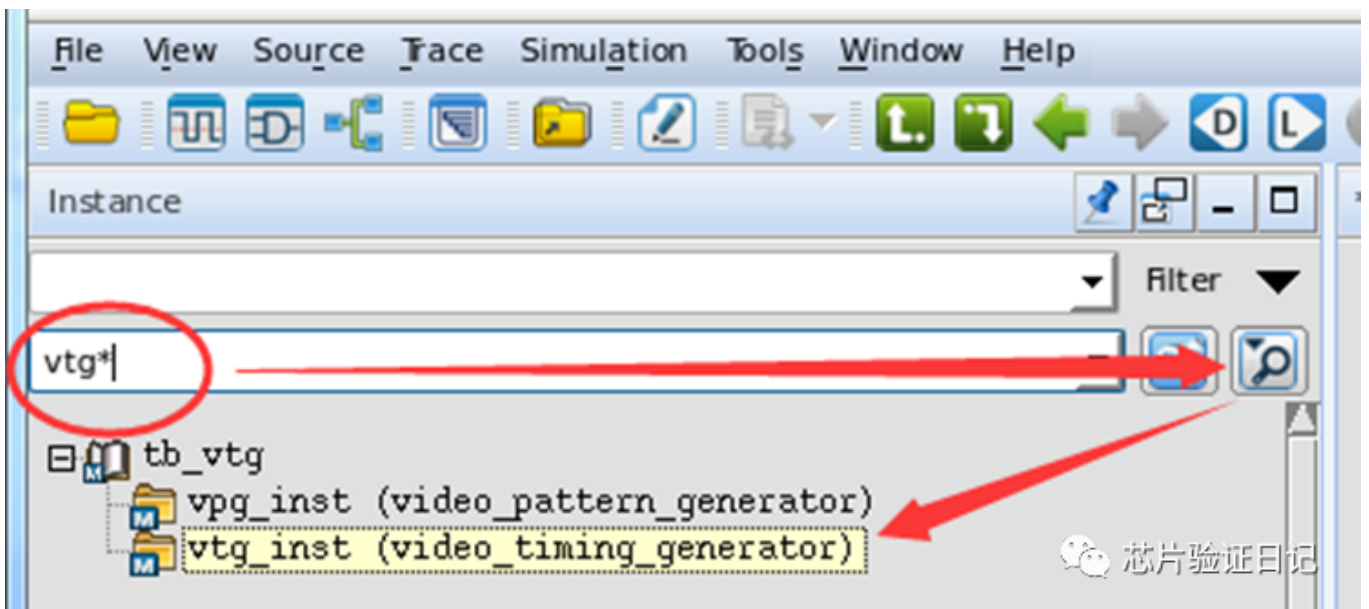
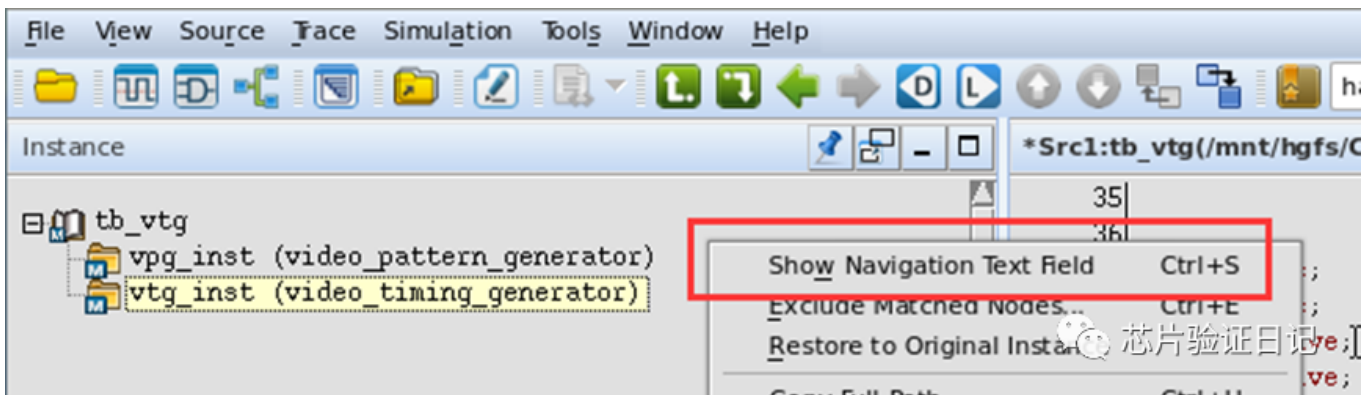
[2] 层次浏览器窗格中寻找实例 (instance)

在利用 Verdi debug时, 首先需要找到我们负责模块的实例 (instance), 再选择相应的信号来观察它们的行为来debug。当设计很庞大时, 要找到底层的一个小模块, 如果用鼠标从 top一层一层的往下点, 效率会比较低; 又或者我们只负责了一个小模块, 对整个设计的层级关系并不清楚的时候, 找到我们的底层设计会变得非常困难。

这时, 只要我们知道底层模块的实例名, 就可以利用Verdi的“Show Navigation Text Field”功能快速查找了。操作方法如下图,

a) 在Verdi的层次浏览器窗格 (Verdi界面的左部) 点右键, 弹出菜单里选“Show Navigation Text Field”或快捷键“Ctrl+S”

b) 在输入对话框里输入相应的实例名, 再点右边的搜索按钮就可以了。



[3] 搜索源代码

源代码窗格里提供了三种搜索指令，功能各不相同，用来应付不同的使用场景。

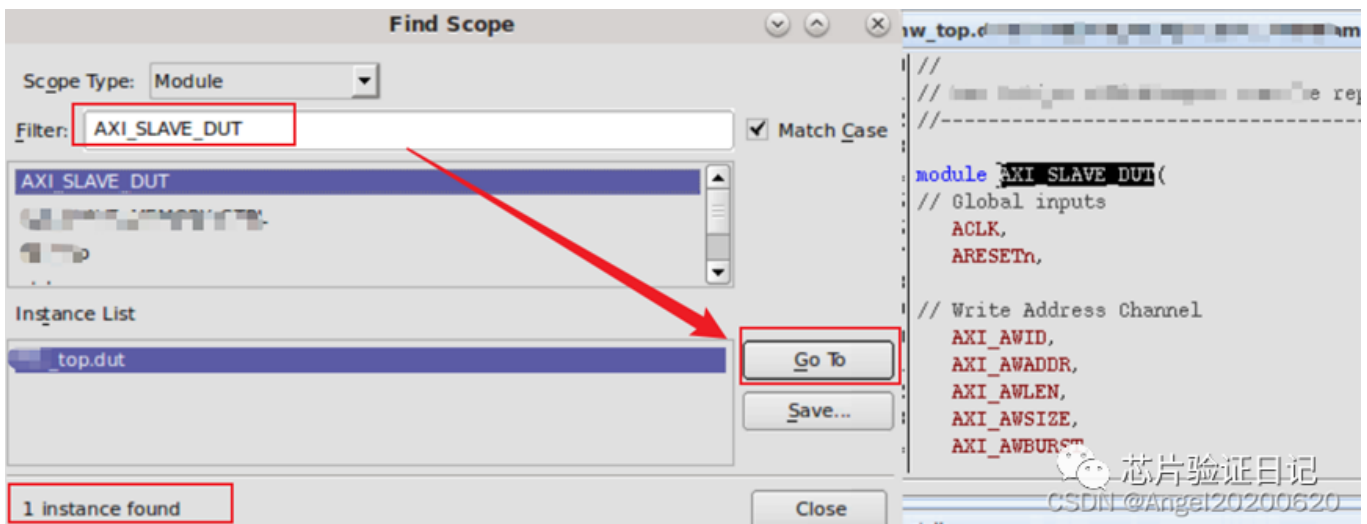
1) Find scope

Find scope适用于整个chip里面的查找。前面提到的应用场景如下：

A)Source -> find scope或者 按shift+S

B)在弹出对话框里Scope Type选module

Filter内填要找module关键字；Instance list 里面选择一个，点Go To，source code就会打开该module。



另外，对话框最下面列出了整个设计中找到的个数，可以知道该module被instance次数。

2) Find Signal/Instance/instport

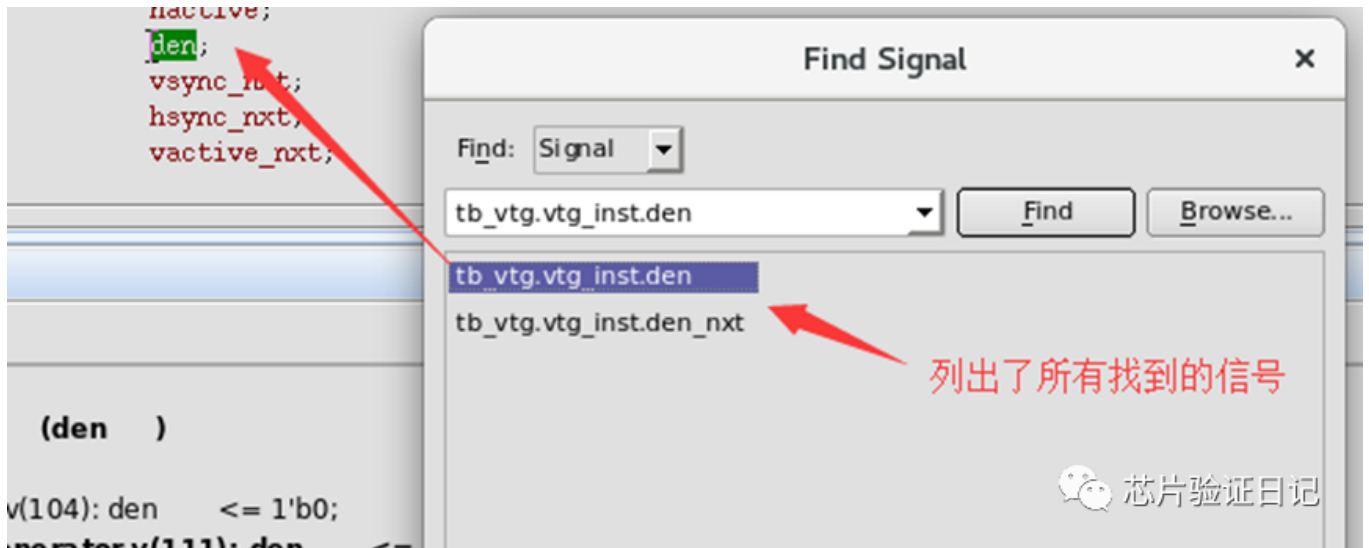
这个指令可以查找指定模块内的信号、实例或输入输出端口。这个指令的好处是可以把要查找的内容都列出来，想看哪一个就用鼠标点一下，就会切换到源代码窗格对应位置。当模块内要查找的signal (instance or input/output port) 比较多时，可以快速的找到想看的signal。

操作方法如下：

source -> Find Signal/Instance/instport 或者shift+A

选择要查找的类型：signal, instance or instport

输入框里面输入信号名，点Find按钮，下面会列出所有找到的信号,点击某个信号，就会到达source code 里面对应的信号



3) find string

Find string应该是debug 过程中使用频率最高的查找指令了。只要在输入框里面输入部分或全部要查找的字符串，再点一下find 按钮就可以了。

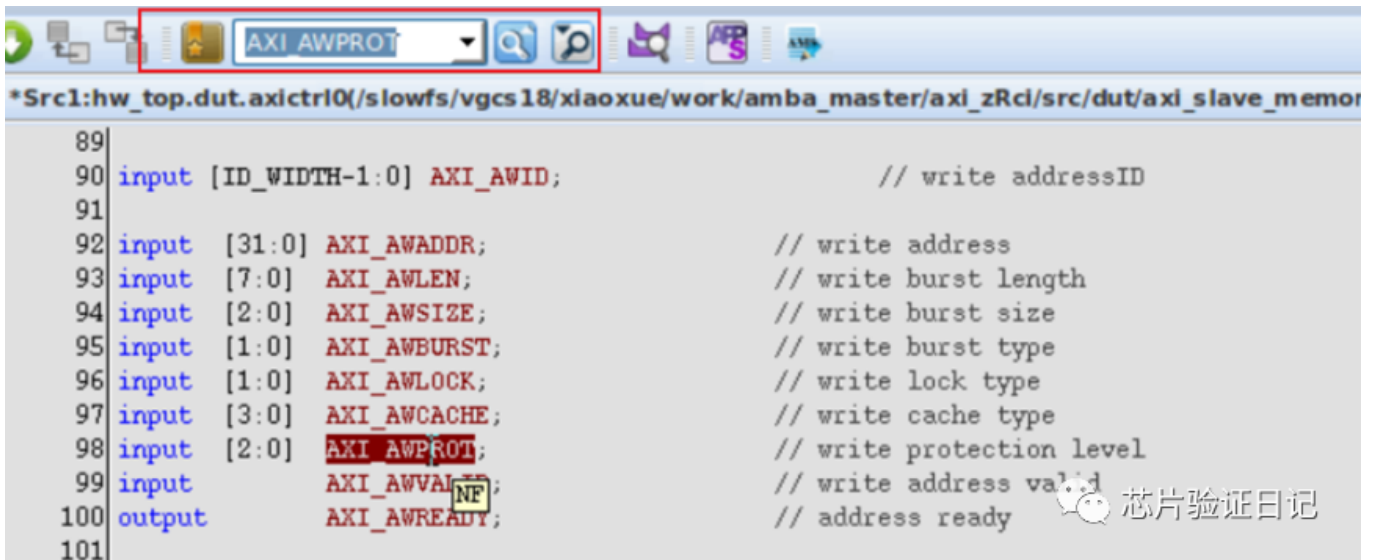
这个指令可以在当前文档（current File）或者全部文档（All file）里面查找，因为字符串查找匹配得比较多，要根据需要选择在当前文档或全部文档查找，以提高查找效率。

因为使用频率高，Verdi也很贴心的在默认工具栏里放置了Find string的输入窗口，如下：



输入栏输入字符串有几种方法

(1)手动输入，好处是可以只输入部分字符，不用全部输入信号名(2)点击信号，按中键拖入输入栏，然后点向上或向下查找，找同名信号(3)点击信号，按快捷键“Ctrl+F”，信号名就会自动输入到输入栏。这个快捷键很好用。

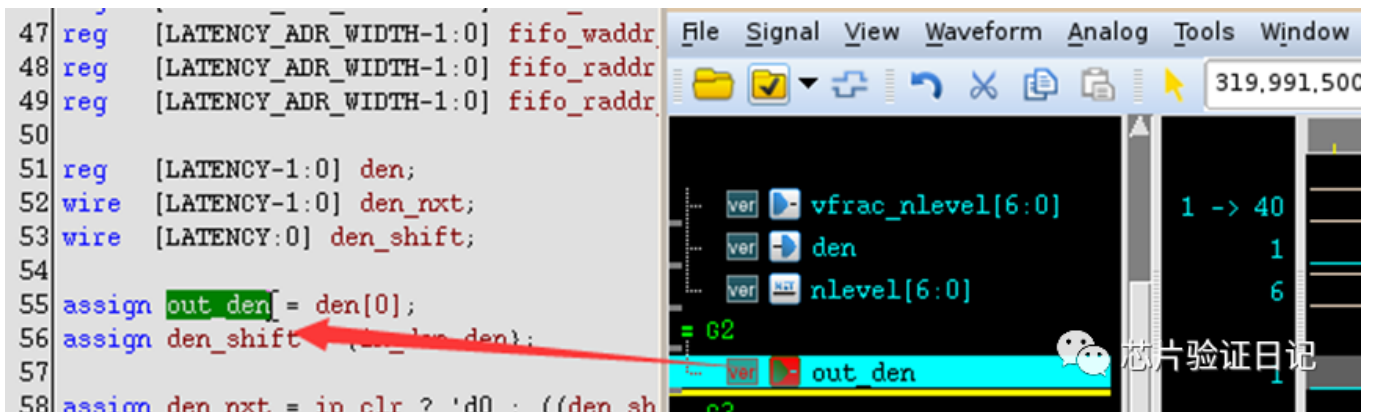


[4] nWave里面的信号追溯源代码

Debug过程中，往nWave 里添加信号后，查看了信号波形，如果看到异常往往要回头看一下源代码。如果一时找不到了，find string当然可以找得到，但是需要输入一串字符，可能就不那么乐意了。

其实Verdi很贴心，可以直接把nwave里面的波形拖到nTrace里面，可以快速的找到要找的信号。

操作方法：nWave里面选择信号，如out_den。按住中键，直接拖到source code plane，然后放开，就会显示你要找的信号了

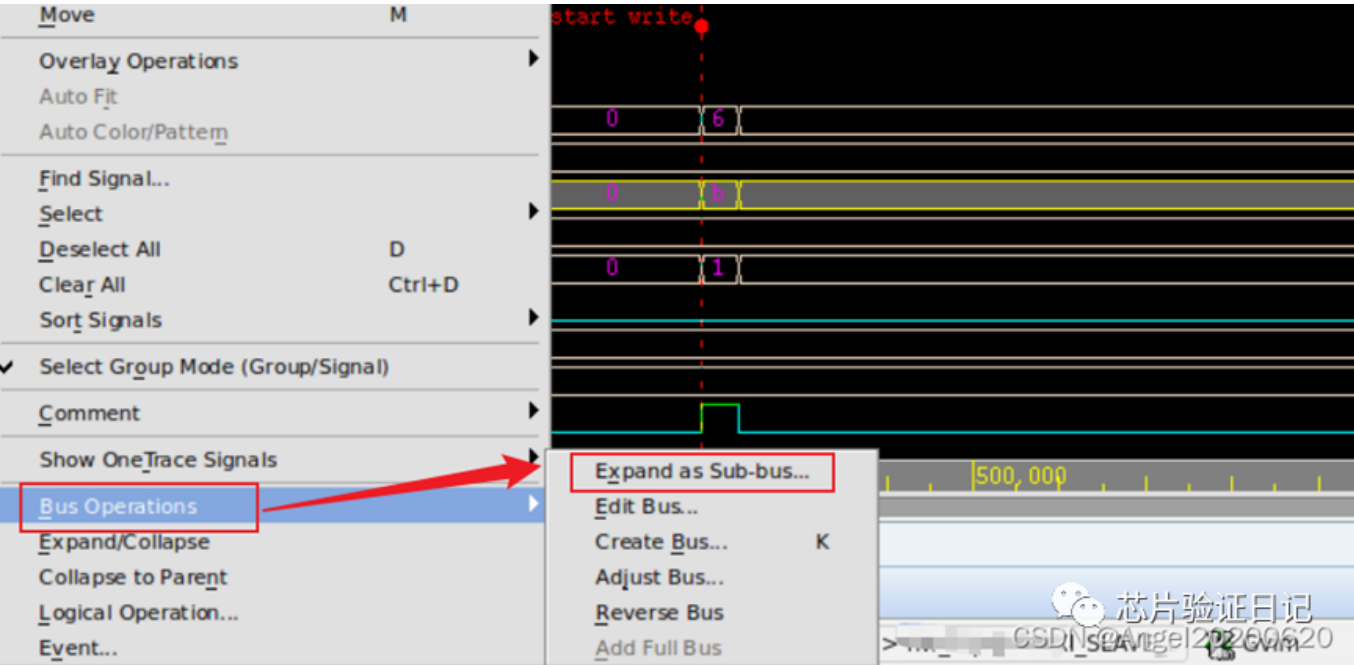


上述内容介绍了Verdi常用的几种搜索查找功能和技巧。希望对大家有帮助，debug效率更上一个档次。如果你有什么好想法，也欢迎后台留言一起讨论。

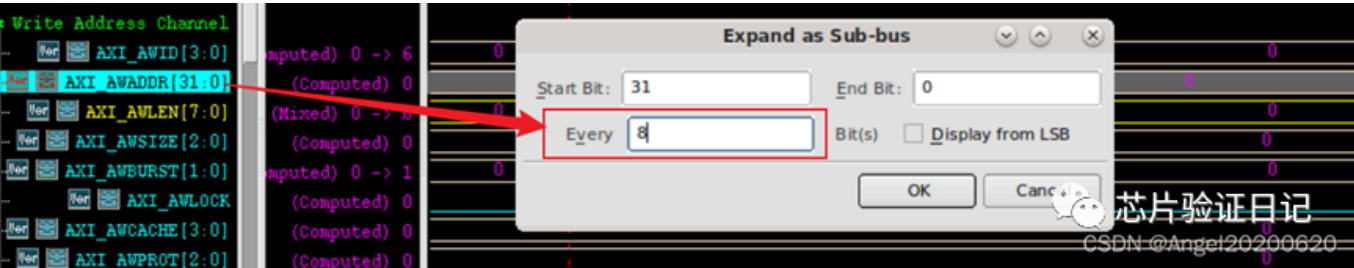
4、总线拆分

分析波形的时候有时候需要把一个多比特的bus拆分成几个位宽较小的bus，以方便观察数值。

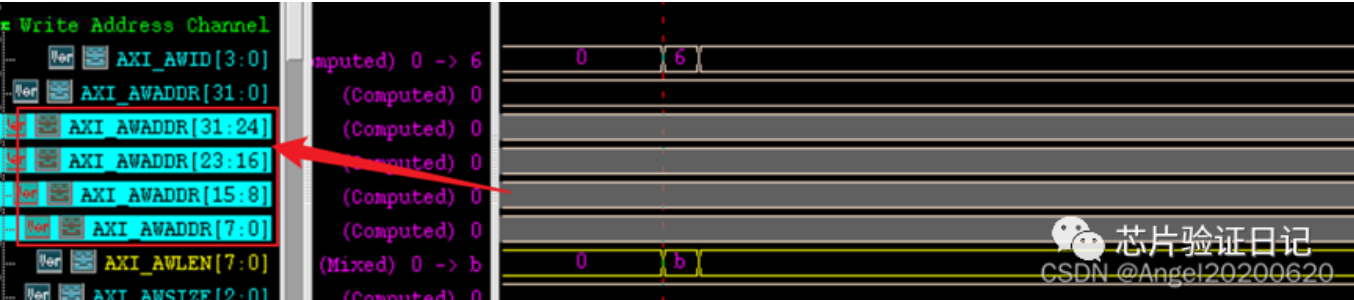
右键/Signal -> Bus Operations -> Expand as Sub-bus



将32bit分割为4个8bit bus.



分割结果：

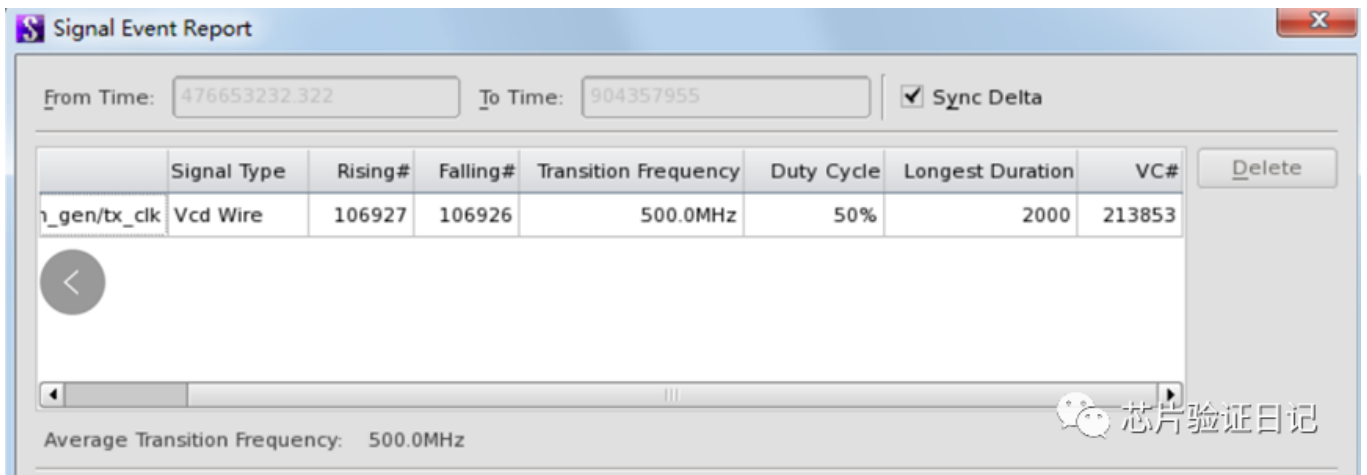


5、统计

统计出某个信号在特定时间段内的跳变的次数。这个结合下一节的逻辑运算，可以把一些复杂的波形分析用直观的形式表现出来。还是右键点击波形窗口左边的某个信号名字，在弹出菜单中选择Signal Event Report。



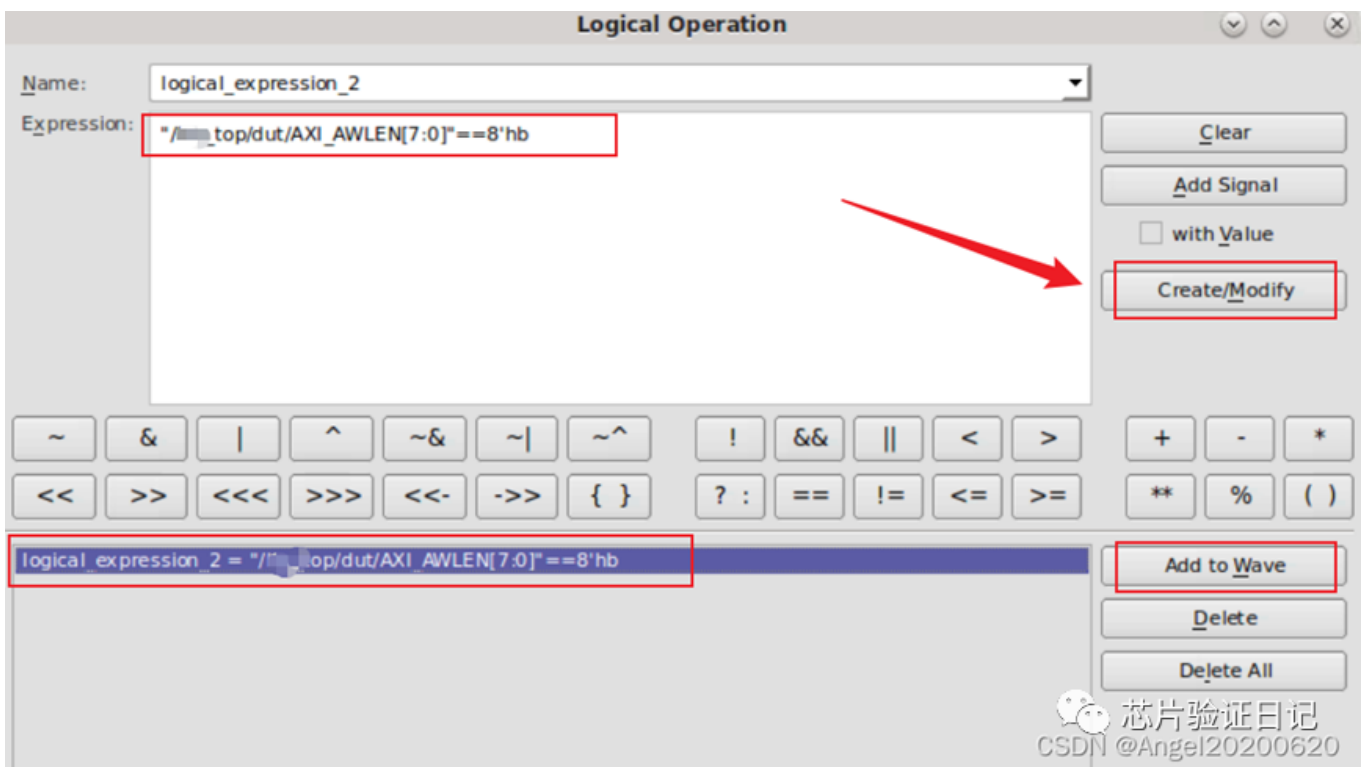
在弹出的对话框中，可以关注感兴趣的内容，比如Rising#，指的是默认Markers标定的时间段内，该信号的上升沿次数。这样就不用很辛苦的放大波形，用眼睛一个一个去数了。



6、逻辑运算

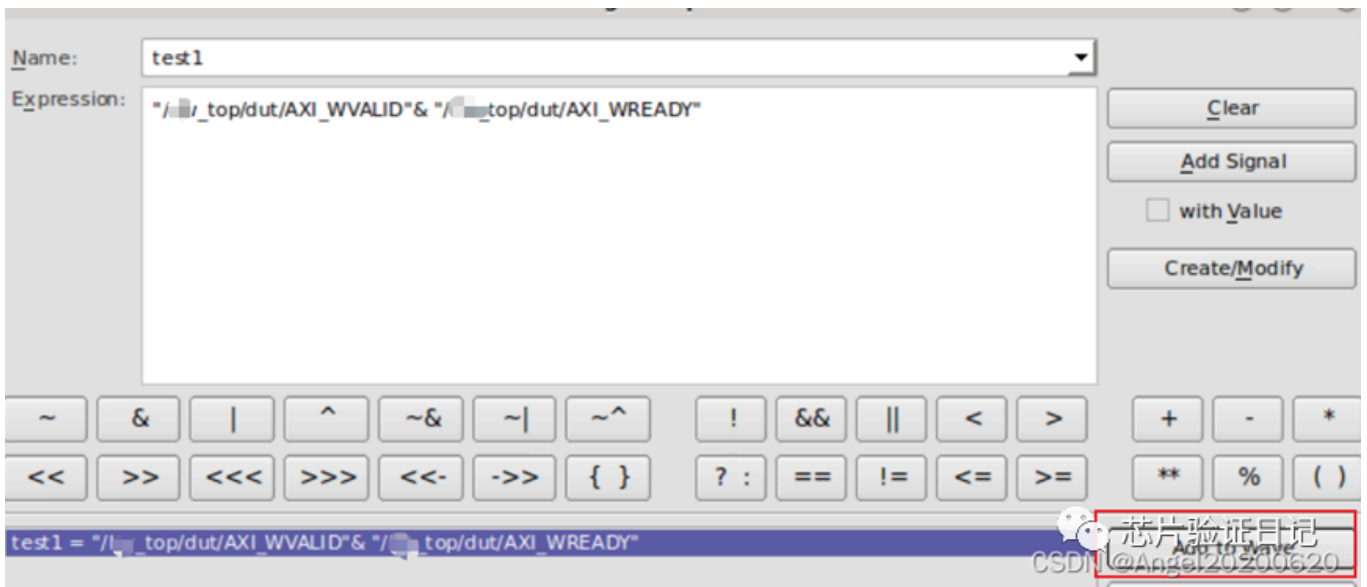
在一个频繁变化的信号、尤其是多比特的bus信号中观察是否出现了某个值，有时候可以通过搜索来追踪。也可以通过对现有信号执行的逻辑操作创建新信号，或者搜索信号值。但是如何一目了然地看到呢？右键/Signal -> Logical Operation，通过新建一个信号，该信号是待观察信号的某种逻辑表达式。

数值搜索



逻辑操作

例如：两信号进行与运算，也可以进行其他运算。



例子中作为只读的`default_file`文件，可以放一些自己习惯或者喜欢的设置。比如`tab`等于4还是8，各种界面的字体，以及其它一些每次启动Verdi都希望生效的设置。`working_file`用来保存本次使用更改的设置，如果有想长期预设的，可以从中copy出来到`default_file`，这样方便升级迭代。

通过如下常规的参数调用

```
-rcFile some_path/some.rc
```

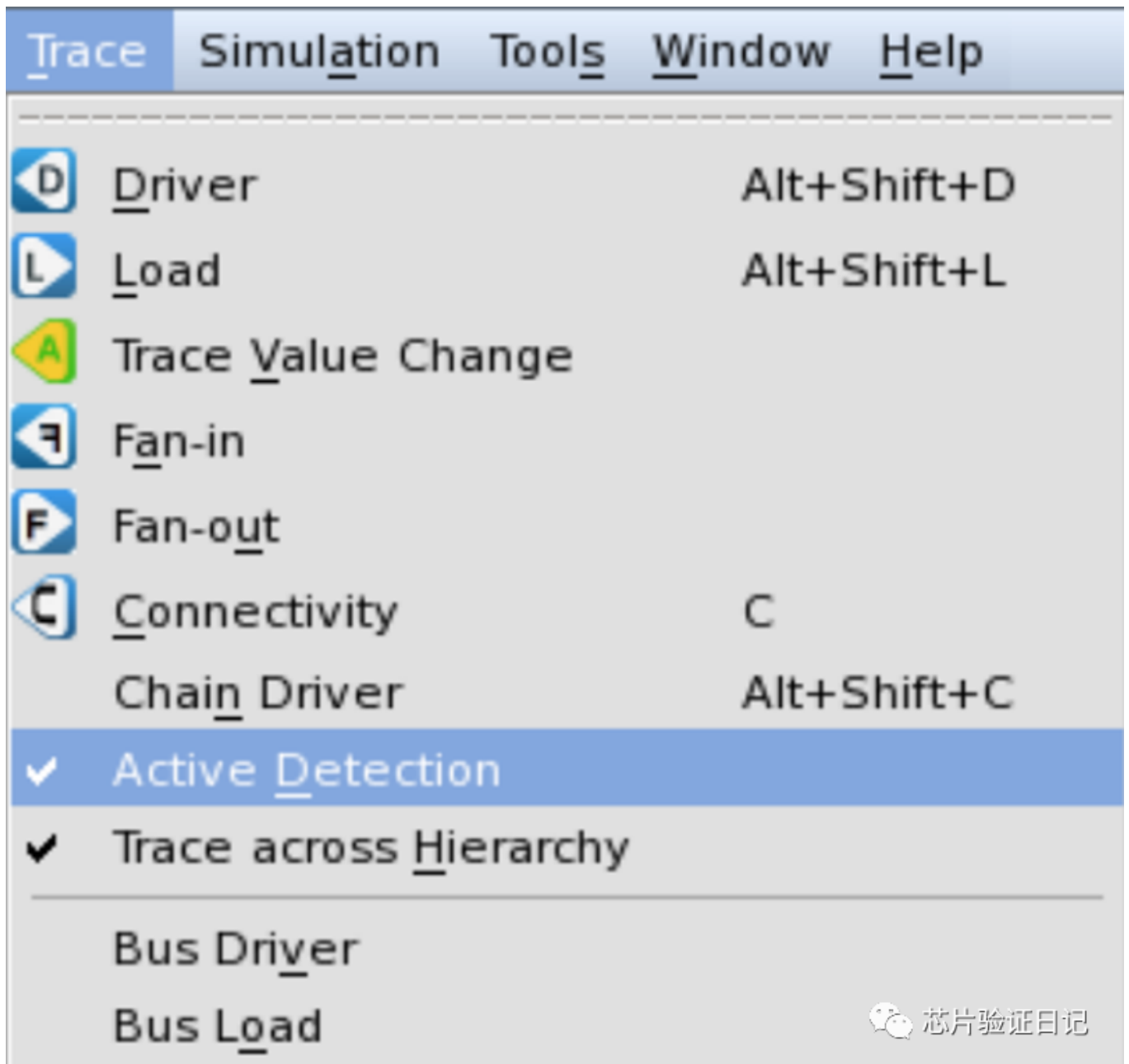
也可以读入一个rc文件实现类似预设功能，不过这种方法的缺点是这个rc文件会被当前调用覆盖。后果就是下次启动的Verdi会带有上次使用的痕迹，比如打开波形文件的目录位置。用的次数多了，不胜其烦。干脆用`manage.rc`来彻底的解决。

另外，亲测命令行键入 `-managercFile` 的方式也可以通过设置环境变量来替代，如在`.cshrc` 中添加

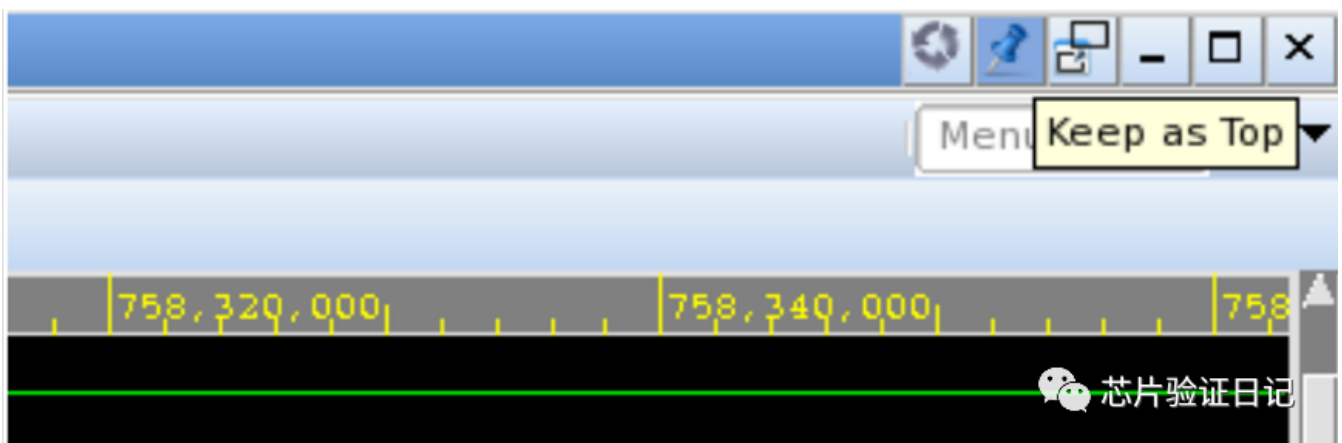
```
setenv NOVAS_MANAGE_RC_FILE some_path/manage.rc
```

8、Trace代码过程中波形窗口保持前置

不知道从哪个版本开始，Verdi默认打开了`Active Detection`，Trace窗口在trace过程中会被激活前置，造成波形窗口被遮挡，也是不胜其烦。当然这也属于个人喜好问题。



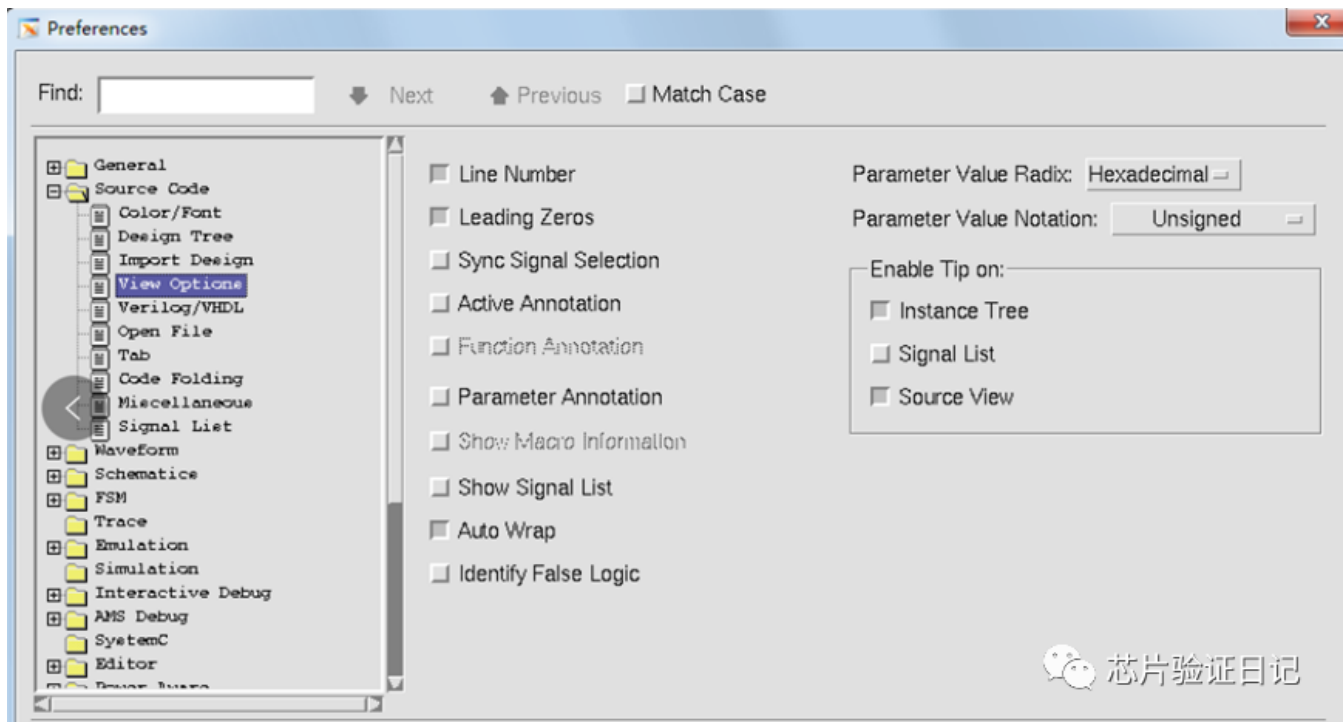
那如何保持波形窗口始终在前呢？首先需要在Trace下拉菜单中取消 **Active Detection**，然后点击波形窗口右上角的 **Keep as Top**。



这样在代码Trace时，波形始终可以看到，方便debug。Active Detection可以在rc文件中预设。Keep as Top还没有找到预设的方法。

9、Parameter 提示的默认进制

在Verdi中，如果把鼠标放到Verilog代码中的parameter上，可以提示其值是多少。因为各种原因，有时候会以二进制的形式提示，如果这个parameter是32比特，那把这个二进制看明白也太费眼睛了。简单的话还是16进制比较易读。这个可以在下图中的Parameter Value Radix中设置。当然也可以把它加入rc文件进行预设。



10、使用 Verdi 分析 NBA Delay 的问题

NBA 就是 NonBlocking Assignment，非阻塞赋值 的缩写。它通常用来描述一个用时钟沿触发的寄存器。在 Verilog 普及早期，人们通常会在 NBA(Nonblocking Assignments) 的<= 之后加个单位延时#1来解决早期仿真器行为不一致的问题。

所谓习惯成自然，NBA Unit Delay 在某些设计中一直沿用下来，那么如何去理解这个延时的作用呢？SNUG 上的知名作者 Clifford E. Cummings 在 2002 年有篇文章阐述的挺详细（Verilog Nonblocking Assignments With Delays, Myths & Mysteries）下面给出一些代码片断，同样感兴趣的朋友可以自己完善并用 VCS 仿真生成波形，然后用 Verdi 打开。

首先创建三个时钟，注意两个二分频时钟的创建方式不同。

```
1 always #(10/2) clk = ~clk;
2 always #(20/2) clk_div2_direct = ~clk_div2_direct;
3
```

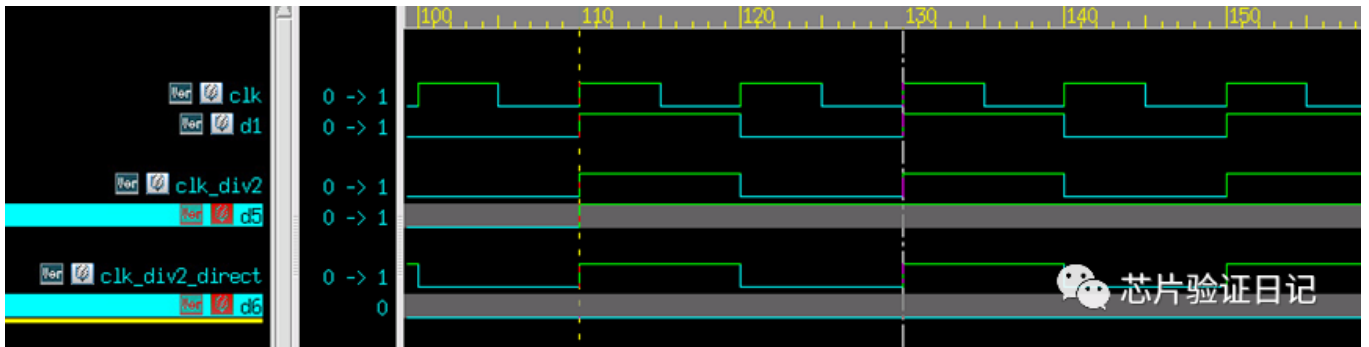
```
4  always @ ( posedge clk or negedge rst_n ) begin
5      if ( ~ rst_n ) begin
6          clk_div2  <= 1'b0;
7      end
8      else begin
9          clk_div2  <= ~clk_div2;
10     end
11 end
```

然后用 clk 触发一个不停翻转的寄存器 d1，再分别用两个二分频时钟去采样，保存在 d5 和 d6 中。

```
1  always @ ( posedge clk or negedge rst_n ) begin
2      if ( ~ rst_n ) begin
3          d1  <= 1'b0;
4      end
5      else begin
6          d1  <= ~d1;
7      end
8  end
9
10 always @ ( posedge clk_div2 or negedge rst_n ) begin
11     if ( ~ rst_n ) begin
12         d5 <= 1'b0;
13     end
14     else begin
15         d5 <= d1;
16     end
17 end
18
19 always @ ( posedge clk_div2_direct or negedge rst_n ) begin
20     if ( ~ rst_n ) begin
21         d6 <= 1'b0;
22     end
23     else begin
24         d6 <= d1;
25     end
26 end
```

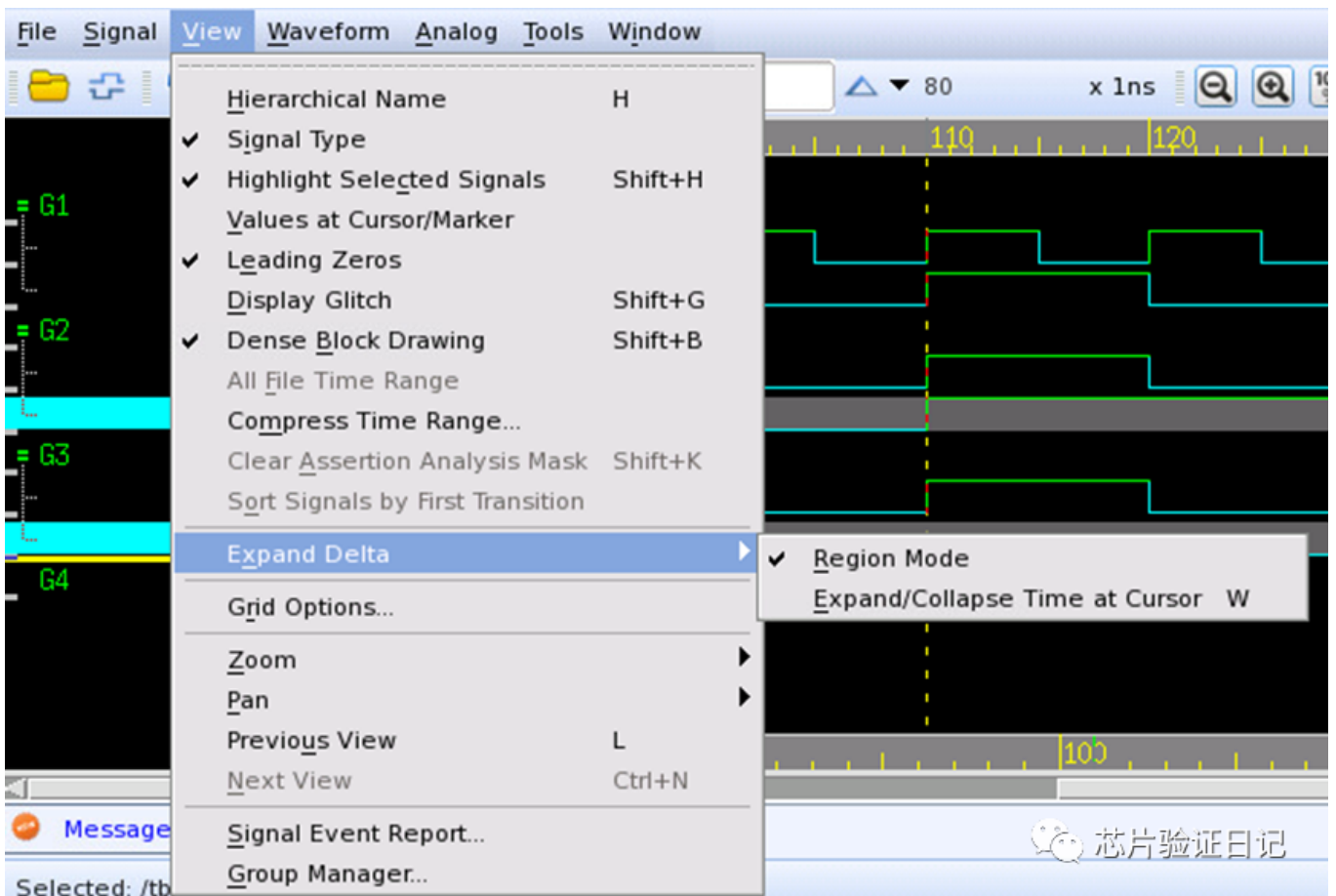
用 VCS 仿真的时候，加上编译时选项+fsdb+region和运行时选项+fsdb+delta。

然后用 Verdi 打开生成的 fsdb 波形。正常的话，可以看到 d5 和 d6 的波形是完全不同的，一个是常0，一个是常1。

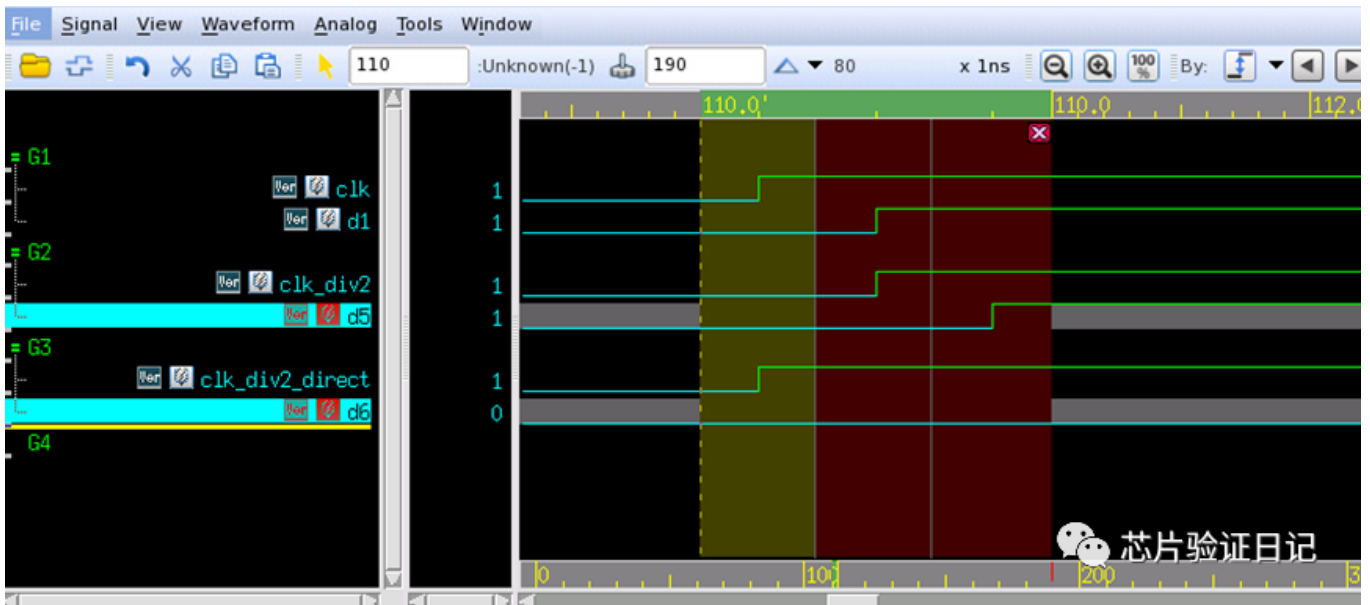


只从上图分析的话，两个二分频时钟的波形是完全一致的，被采样的数据是同一个，那么采到的数据也应该是相同的。但我们知道这两个二分频时钟的生成方式是不同的，那么不同在什么地方呢？

在 Verdi 的 nWave 窗口中选中 View - Expand Delta - Region Mode，把 Cursor 移动到二分频时钟的上升沿，然后点击Expand/Collapse Time at Cursor，简单点的话直接按 w 键。



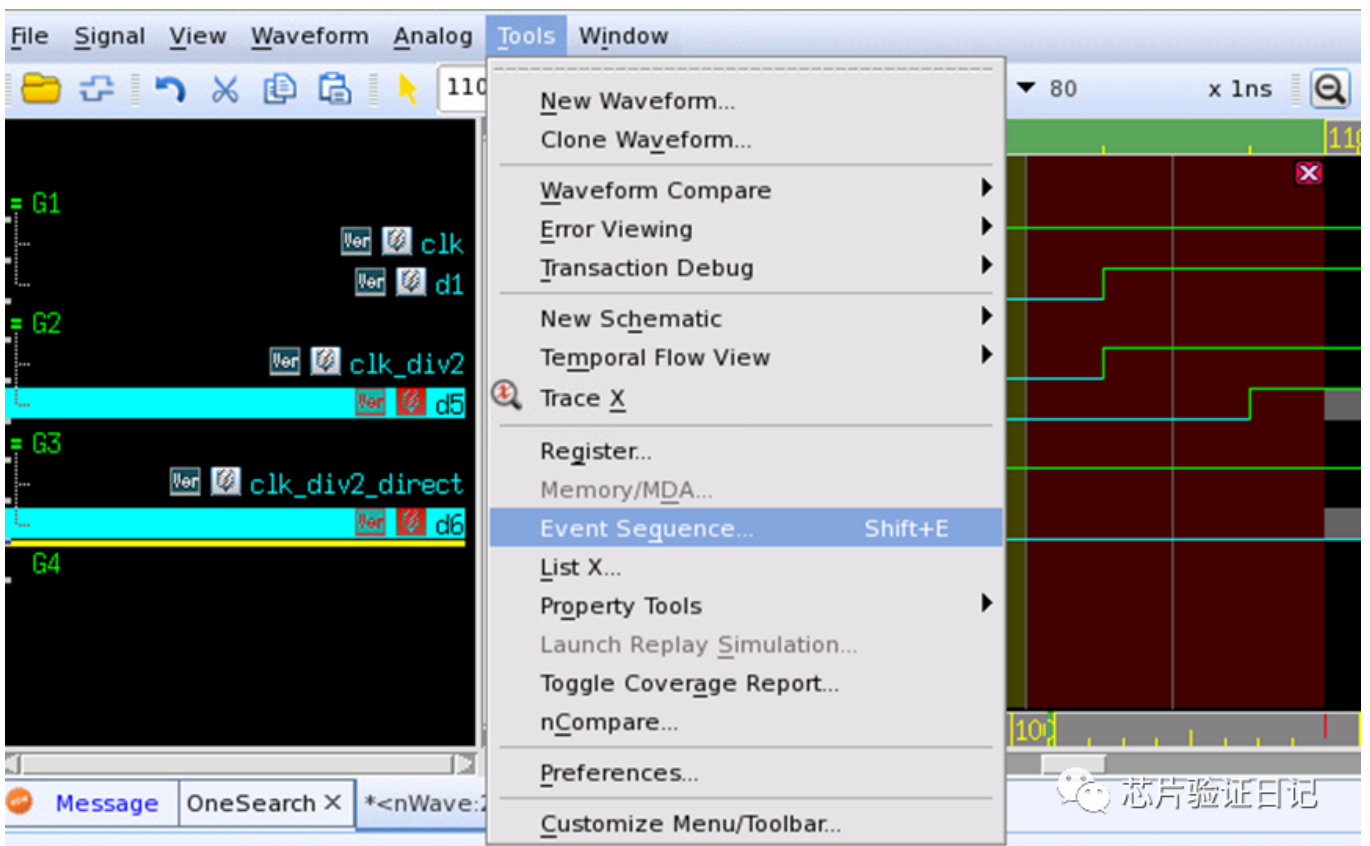
然后就可以看到这个仿真时刻的 Simulation Region，里面展示了各个 Event 和 Region 的关系。



这个例子中，只存在绿色的Active Region 和 红色的 NBA Region。在某些glitch debug的例子中，还能看到Re-Active Region和Re-NBA Region。

Region的展示可以帮助我们很好的理解Verilog 规范中的定义的信号调度。

此外，Verdi还可以通过点击Tools - Event Sequence来显示图形化的Event序列。

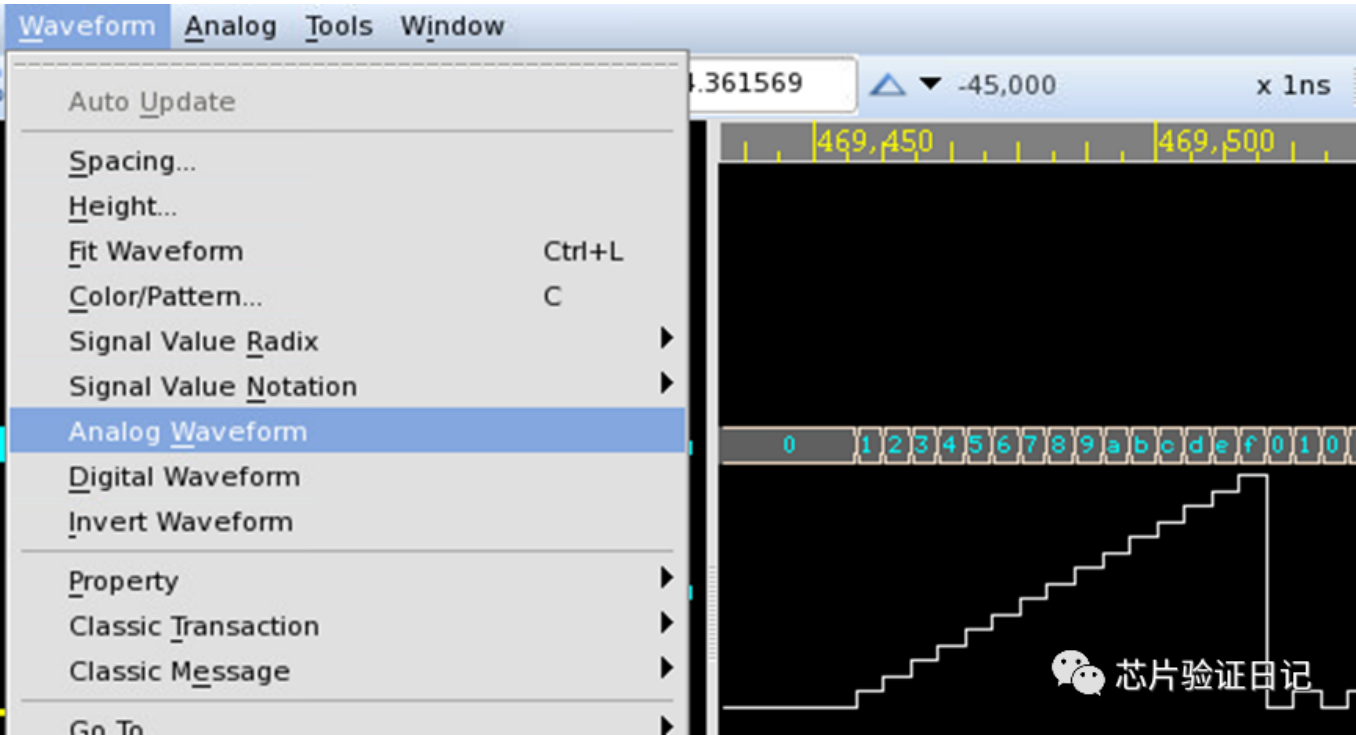




11、数字信号的模拟化呈现

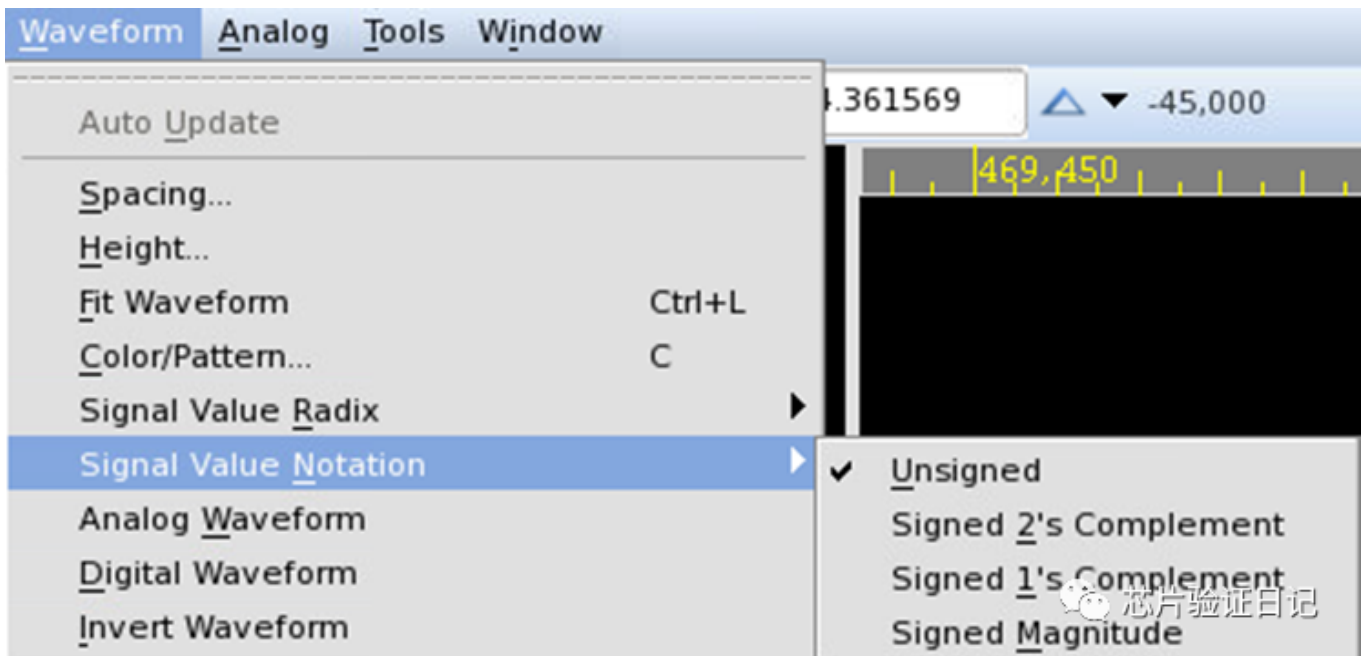
Verdi 提供了既简单又直观的操作，去绘制数字信号的模拟变化趋势。尤其是在观察周期性计数的数字信号时，例如调制频率的变化，还有 FIFO 读写地址的变化，这种方式基本是可以做到一目了然的。

操作方式：nWave 窗口菜单 - Waveform - Analog Waveform



对于 Verdi 这个工具来说，它对于数字信号本身的含义目前还没有智能到可以自动识别。所以我们需要指定特定数字信号的表示方式（Notation），是无符号、二补码还是补码等等。

操作方式：nWave 窗口菜单 - Waveform - Set Notation

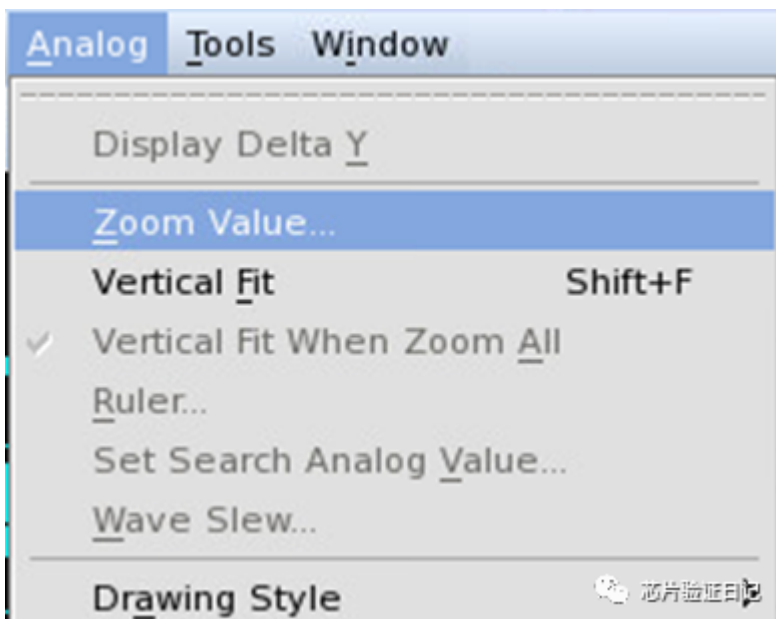


12、手动调节模拟波形的显示范围

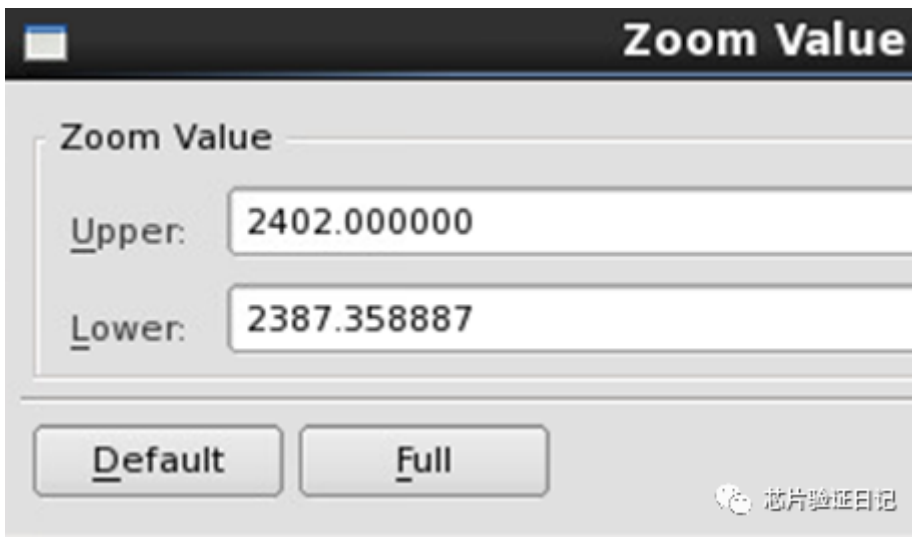
还有一个可能遇到的局限性就是模拟波形的显示范围，Verdi 在绘制数字信号的模拟波形时，默认会按照整个波形的动态范围来绘图。如果一个数字信号的初始值和工作时实际的动态范围差别很大，那么观察实际工作时的模拟波形时，波形变化的幅度相比整个动态范围就会太小，无法明了的看到小动态范围的变化趋势。

还好Verdi提供了手动调节显示范围的功能。

操作方式：nWave 窗口菜单 - Analog - Zoom Value...



根据需要调节上限和下限，调节过程中波形是实时更新的。



对比效果如下图，第一个信号是默认范围的，动态变化缩成一条直线了，没有观察价值。第二个信号是调整了范围的，可以清楚的看到变化趋势。第三个是原始的数字信号。



13、自动调节：切换 FSDB 文件

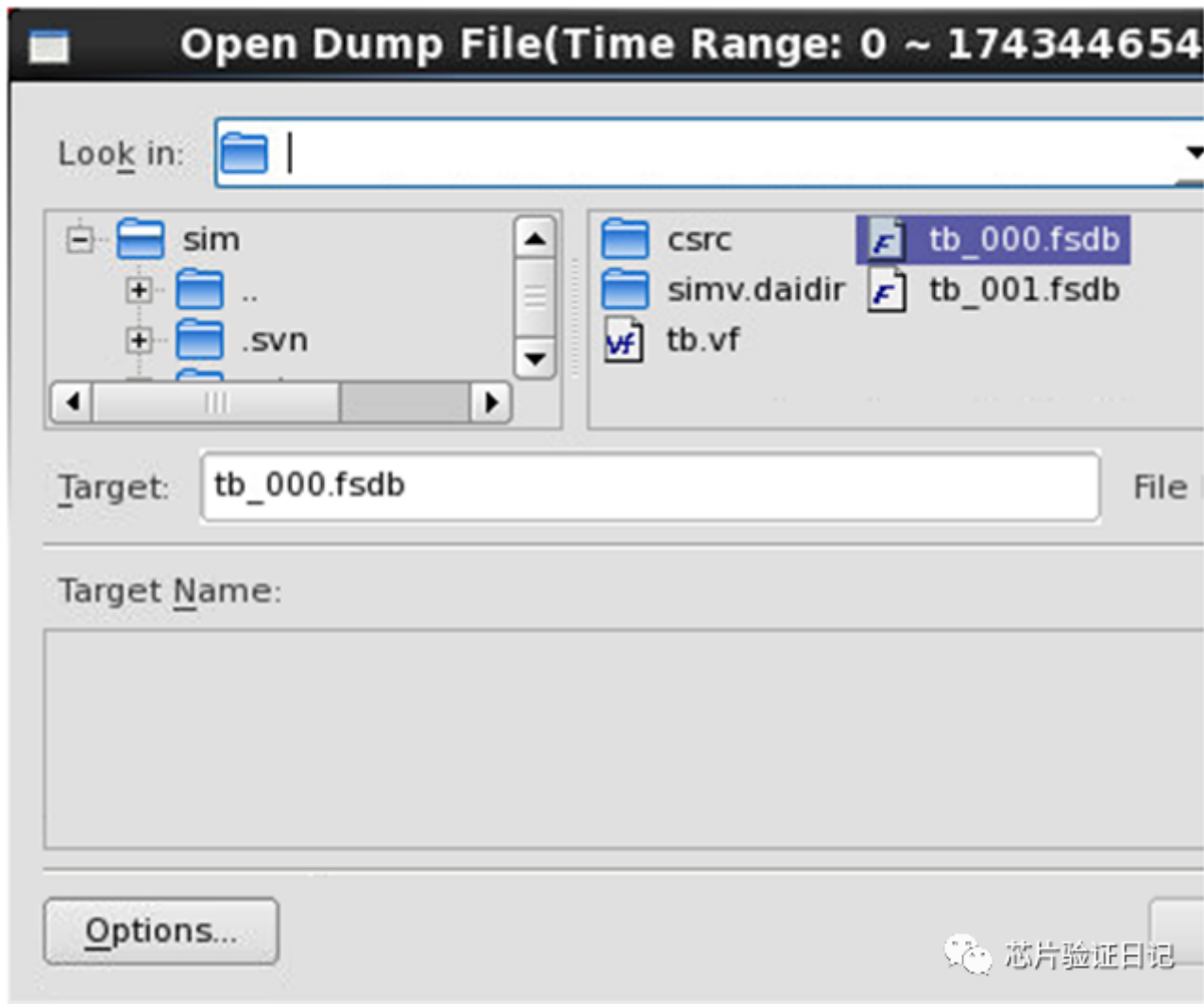
还有一个比较笨的解决方法是，写出FSDB 文件时，根据特定信号，从数字信号开始动态变化的时候，做一个文件切换。这样就把数字信号初始化部分的波形分离出去。当我们着重分析动态变化的部分时，就不需要加载初始化部分的波形。Verdi在绘制模拟波形的时候，只会看到小动态范围的数值，绘制出来的变化趋势就足够清楚了。

```
1 $fsdbDumpfile("tb_000.fsdb");
2 $fsdbDumpvars(0, tb);
3 @(posedge some_trigger_signal);
4 $fsdbSwitchDumpfile("tb_001.fsdb");
```

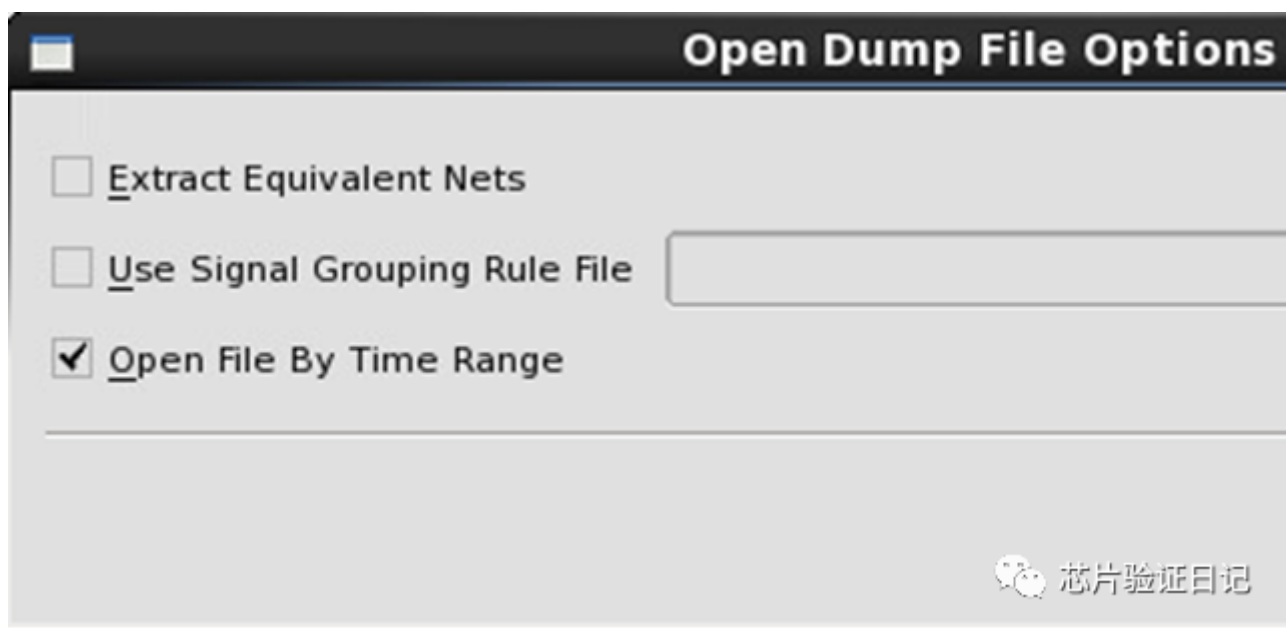
14、自动调节：加载特定时间范围

当然作为团队项目的螺丝钉，有时候不方便控制FSDB 文件的产生方式，那么在加载FSDB 文件的时候，可以选择指定一个时间范围。感谢读者 @Erik Li 提供的这个技巧。

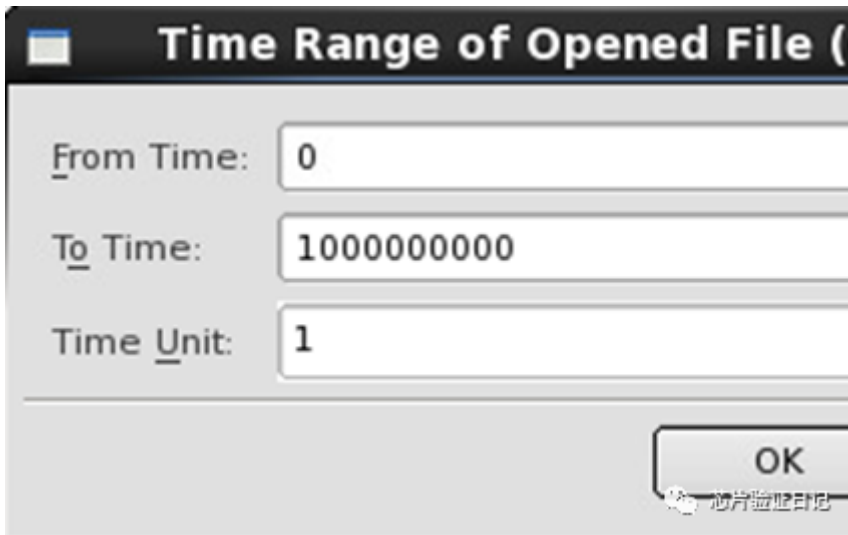
操作方式：nWave窗口菜单 - Open，选择 FSDB 文件到 Target Name 后，点击左下方的 Options... 按钮。



在弹出的对话框中选中 Open File By Time Range，然后点击 OK 按钮。



再点击 OK 按钮，在弹出的对话框中就可以选择感兴趣的时间起止范围了。



再分享一个最近才发现的 Verdi 调用参数。

```
1 verdi -preTitle "some text"
```

螺丝钉做久了，需要同时打开的 Verdi 窗口数量与日俱增。怎么从密密麻麻的任务栏图标中找到要找的那个，是个麻烦但不紧急的问题。除了贯彻断舍离、及时关闭短期内不需要的窗口之外，用上面的选项为每一个 Verdi 窗口做个简洁唯一的命名，也是一个不那么有效的解决之道。

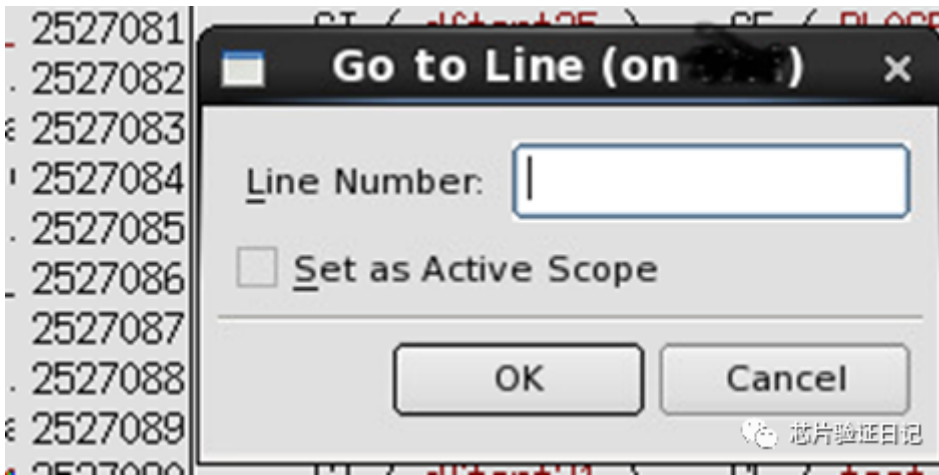
15、跳转到指定行号

其实日常的频繁、快速地查看 Verilog 源代码，一般是在独立的文本编辑器中完成的。比如，VCS 编译产生的 log 文件中，对应的 Warning、Error 的后面会跟着相关源文件的路径、行号。如果用的编辑器是版本比较新的 Vim，那么简单的 gF 指令就可以迅速的从 log 文件跳转到对应源文件，而且光标会体贴的定位在出错行。如果是仍未默认支持的旧版 Vim，那么自己可以添加一个函数，可以参考这篇文章。

这种方式在门级仿真的场景中会不太灵光。因为门级网表通常都非常巨大，用 Vim 类的编辑器打开一次会非常缓慢，频繁跳转造成的等待就更难以接受了。

浏览门级网表诸如此类的超大文件，用 Verdi 是个不错的选择。在 Verdi 的 Source 窗口中，如何快速跳转到指定的行呢？

用鼠标拖动当然是种办法。但更快捷的是，在 Source 窗口下，按 g 键。会弹出一个小窗口，输入行号，点击确定即可。

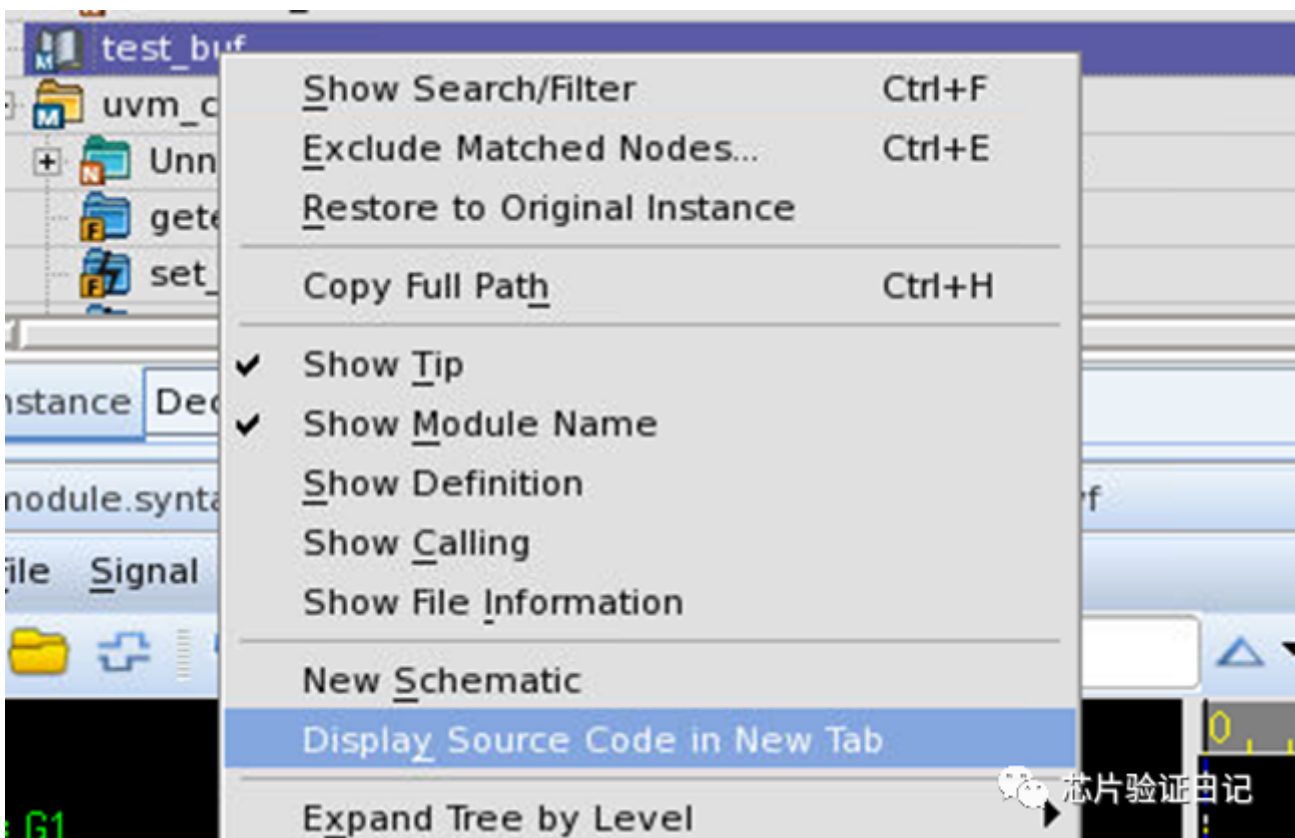


16、源文件之间或不同逻辑层次的切换

有时候在 Verdi 中需要频繁的在两个 Verilog module 之间做切换，做一些对比的工作。如果这两个 module 所处的层次距离比较远，那么快捷的切换就显得特别重要了。这里介绍两个小技巧。

第一个是利用 Verdi 支持多 Source Tab 的功能。在两个不同的 Source Tab 中分别打开不同的 module，通过鼠标切换 Source Tab 来实现源文件或者逻辑层次的切换。

打开新的 Source Tab 的比较方便的方法是，在感兴趣的 Instance 上点击右键，然后在右键菜单上点击 Display Source Code in New Tab.



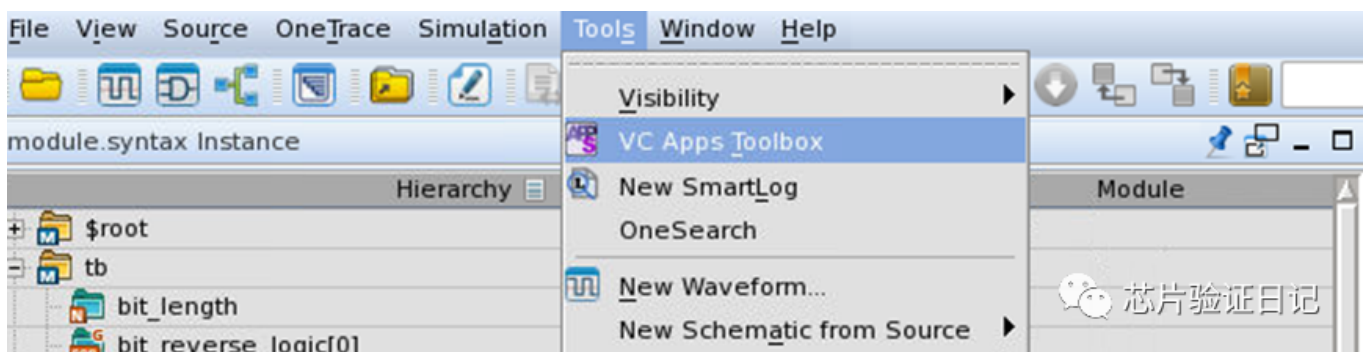
效果类似下图：

```
80    input      H
81 );
82
83    assign Y = A;
84
85 endmodule //test_buf
86
```

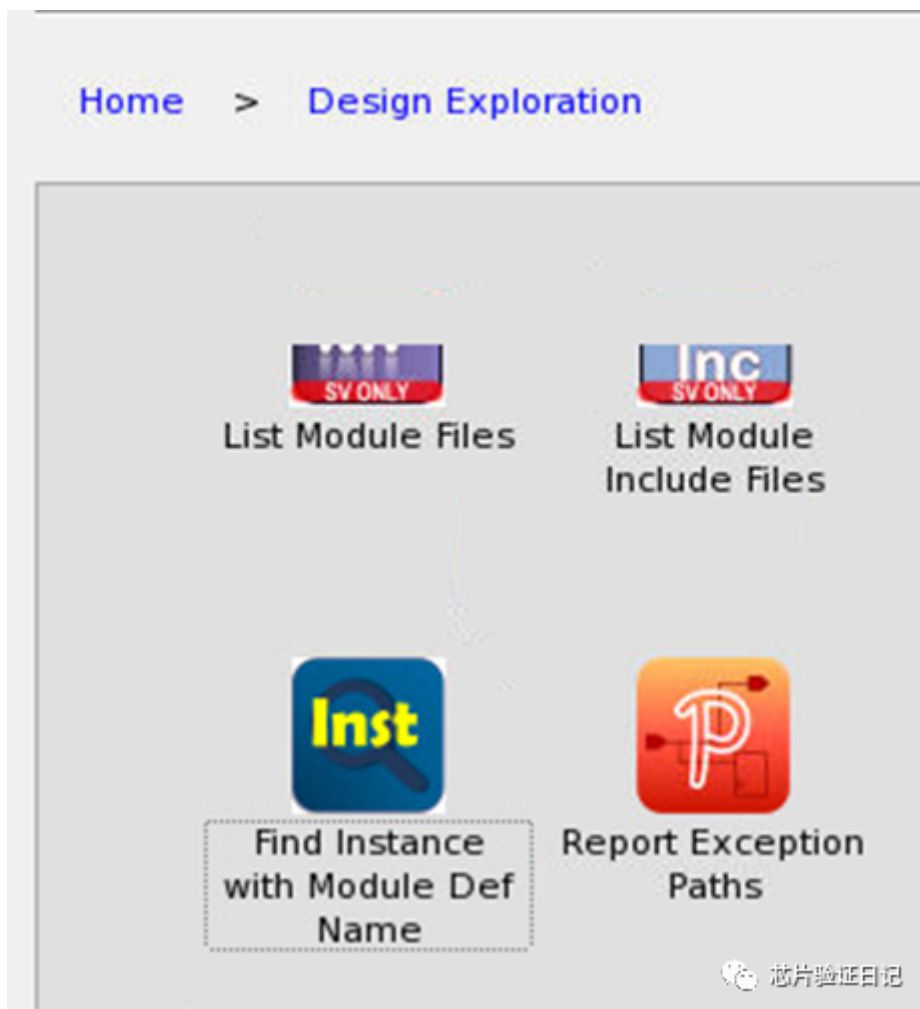
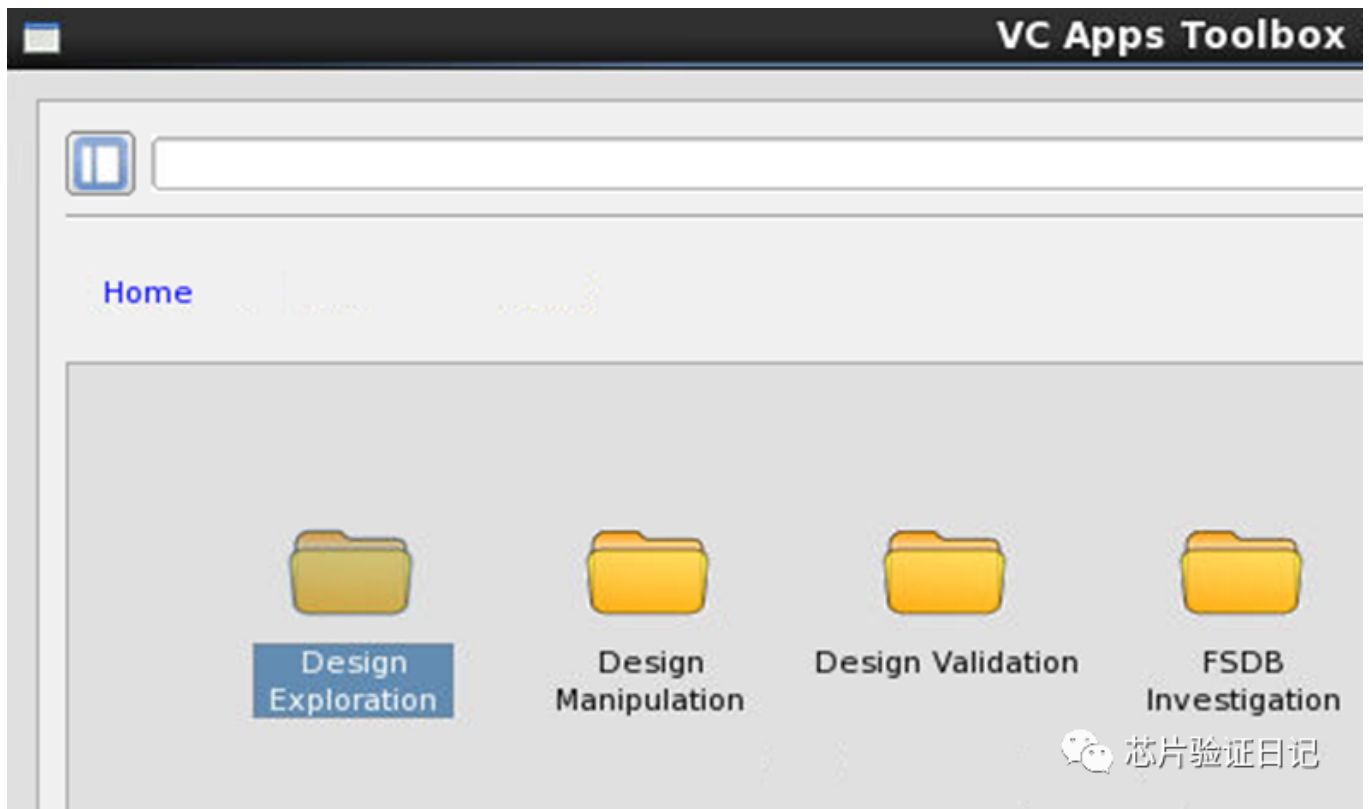


第二个技巧是利用 VC Apps 中的Find Instance with Module Def Name。

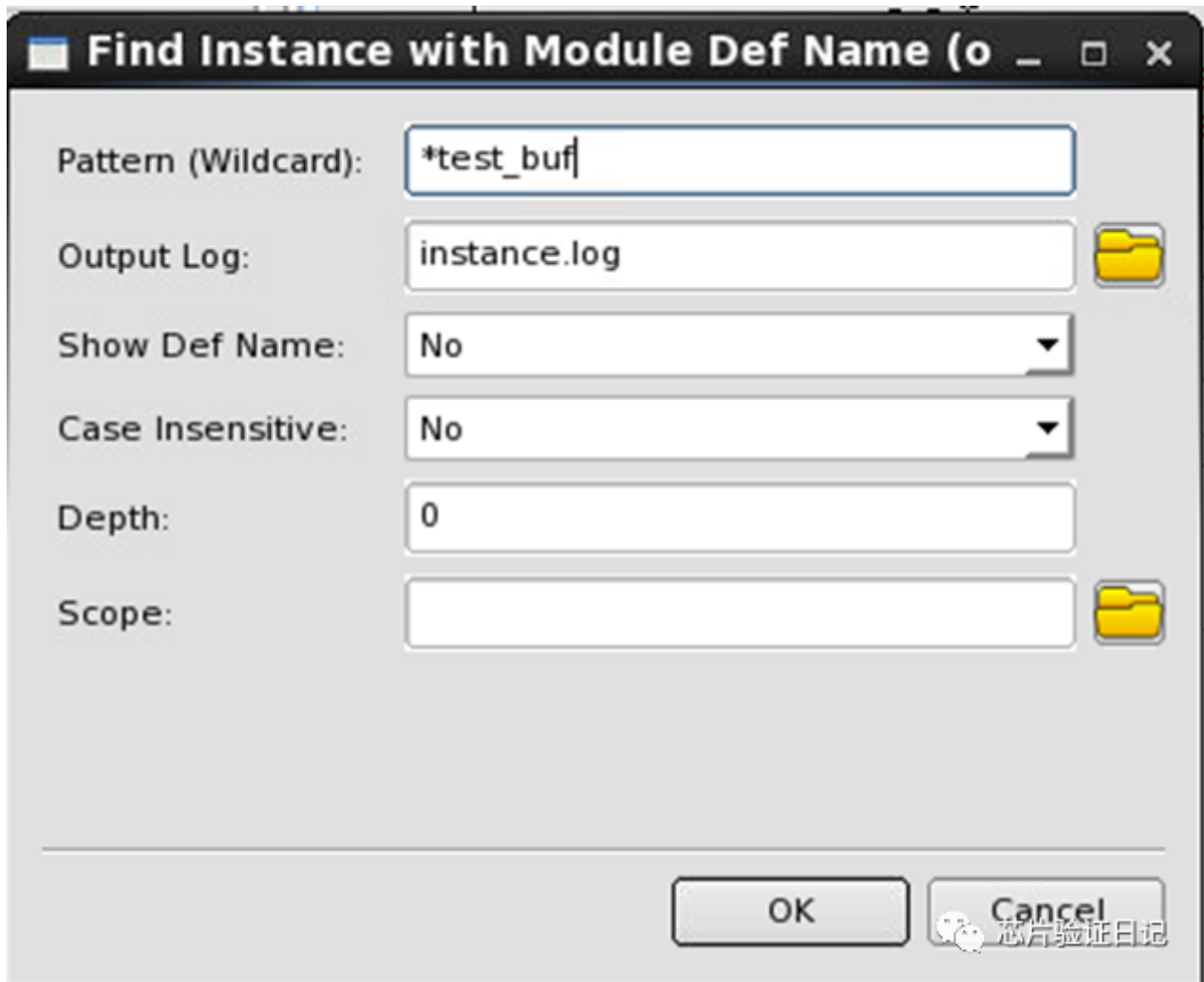
首先打开 Tools 菜单中的 VC Apps Toolbox。



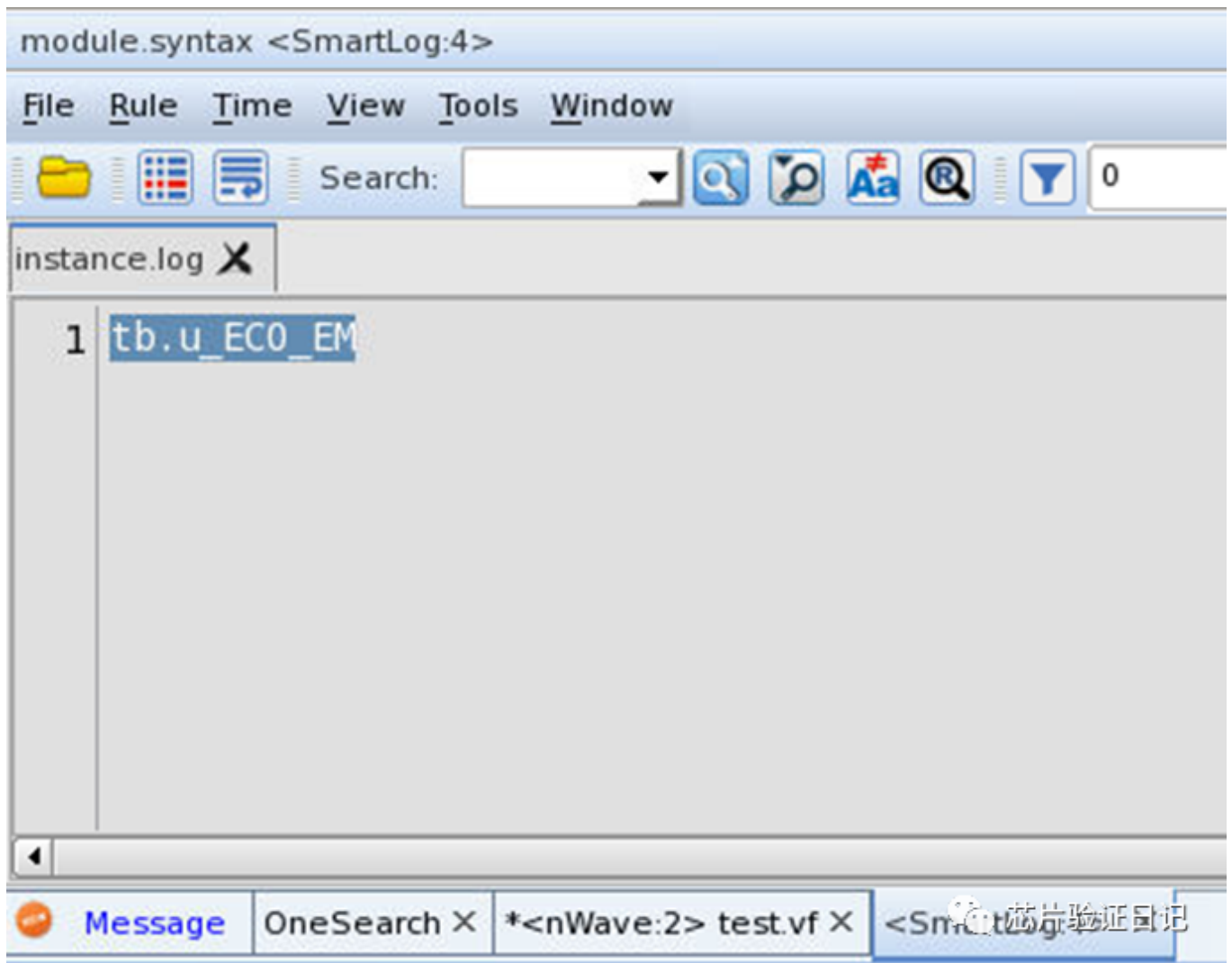
在 Design Exploration 目录中找到上述名字的 App，然后双击执行。



在出现的对话框中输入感兴趣的 module 名字，支持通配符。然后点击确定。



在 Verdi 下部的调试窗口中会出现一个新的大概叫做 SmartLog 的页面，其中列出了按照 module 名字搜索出来的所有 Instance。单击某个 Instance，Source 页面就会跳转到对应的地方。

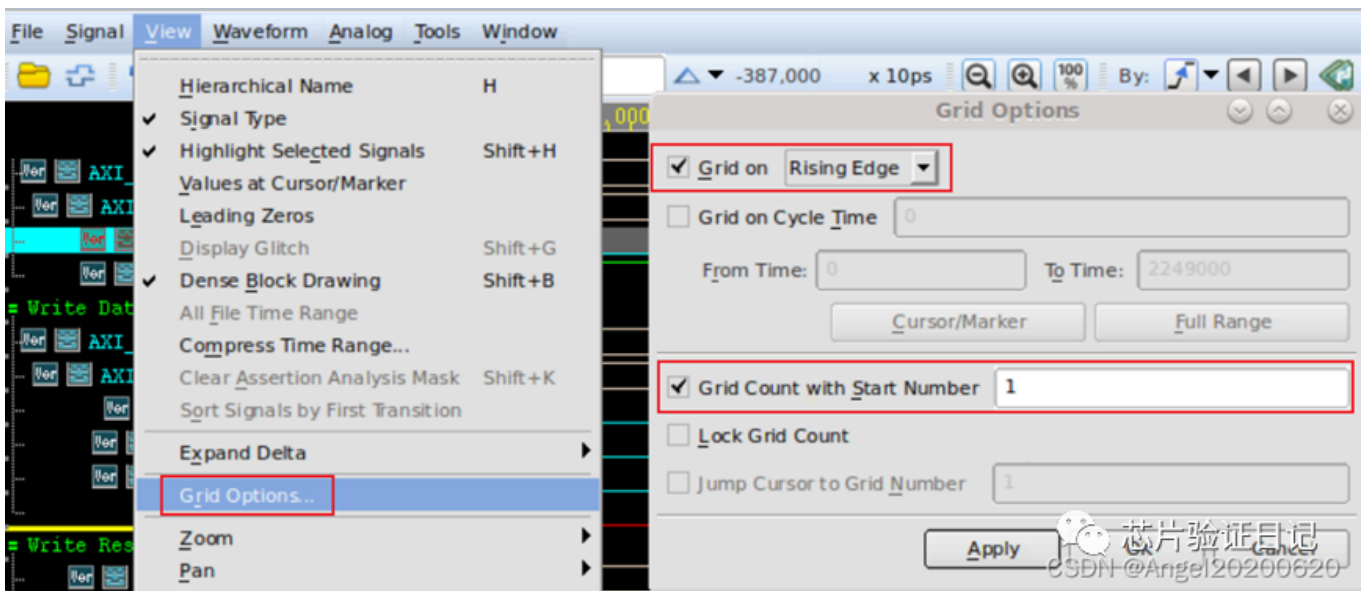


重复执行上述过程，但搜索不同的 module name，会形成独立的 SmartLog 页面。这样通过切换 SmartLog 页面，也可以很方便的在多个设计中切换。

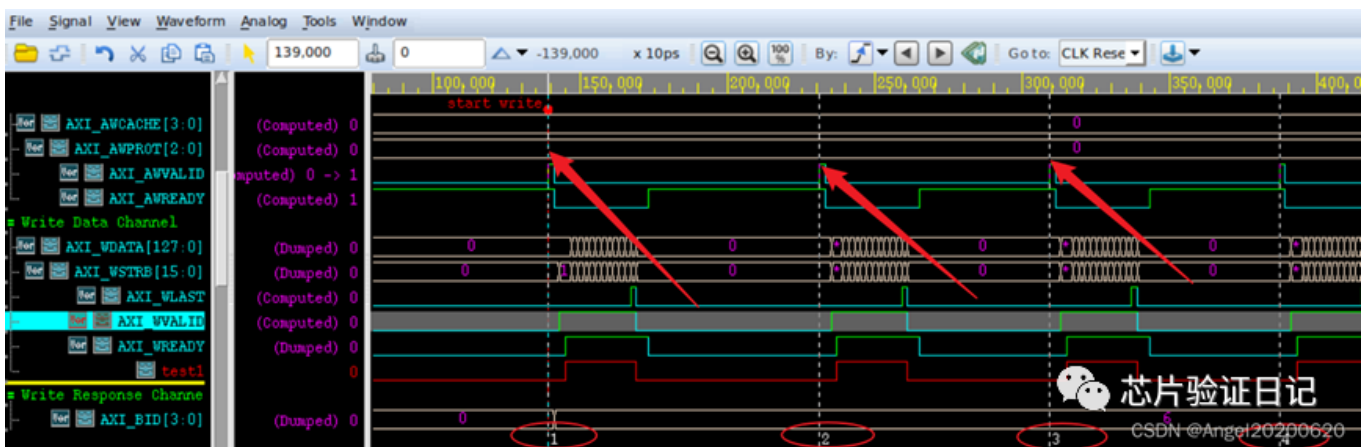
17、计数

[1] Grid Options网格计数

可以指定在波形窗中绘制网格线。选项包括Rising Edge, Falling Edge和 Both Edges。View -> Grid Options 选中Grid on勾选 Grid Count with Start Number，默认是0，也可以按需设置

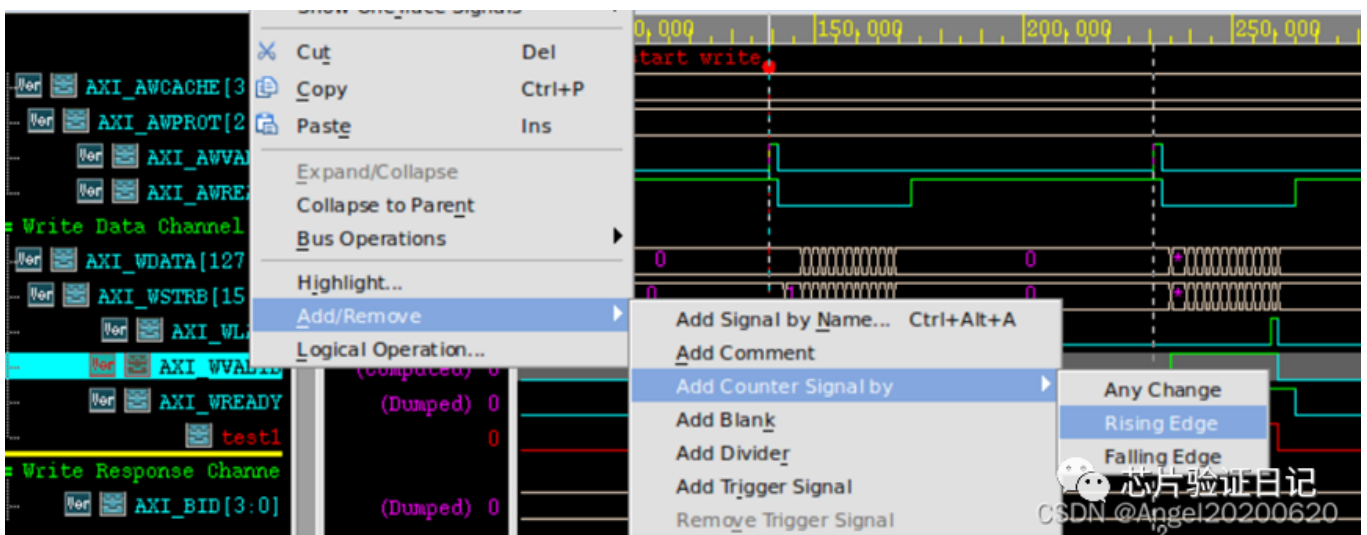


如图，以AXI_AWVALID上升沿为例，网格线下方出现了计数，用于统计上升沿个数。

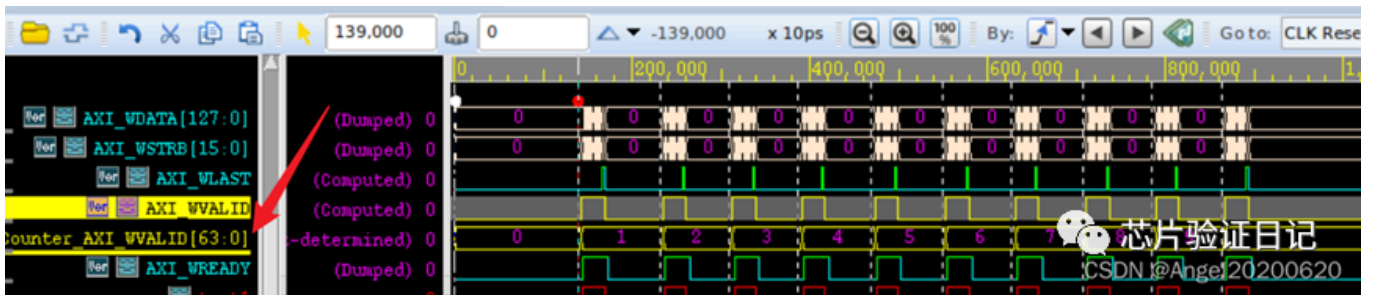


[2] 自动计数

右击AXI_WVALID信号，选择Add Counter Signal by，选择上升沿、下降沿、任意沿中的一个。



选择AXI_WVALID信号的上升沿为例子，波形中自动添加一个counter用于统计。



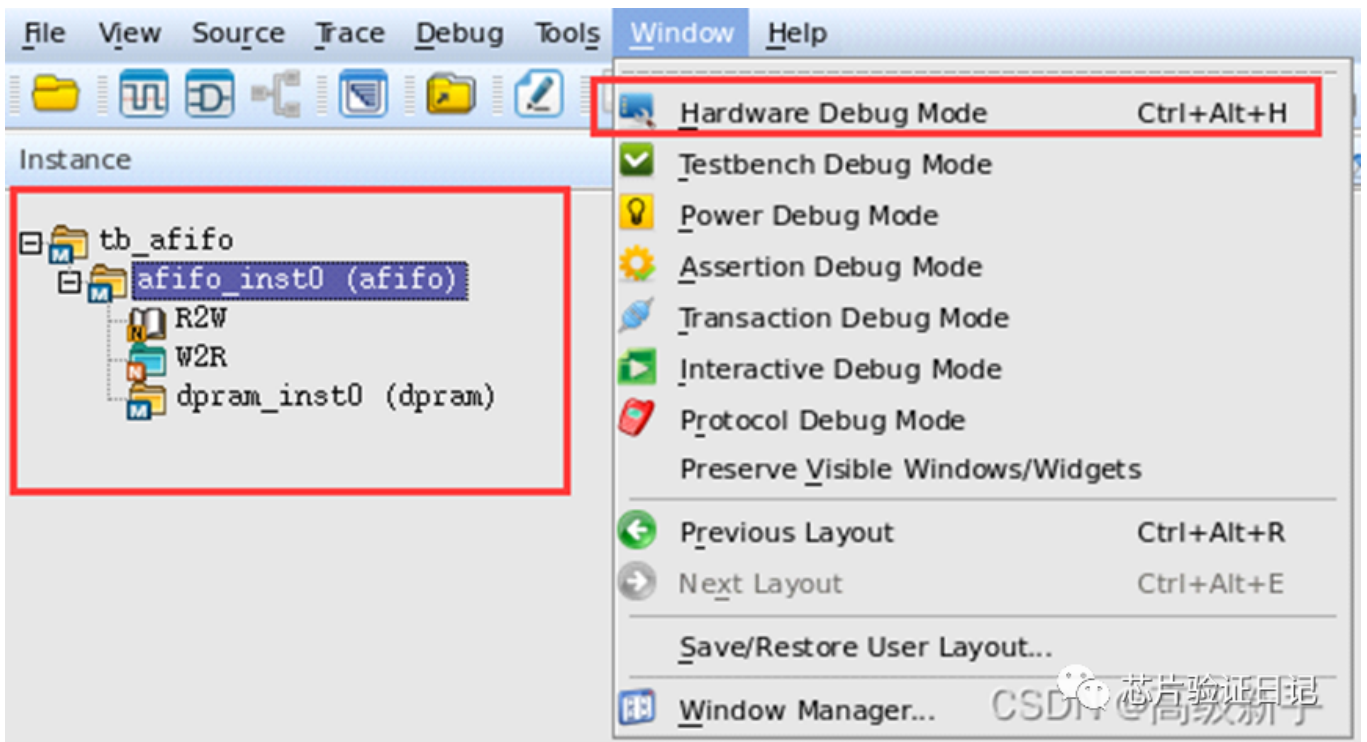
[3] 还有个fsdb转文本的功能：fsdbreport

18 用verdi打开覆盖率文件查看覆盖率

打开一个verdi——>tools——>coverage；在新打开的界面中选择——>file——>open/add database——>选择相应的.vdb文件——>more options——打开cov.hier文件

注：覆盖率文件为.vdb文件；.hier文件为对应工程目录的层级结构文件，对整理覆盖率有帮助

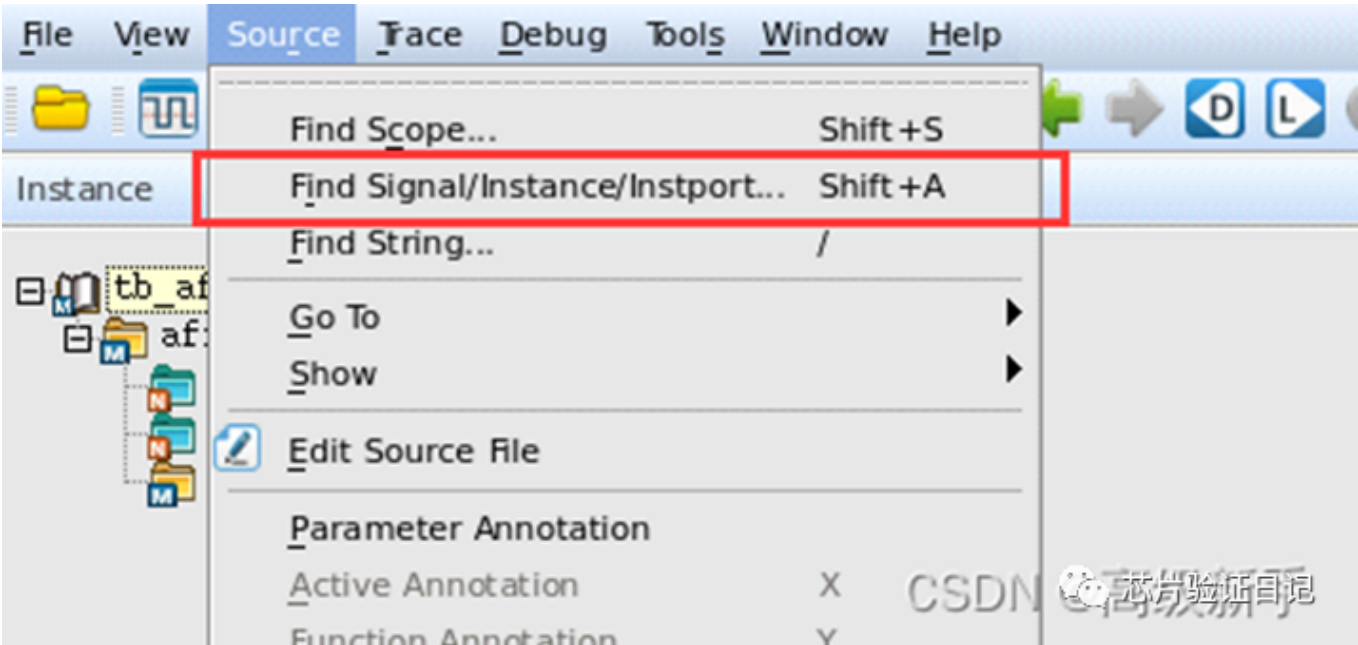
19 查看包含的设计架构



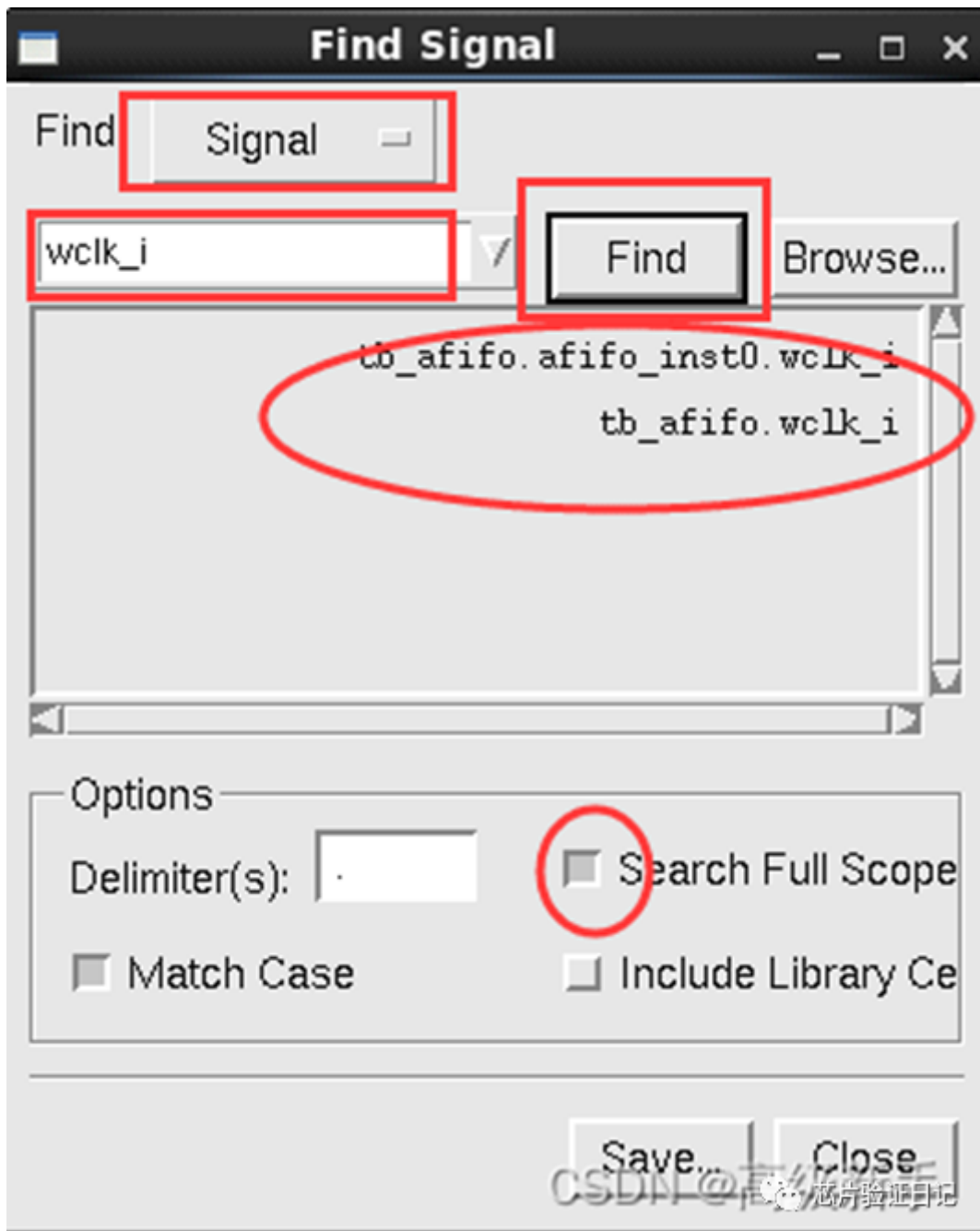
20 查询模块实例化位置

① 在设计结构中双击实例化模块的名字 ->源代码窗口高亮了设计部分的模块名。② 再双击高亮的模块名->看到在上层模块中的哪一行被调用了

21 在nTrace中如何查找模块和信号

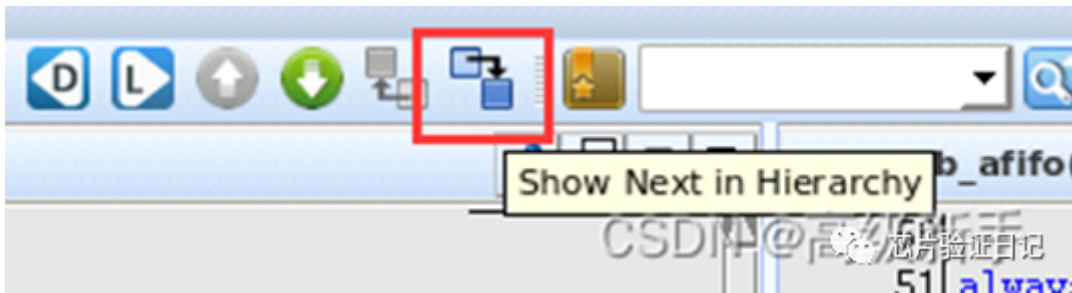


或Shift+A:



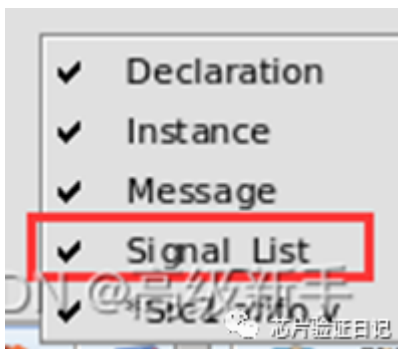
搜索通配符 *

22 查找位于不同层次的驱动信号【跨层次查看信号，看最终驱动谁，或者看最终被谁驱动】



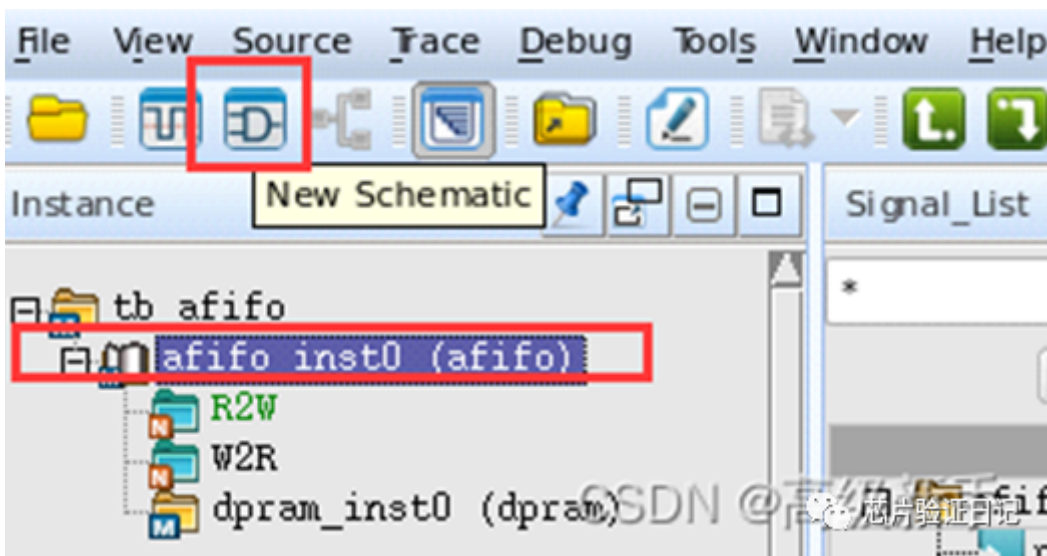
23 如何查看设计有哪些信号

通过最右下方signal_list来查看，可以选择查看输入/输出等端口类型。



24 如何打开原理图

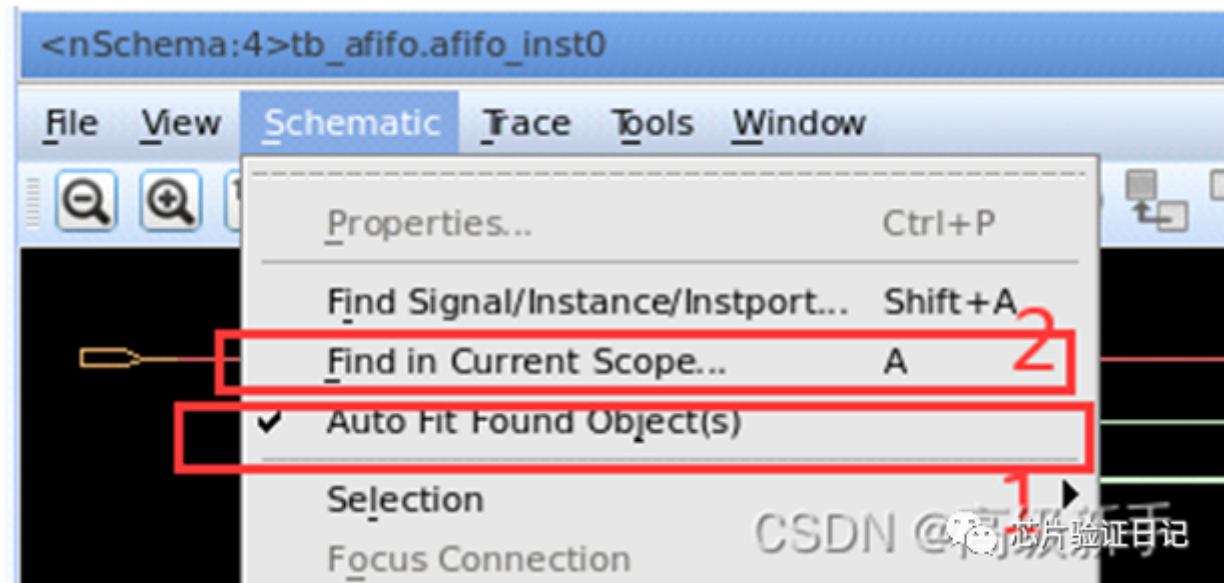
选中设计-> new schematic

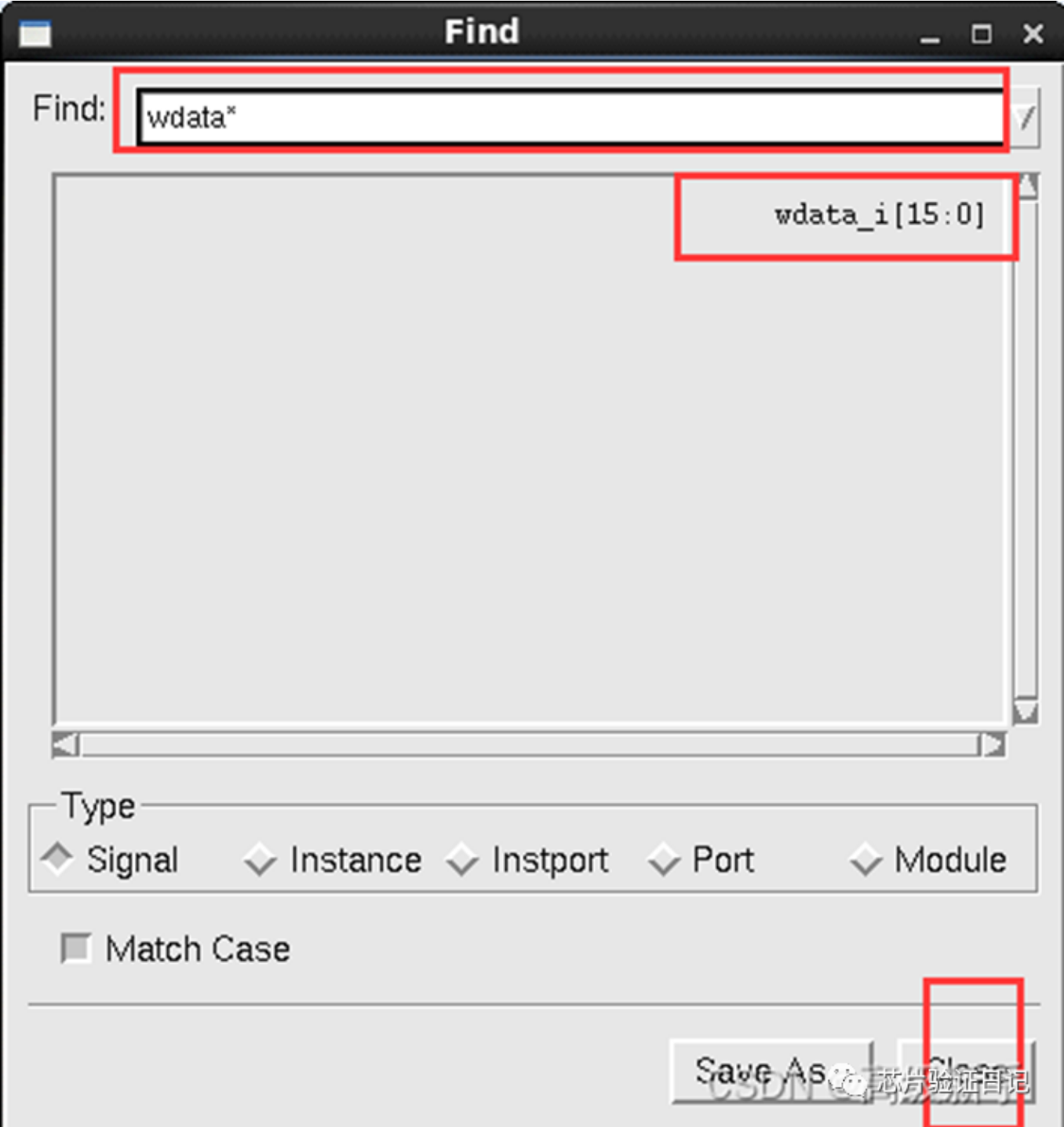


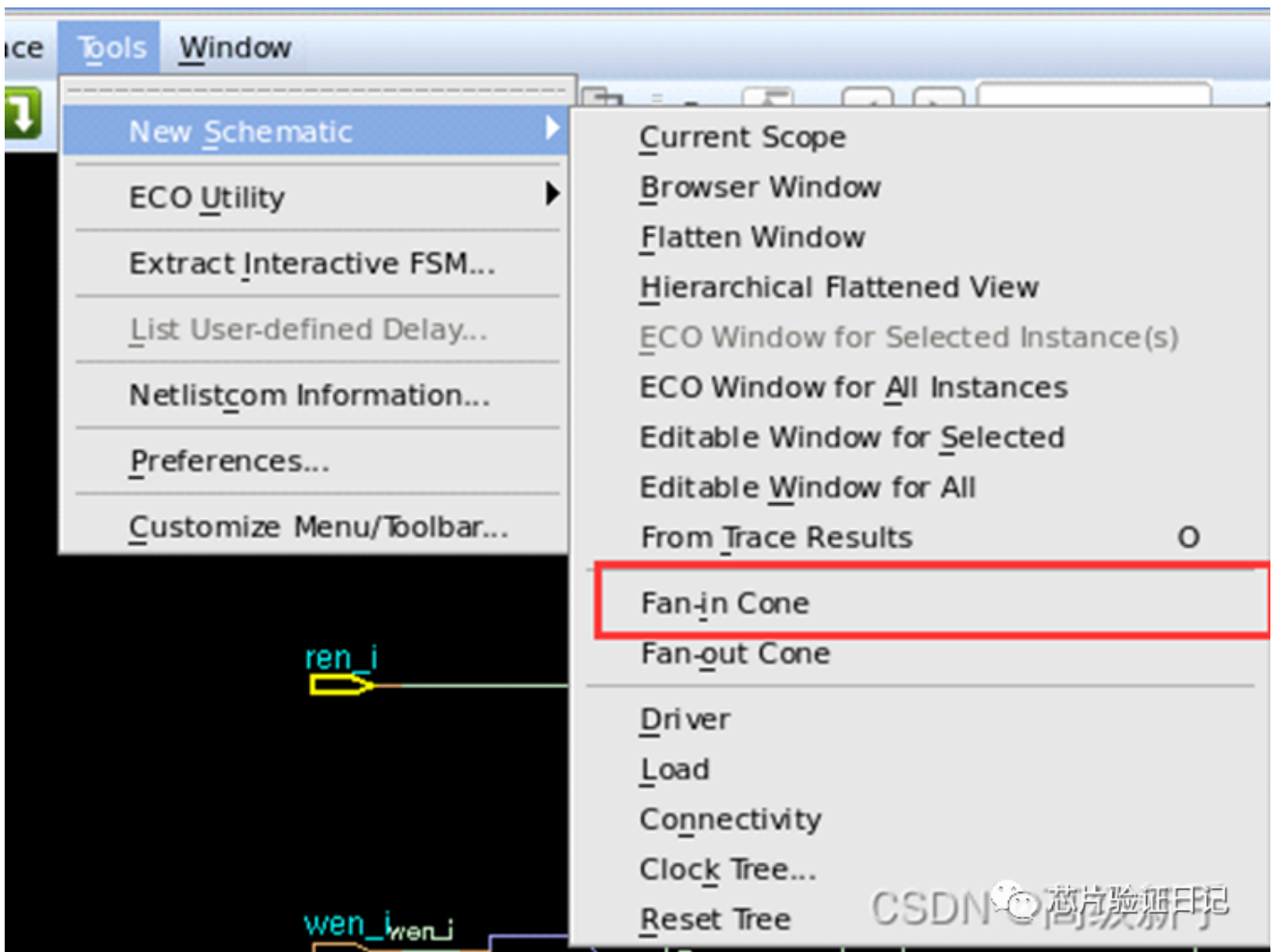
25 如何查看nSchema Window中符号对应的源代码

双击对应的功能单元电路图，即可跳转到描述该单元的源代码

26 使用 Fan-In Cone追踪某个信号的驱动逻辑







首先要用上面的办法来查找信号，高亮选中；调用该工具：

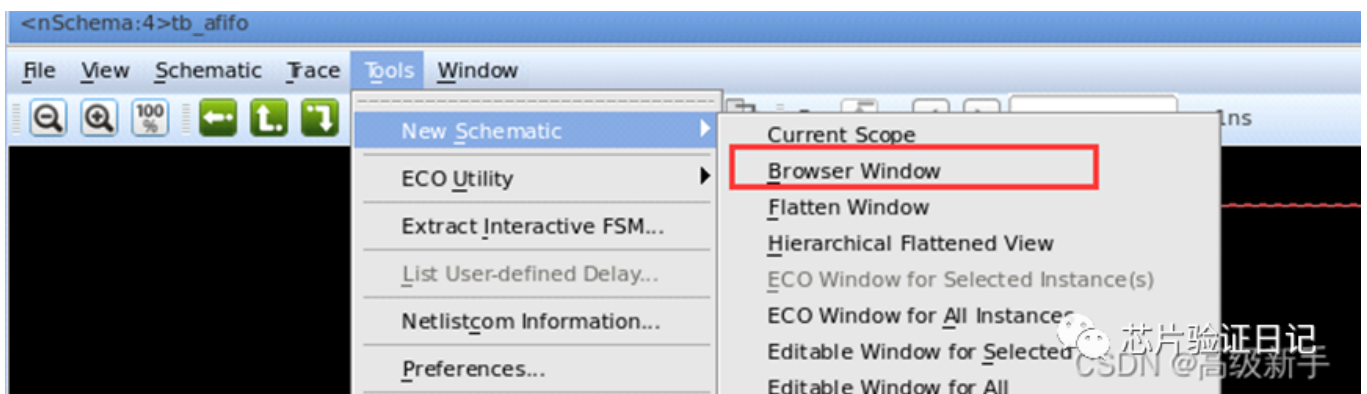
这样就可以看到某个信号是如何生成的（可以通过View选项来显示信号名）

27 如何产生 partial hierarchy schematic

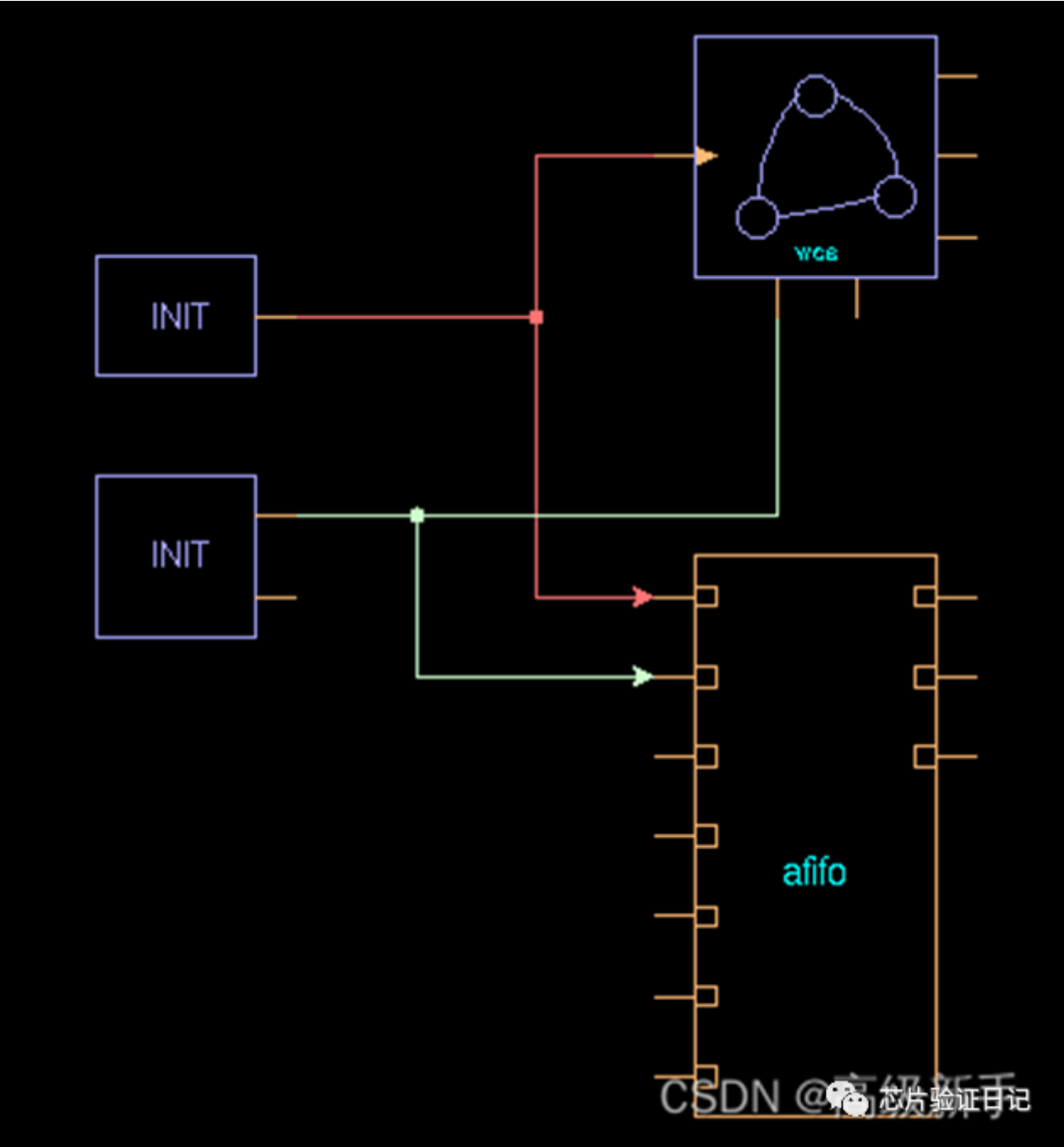
用来查看与选择特定信号有关的逻辑/模块

①选择需要查看的信号（可以通过shift键来选择多个信号）

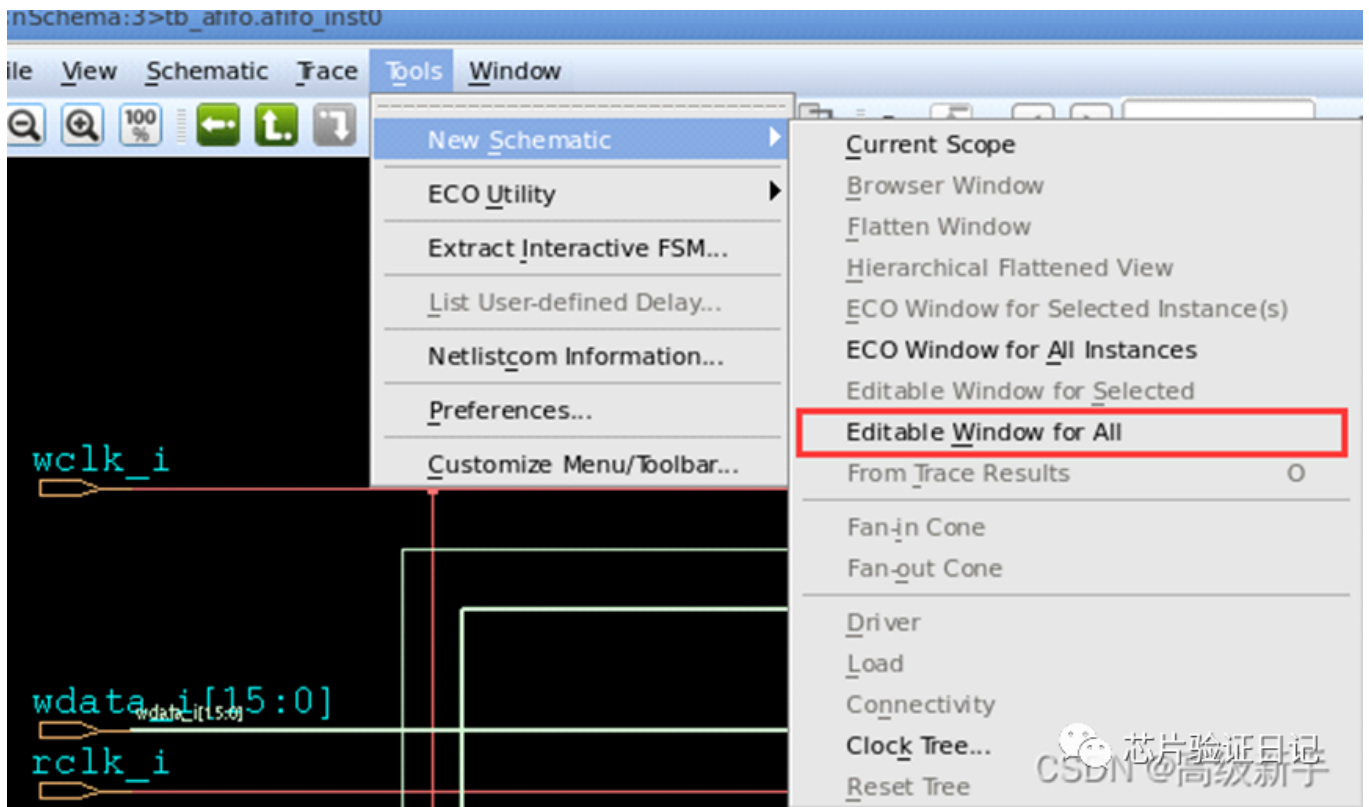
②：



选择查看与wclk和wrst_n有关的信号/模块/逻辑：



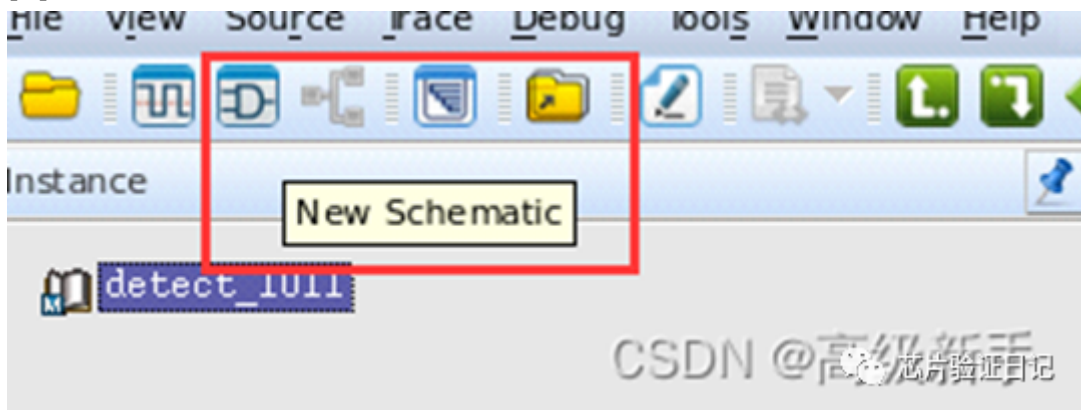
28 如何移动原理图中的器件



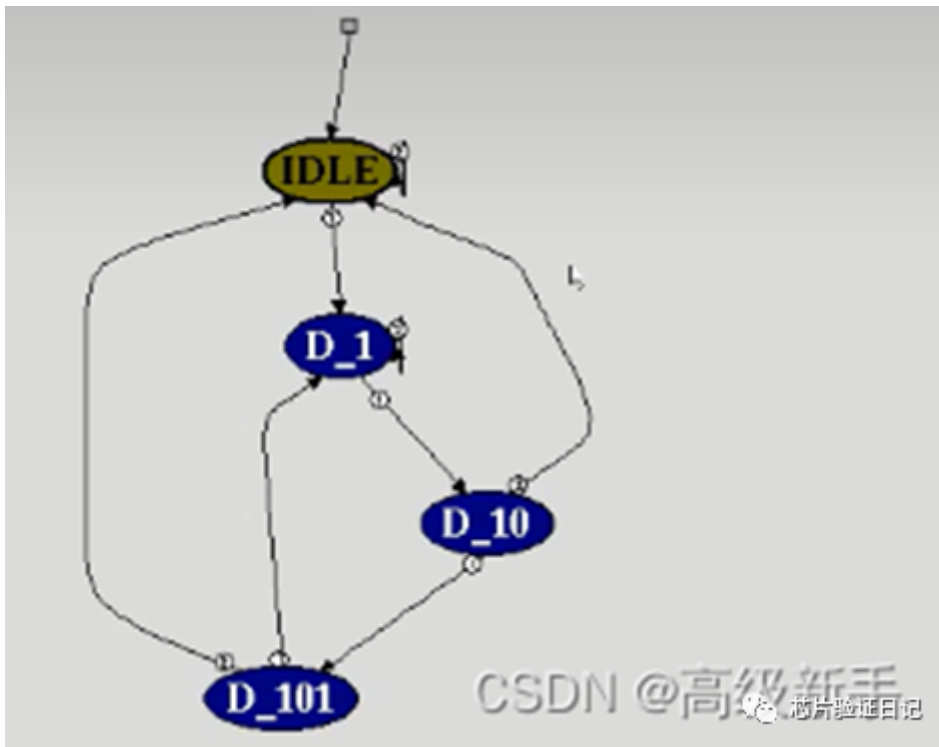
鼠标左键选中，按住右键进行拖拽。

29 状态机相关操作

[1] 打开原理图



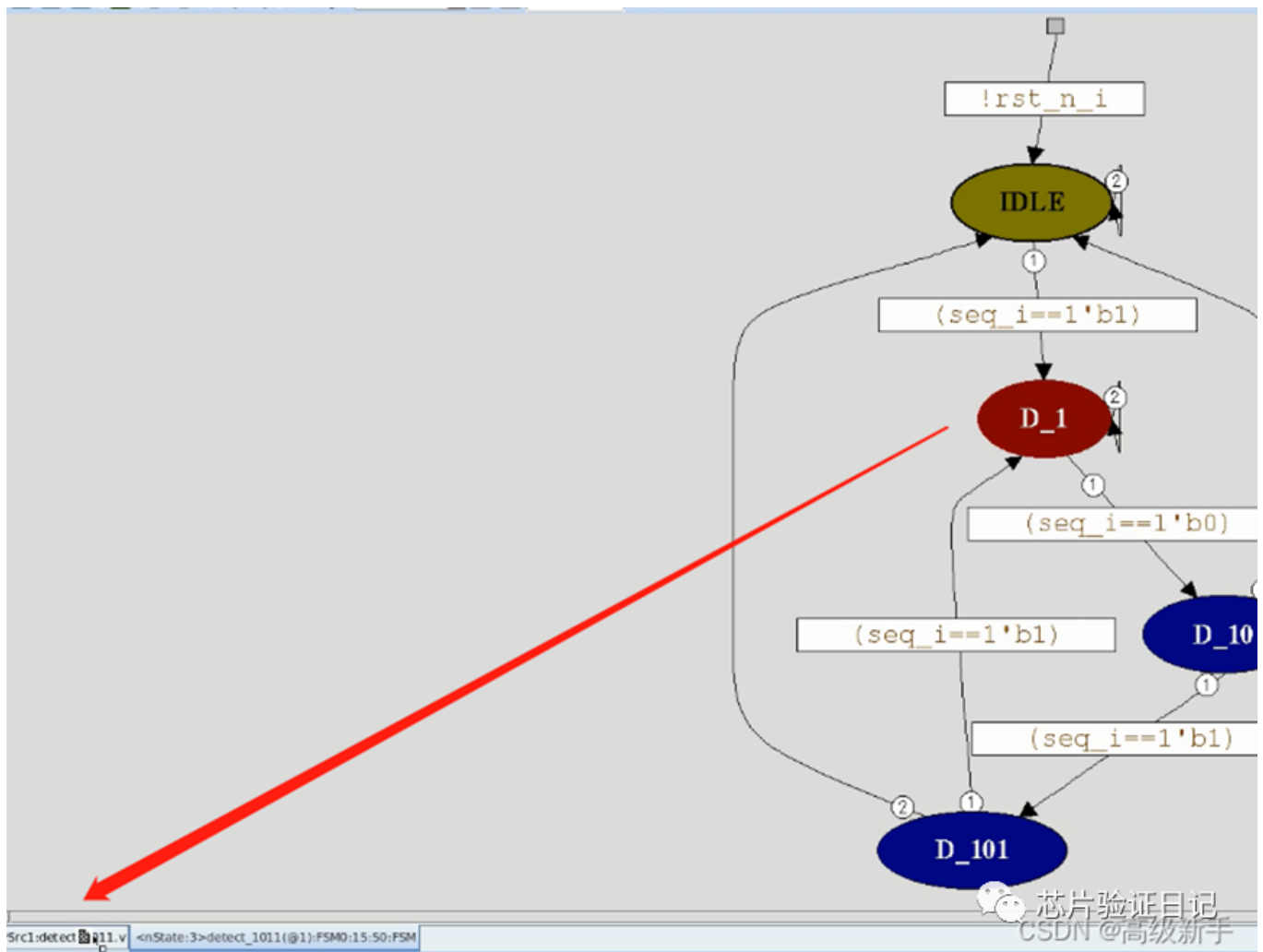
[2] 双击相应状态机视图后，呈现状态机图



[3] 查看状态的执行和跳转

[4] 查看状态以及节点的相关源代码

选择状态，然后使用鼠标中键拖拽状态到nTrace中的源代码视图中即可



[5] 查看在什么时候状态机转移到某个状态? 《与tb有关》

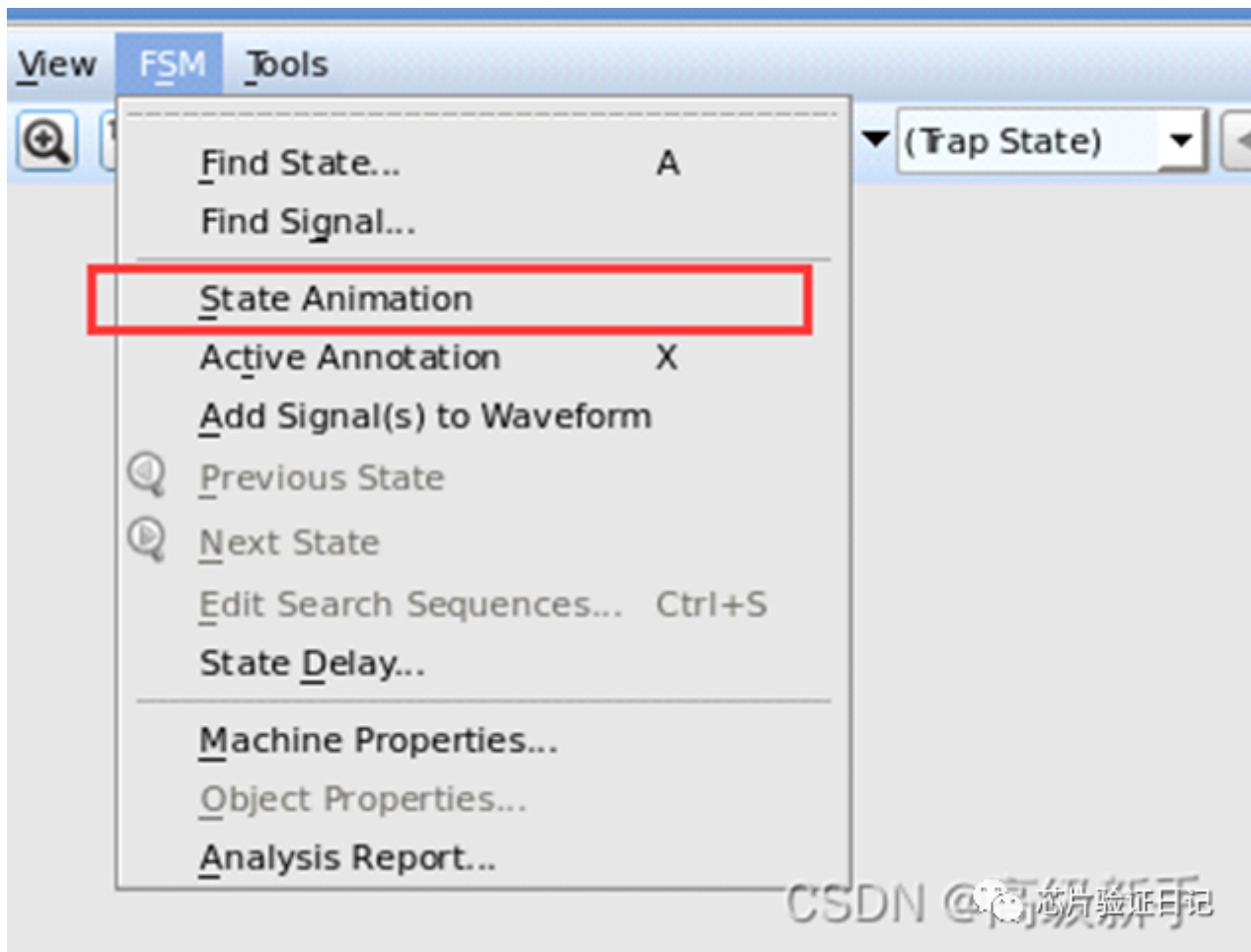
```
*Verdi3* : End of traversing.  
^CInterrupt at time 378859370  
ucli% exit
```

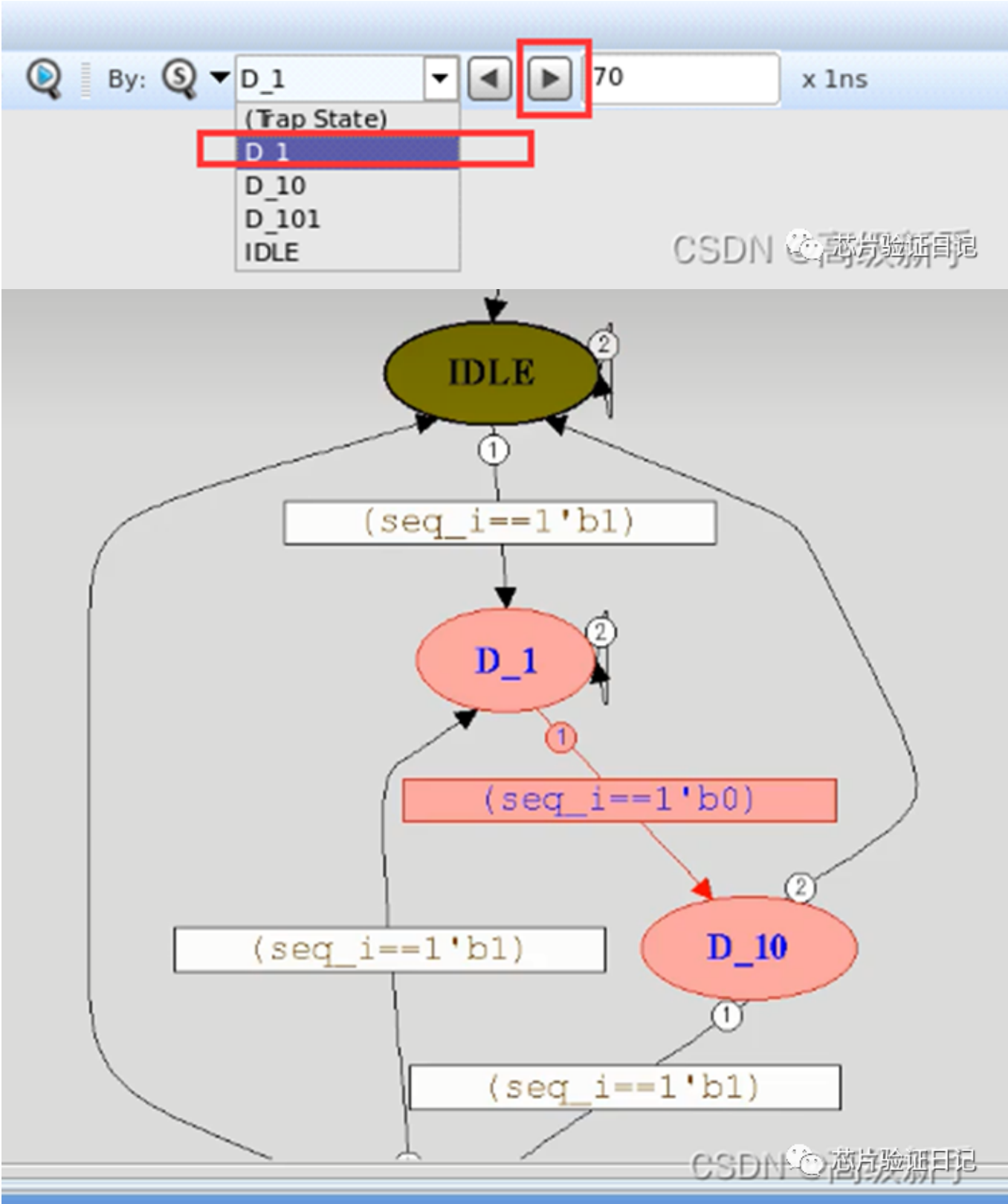
CSDN @高级新手

```
#1000;  
$finish;  
end
```

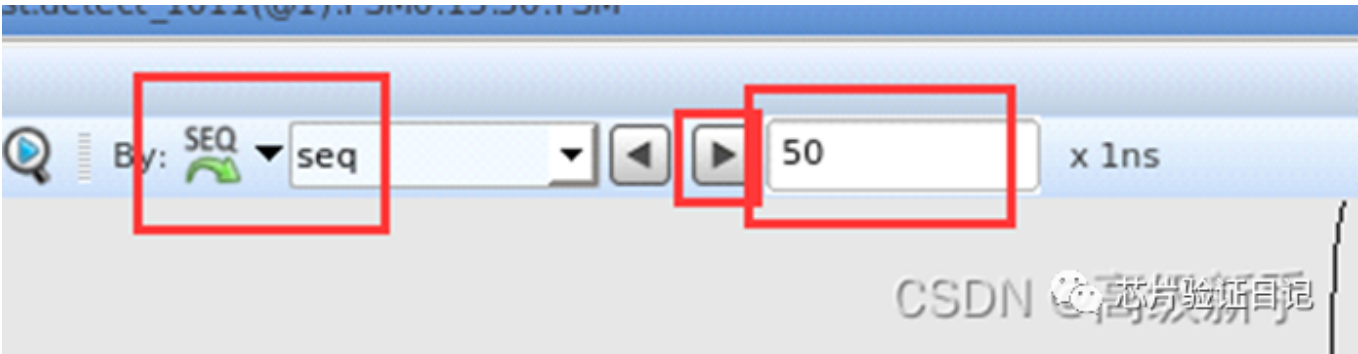
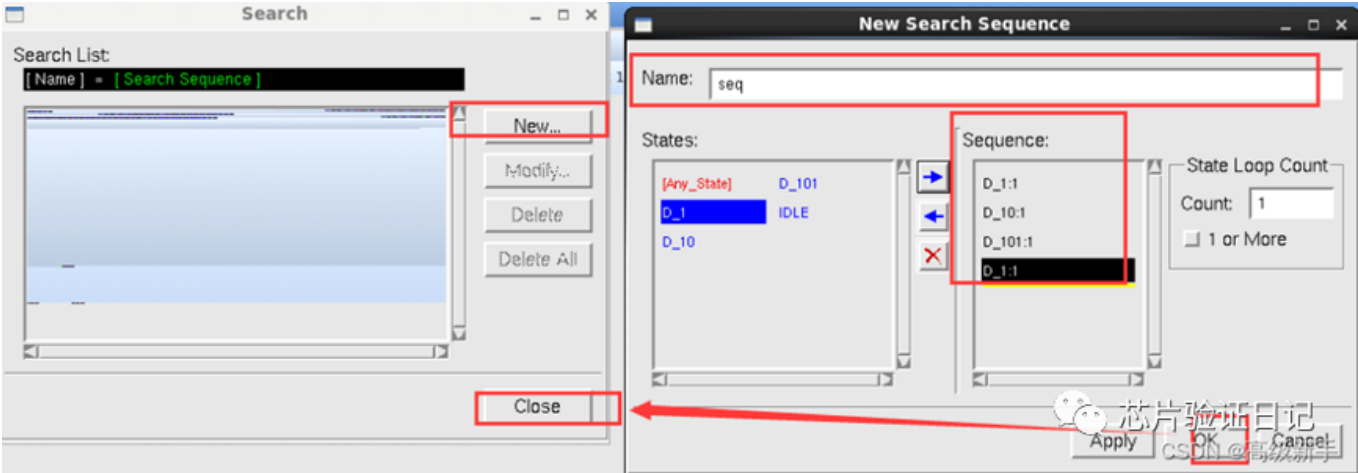
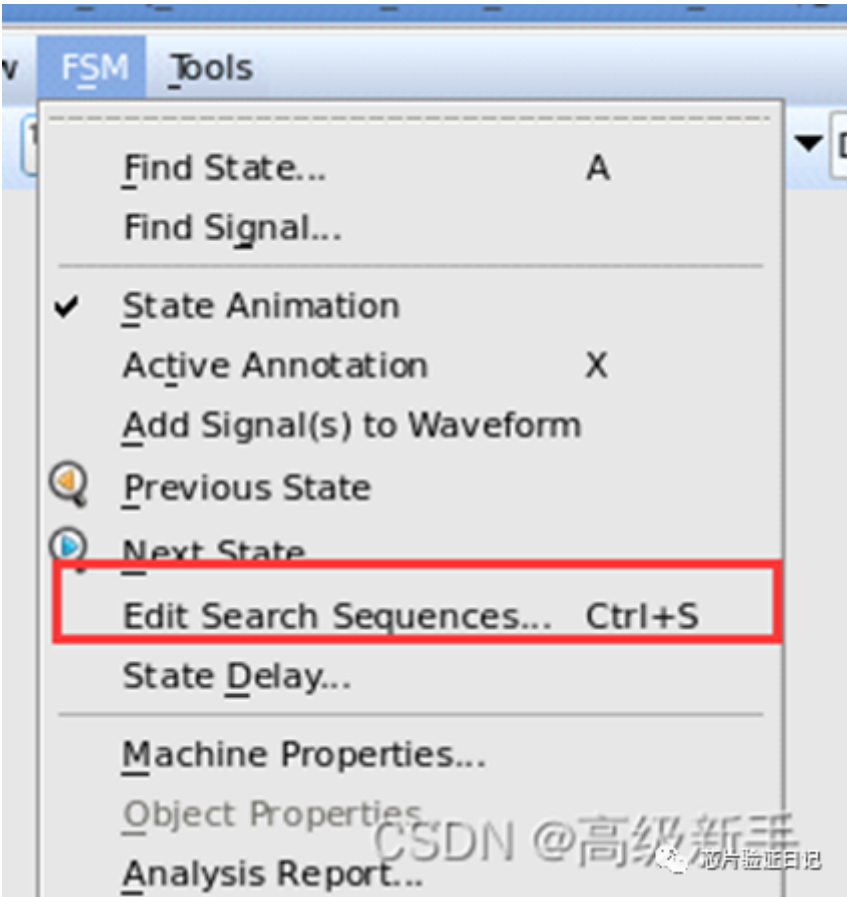
CSDN @高级新手

① 要依靠tb ② 打开nstate后, 要导入波形 ③

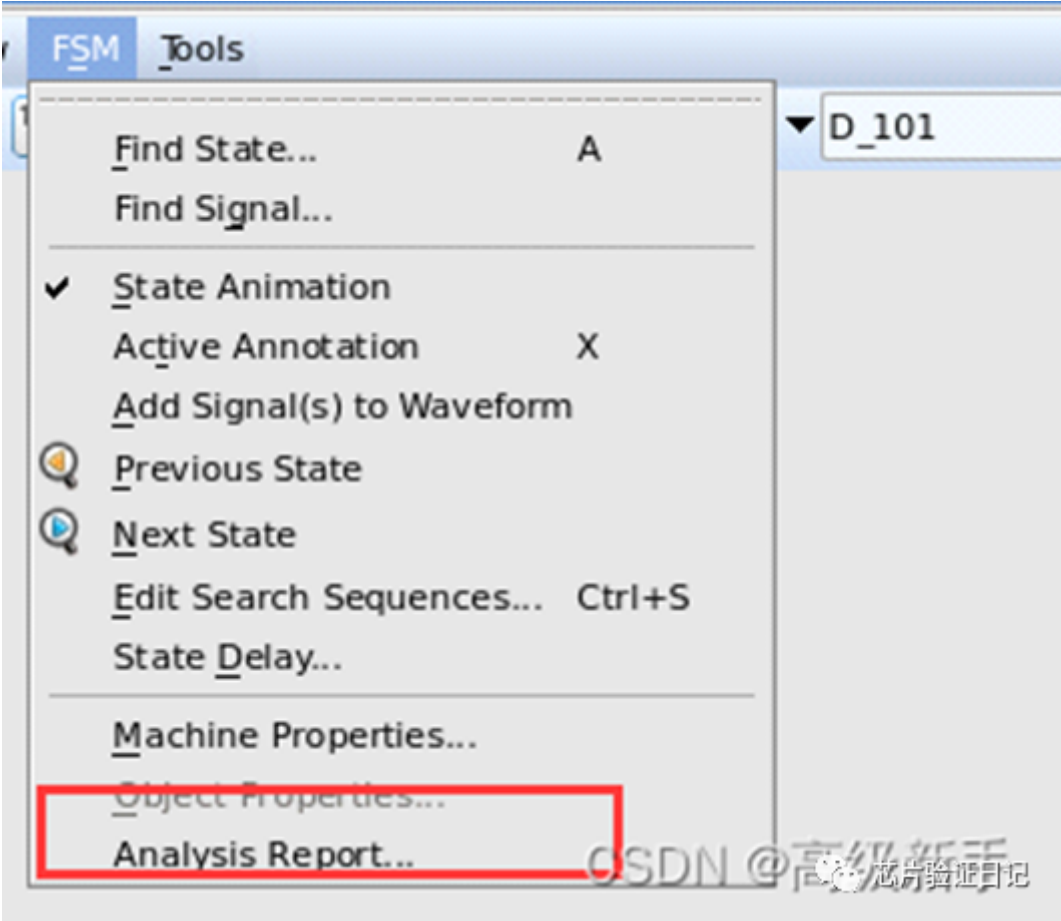


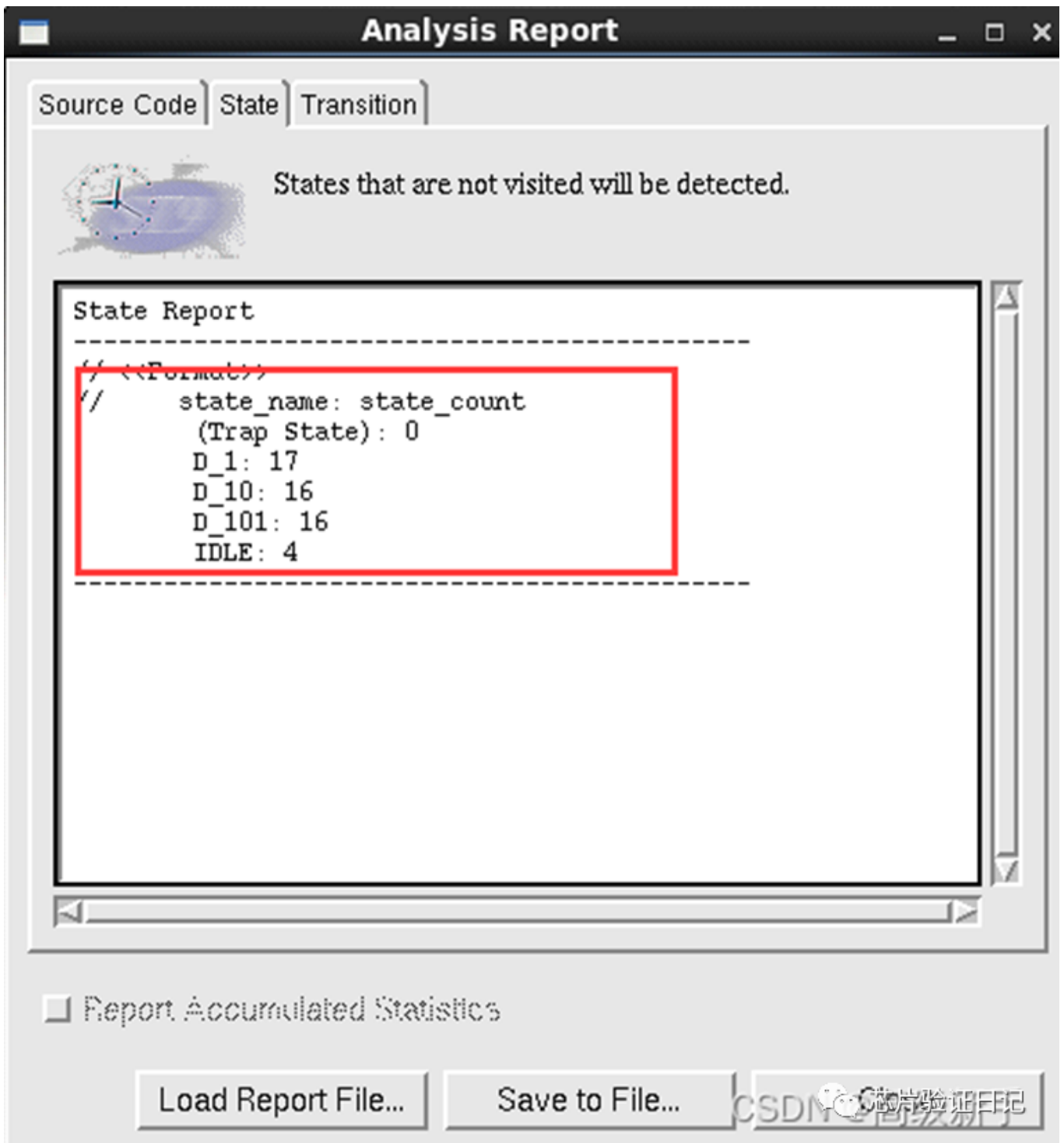


[6] 查看某一种状态转移【状态循环】情况是否会发生

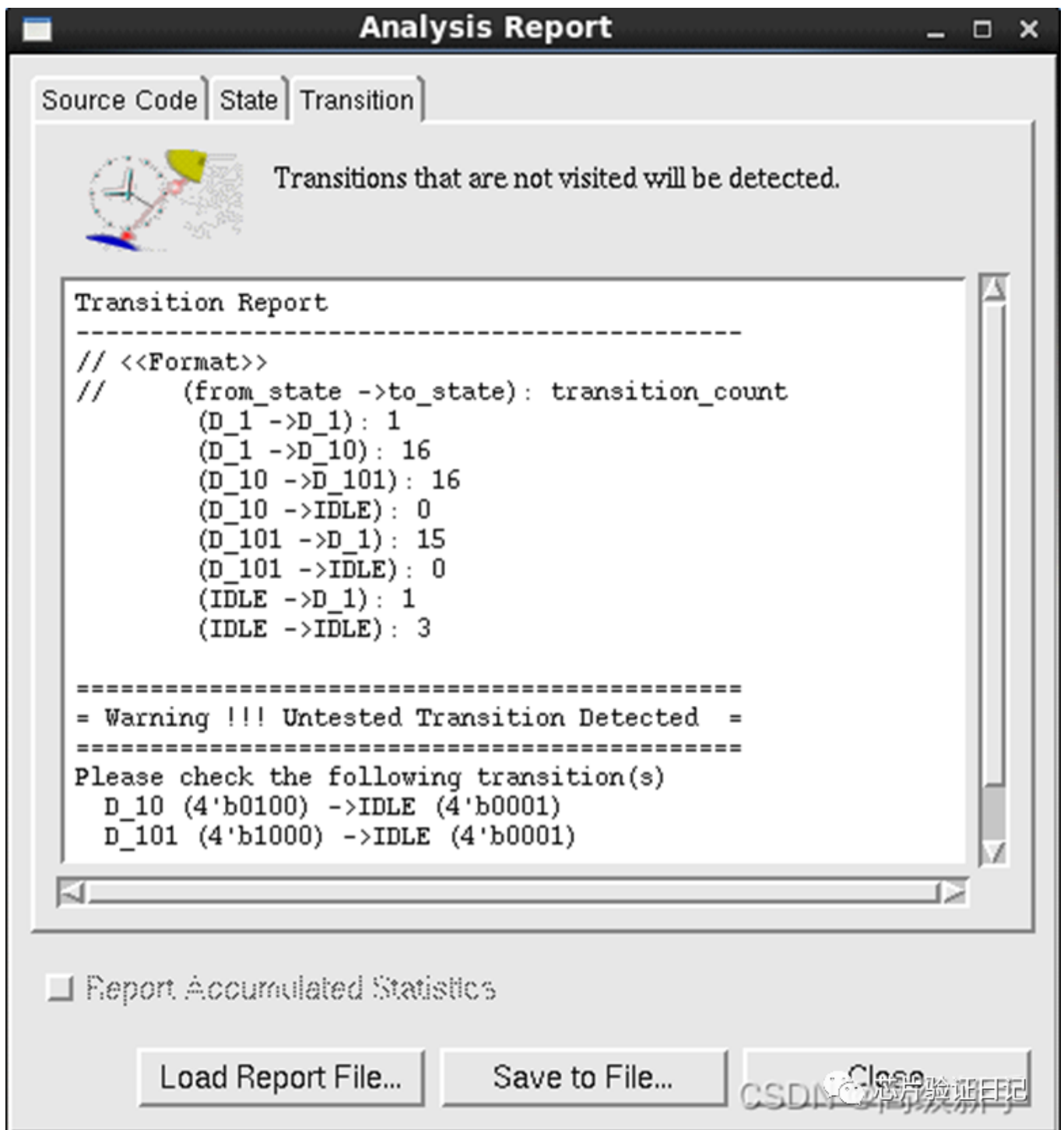


[7] 如何查看某个状态被执行了多少次（与TB有关）





[8] 如何查看某个状态转移到另一个状态的转移次数（与TB有关）



(五) Verd波形(含emulation)加载

1 打开波形

打开常规波形：

```
1 verdi -dbdir ./simv.dair -ssf debug.fldb
```

打开emulation的波形：

```
1 verdi -dbdir ./simv.dair -ssf debug.zwd
```

打开emulation转换前的波形：

```
1 verdi -emulation --zebu-work zebu.work --timescale 1ns --input debug.ztdb
```

打开fsdb波形和编译文件

```
1 verdi -elab com_path -ssf fsdb_path
2 com_path: 编译产生的临时文件存放的路径；例：*.daidir/kdb.elab++;
3 fsdb_path: 编译产生的fsdb的路径；
```

2 打开 source code

如果没有kdb文件，可以通过flist打开source code

```
1 verdi -f tb_top.flist
```

3 加载设计

verdi加载设计的脚本命令

```
verdi -sv -f tb_top_filelist.f -top tb_top &
```

告知软件支持SystemVerilog,加载设计的文件列表，设置设计顶层，后台执行

4 重新加载设计和波形

● 重新加载设计

◆ nTrace->File->Reload Design

● 重新加载波形

◆ nWave->File->Reload

选中波形或设计文件L (shift + i) 重新加载波形或设计文件，在新一次仿真完成之后Reload即可。

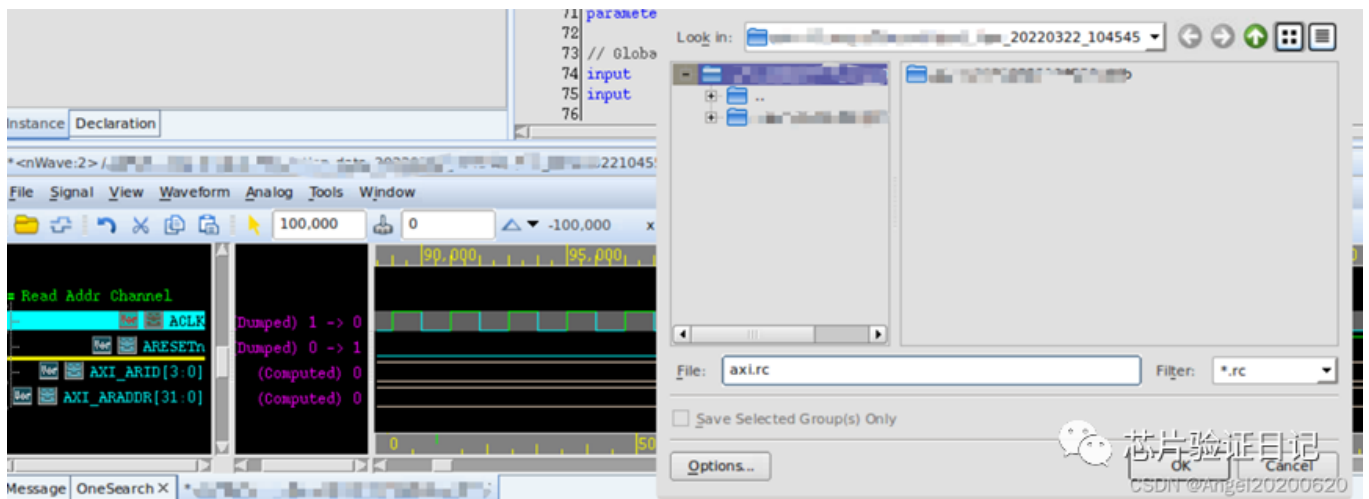
5 波形文件保存*.rc

● Save/Restore Signal

- ◆ 将当前所有信号存成一个*.rc文件
- ◆ 相当于存储了当前查看波形的工程
- ◆ 重新打开波形界面时，Restore信号

CSDN 芯片验证日记

使用verdi保存已经拉出来的信号方便下次使用，按键**shift+S**可以将波形保存为xxx.rc格式文件，使用波形文件时，按快捷键r打开界面，选择波形。可以避免下次打开重新添加信号。



(六) Verdi常用快捷键总结

- 1 **ctrl+w**: 将鼠标指向需要添加到波形上的信号名，通过该快捷键将该信号添加到波形上；
- 2 **shift+l**: 当鼠标指向波形区时刷新波形，当鼠标指向代码区（src1）时刷新代码；
- 3 **z**: 缩小波形，显示更长时间的波形；
- 4 **Z**: 放大波形，显示更短时间内的波形；
- 5 **f**: 显示全局的波形；
- 6 **shift+s**: 将当前存在于到波形区的信号保存为*.rc后缀的信号列表；
- 7 **shift+m**: 波形区快捷添加标记时刻；
- 8 **ctrl+d**: 删除所有信号列表；
- 9 **x**: 在代码区快捷查看所有信号的当前状态；
- 10 在波形区域固定curson（鼠标左键）和marker（鼠标中键）的距离；
- 11 **h**: 在波形区快捷查看信号的全路径；
- 12 **c**: 自定义信号的颜色和粗细；
- 13 **t**: 按一次切换一次信号颜色；
- 14 **m**: 将信号移动到黄线位置；
- 15 **y**: 移至中央并保持居中，再按取消固定居中；
- 16 **n**: 指针跳到信号的下一个跳变沿；
- 17 **N**: 指针跳到信号的上一个跳变沿；

快捷方式

快捷方式	含义
Shift+s	保存rc文件
shift+c	保存rc文件至原始文件
shift+i	刷新波形
c	改变波形颜色手动选择

(七) 赠送跟文相关的一些文档

关注微信公众号《芯片验证日记》，后台回复”verdi_doc”，即可获得下面截图中的文档。

☐	 CummingsSNUG2002Boston_NBAwithDelays_rev1_3.pdf	2023-02-05 17:12	pdf文件	364KB
☐	 10_验证与VCS使用.pdf	2023-02-05 17:10	pdf文件	779KB
☐	 09_E课网_Verdi基础教程.pdf	2023-02-05 17:10	pdf文件	4.74MB
☐	 08_Verdi SVTB Interactive Simulation Debug User Guide (Version 2018.09) .pdf	2023-02-05 17:10	pdf文件	5.78MB
☐	 07_Verdi Transaction Debug Platform User Guide (Version 2016.06) .pdf	2023-02-05 17:10	pdf文件	4.94MB
☐	 06_Verdi Advanced Training AMD Orlando_2011.12_Part Two.pdf	2023-02-05 17:10	pdf文件	5.64MB

参考文章

[1] https://blog.csdn.net/qq_21842097/article/details/116144372

[2]公众号跟IC君一起学集成电路，《Verdi使用技巧》

[3]公众号issoc,《使用verdi的几个小技巧》

[4] https://blog.csdn.net/sinat_43629962/article/details/123664144

[5] https://blog.csdn.net/Arvin_ing/article/details/127885457

相关文章推荐

VCS常用仿真选项开关及步骤总结

分享一份EDA虚拟机环境变量设置（仅供学习参考）

感谢关注微信公众号《芯片验证日记》



芯片验证日记

分享芯片验证相关的知识。

94篇原创内容

公众号

收录于合集 #EDA 17

上一篇

分享一份EDA虚拟机环境变量设置（仅供学习参考）

下一篇

分享一些形式验证(Formal Verification)的经典视频

喜欢此内容的人还喜欢

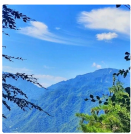
快速理解总线协议

芯片验证日记



AMBA_CHI系列(五)-几个重要的概念

芯片验证日记



USB系列(六)--USB设备的插入检测机制和枚举过程

芯片验证日记

